GovAsset Platform — Complete Presentation Guide

> Read everything below and you will understand your system fully. All answers use simple language — no unnecessary technical words.

Table of Contents

#	Part	What it explains
13	Integration With Other Groups	How they call our API, ngrok guide, WhatsApp message template, FAQ
14	Why Each Ministry Needs a Domain	Why not just use /ministry/1/ in the URL
26	The Full 10-Group Project	Structure, Group 1's role, dependency matrix
	DEPLOYMENT & SETUP	
11	Deployment to Cloud	What you need for production
22	Making the System Live	Production setup, domains, NGINX proxy, X-Forwarded-For
23	Dev Setup: Web + Mobile	How two developers work together
25	Important Commands & Setup	PostgreSQL, start Keycloak, start Django, troubleshoot
	PRESENTATION PREP	
15	Presentation Tips	Common panel questions, demo sequence
16	Quick Reference	Every key file and what it does
18	Glossary	Simple definitions of every technical term
19	Remember This for the Panel	Key sentences to memorize
20	CSS Quick Reference	How to change colors, text size, backgrounds
27	Tonight & Tomorrow Checklist	What to do before the panel
28	Defense Strategies	What If scenarios — crash, freeze, missing features
29	Confidence Tips	Posture, eye contact, voice, recovery

PART 1: DJANGO REQUEST FLOW — What happens from URL to response?

> **This part covers:** The basic 11-step flow.
> For a more detailed walkthrough with code examples, see **Part 18: Complete Request-to-Response Flow**.
>
> **In this part:**
>
> - 11-step request flow
> - Our project's specific flow (Keycloak)

11-step request flow

When someone types a URL or clicks a link, here is the exact path:

```
Browser types: http://moh.localhost:8000/assets/

Step 1: Browser sends the URL to your computer (localhost)
Step 2: Django receives it and looks at the URL path: /assets/
Step 3: django-tenants middleware checks: "What domain is this?" → moh.localhost
Step 4: django-tenants looks up "moh" in the Domain table → finds Ministry of
Health
Step 5: django-tenants switches the database to moh_schema
Step 6: Django matches /assets/ against urlpatterns in urls.py
Step 7: Finds asset_list_view (or AssetListCreateAPIView if API)
Step 8: The view function runs — it queries the database
Step 9: The database only sees moh_schema's tables (data isolation!)
Step 10: The view returns an HTML page or JSON response
Step 11: Django sends the response back to the browser
```

Key point: django-tenants steps 3-5 happen on EVERY request, BEFORE your view code runs. This is why `TenantMainMiddleware` must be the first middleware in the list.

Our project's specific flow:

```
User → Login Page → Clicks "Sign in with Government SSO"
  → Django redirects to Keycloak login page (separate website)
  → User types username/password on Keycloak
  → Keycloak redirects back to Django with a code
  → Django exchanges code for tokens (server talks to server)
  → Django verifies the token signature (proves it's really Keycloak)
  → Django finds or creates the user in its database
  → User is redirected to /dashboard/
```

PART 2: MULTI-TENANCY — What does it mean?

> **In this part:**
>
> - What multi-tenancy means
> - Wrong approach (ministry_id column)
> - Our approach (PostgreSQL schemas)

What multi-tenancy means

Multi-tenancy means one system serves many customers, and each customer's data is completely separate.

Think of an apartment building:

- The building = one Django application
- Each apartment = one ministry's data
- Each apartment has its own locked door (PostgreSQL schema)
- A person in apartment 3 cannot walk into apartment 5

Our system: Instead of apartments, we have ministries (MOH, MOF, etc.). Each ministry gets their own locked room in the database. Ministry of Health staff ONLY see Health data. Ministry of Finance staff ONLY see Finance data.

NOT multi-tenant (wrong approach):

```
One big table with a "ministry_id" column
  → Risk: A bug might accidentally show Finance data to Health users
  → Risk: A programmer forgets to filter by ministry_id
```

OUR approach (correct):

```
Separate PostgreSQL schemas
  → Each ministry has its own table called assets_asset
  → If a query doesn't filter by ministry, it STILL only sees one schema
  → Data isolation is ENFORCED BY THE DATABASE, not by our code
```

PART 3: django-tenants — How exactly does it work?

> **In this part:**
>
> - How django-tenants works (3 key mechanisms)
> - Evidence in our code

How django-tenants works

django-tenants is a library that:

1. **Changes the database engine** — Instead of the normal PostgreSQL driver, it uses `django_tenants.postgresql_backend`. This custom driver adds `SET search_path = moh_schema, public;` before EVERY query.
2. **Identifies the tenant from the URL** — `TenantMainMiddleware` reads the domain (e.g., `moh.localhost`), looks it up in the `tenants_domain` table, finds which Ministry it belongs to, and sets the schema name.
3. **Creates schemas automatically** — When a Super Admin creates a new Ministry, django-tenants runs `CREATE SCHEMA moh_schema` in PostgreSQL and creates all the tables inside it (assets_asset, organizations_orgunit, etc.).

Evidence in our code:

settings.py:

```
# Custom PostgreSQL backend (line 121)
DATABASES = {
```

```
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend',  # ← This is key
    }
}

# Router tells Django to use this backend for all apps (line 130)
DATABASE_ROUTERS = ('django_tenants.routers.TenantSyncRouter',)

# Which apps are shared (one copy, public schema)
SHARED_APPS = ['django_tenants', 'tenants', 'authentication', ...]

# Which apps are per-tenant (separate copy per ministry)
TENANT_APPS = ['organizations', 'assets']
```

tenants/models.py:

```
class Ministry(TenantMixin):
    # TenantMixin gives us schema_name field automatically
    auto_create_schema = True  # When saved, PostgreSQL creates a new schema
```

Every view that queries tenant data:

```
from django_tenants.utils import schema_context

# Switch to the ministry's private schema
with schema_context(user.ministry_schema):
    # This query ONLY sees that ministry's data
    assets = Asset.objects.all()
```

PART 4: KEYCLOAK SSO — The full login flow (simple version)

> **This part is long. Jump to any topic:**
>
>

> - Admin user management: is_locked vs is_active
> - Two-way sync between Django and Keycloak

Keycloak is a separate program that handles logging in. Django does NOT handle passwords — Keycloak does.

The flow with 3 characters:

```
Character 1: THE USER (opens browser)
Character 2: DJANGO (our app)
Character 3: KEYCLOAK (the identity server)

Step 1: User visits http://moh.localhost:8000/login/
Step 2: Django shows the login page with a "Sign in with Government SSO" button
Step 3: User clicks the button → browser goes to /oidc/authenticate/
Step 4: Django says "I don't handle passwords. Go to Keycloak."
        Redirects to: http://localhost:8180/realms/govasset/protocol/openid-
connect/auth
Step 5: Keycloak shows its own login page (looks different from Django)
Step 6: User types username and password
Step 7: Keycloak checks: "Is this password correct?" If yes, continues
Step 8: Keycloak creates a temporary CODE and sends the browser back to Django
        URL: /oidc/callback/?code=xyz123
Step 9: Django takes the code and calls Keycloak secretly (server-to-server):
        "Hey Keycloak, here's the code. Give me the real tokens."
Step 10: Keycloak verifies the code and sends back: access_token + id_token
Step 11: Django checks the id_token's SIGNATURE (like checking a passport seal)
         - It fetches Keycloak's public key
         - Verifies the signature matches (proves Keycloak signed it)
         - Checks expiration date
Step 12: If everything checks out, Django looks up the user in its database
Step 13: If user exists → login successful → redirect to /dashboard/
Step 14: If user doesn't exist → create PendingAccess → show error message
```

How Django knows the user is real (not a hacker):

Three checks, like a passport check at the airport:

```
Check 1 — THE SIGNATURE (cryptographic):
  Keycloak signs the token with its PRIVATE key (like a government stamp)
  Django verifies with Keycloak's PUBLIC key (like checking the stamp is real)
  If someone forged a token, the signature won't match → REJECTED

Check 2 — THE PASSPORT DETAILS (claims):
  Django checks:
  - Is the token expired? (exp field)
  - Was this token meant for OUR app? (aud field)
  - Did Keycloak really issue this? (iss field)
  If any fails → REJECTED
```

```
Check 3 — THE DATABASE LOOKUP (application):
  Django looks in its own database: "Is this user registered?"
  If no matching user found → REJECTED (shows pending access message)
```

What is the real purpose of Keycloak? (Why do we have it at all?)

This is the most important question. Here is the answer:

Keycloak exists so that one username + password works across MULTIPLE government systems, not just ours.

Imagine the government has 20 different systems:

- Our asset management system
- A human resources system
- A budget system
- A procurement system
- A document management system

Without Keycloak, a user would need **20 different usernames and passwords** — one for each system. With Keycloak, they log in ONCE and access ALL 20 systems.

Analogy: Keycloak is like a government ID card. The ID card is issued by the central government (Keycloak). It works at the health ministry (our system), the finance ministry (another system), the education ministry (another system). You don't need a separate ID card for each ministry.

```
OUR APPROACH (with Keycloak):
┌──────────────────────────────────────────────────────────┐
│  User has ONE account in Keycloak                        │
│                                                          │
│  Our system:     "Is this user real? Let me check        │
│                  Keycloak's signature on their token"    │
│                                                          │
│  HR system:      "Same user? Let me check Keycloak       │
│                  too. Yes, same person."                 │
│                                                          │
│  Budget system:  "Same user. Same ID. Same credentials."│
└──────────────────────────────────────────────────────────┘

ALTERNATIVE (without Keycloak):
┌──────────────────────────────────────────────────────────┐
│  User has SEPARATE accounts in every system              │
│                                                          │
│  Our system:     username: moh_admin, password: abc123   │
│  HR system:      username: aminah,   password: xyz789    │
│  Budget system:  username: ahassan,  password: qwe456    │
│                                                          │
│  User forgets 3 passwords. Writes them on sticky notes.  │
│  Sticky note falls off. Security risk.                   │
└──────────────────────────────────────────────────────────┘
```

Key point: Keycloak is NOT just for us. It is the CENTRAL identity system for ALL government online services. Our Django app is just ONE of many systems that use it.

How Django and Keycloak work together — the relationship

```
KEYCLOAK'S JOB:                DJANGO'S JOB:
┌──────────────────────┐       ┌──────────────────────┐
│  Store passwords     │       │  Store user profiles │
│  Check passwords     │       │  Store roles         │
│  Issue tokens        │       │  Store ministry      │
│  Handle "forgot pwd" │       │  Store permissions   │
│  Handle "new user"   │       │  Store audit logs    │
│                      │       │  Handle API tokens   │
│  Users from ALL      │       │                      │
│  government systems  │       │  Only our system's   │
│  live here           │       │  users live here     │
└──────────────────────┘       └──────────────────────┘

        ↑                            ↑
        │         THEY TALK          │
        │     VIA OIDC PROTOCOL      │
        └────────────────────────────┘

EXAMPLE: User "amina@moh.go.tz"
  In Keycloak:    username + password (the "who are you?" part)
```

```
  In Django:      role = MINISTRY_ADMIN, ministry = MOH (the "what can you do?"
part)
```

Why both? Why not just Django or just Keycloak?

If we ONLY used Keycloak: Keycloak doesn't know what ministry the user belongs to, what their role is, what assets they manage. Keycloak only knows "this username, this password, this email."

If we ONLY used Django: Each government system would have its own login. User needs 20 accounts. No centralized identity.

Does Django also store user credentials (passwords)?

The answer depends on how the user was created:

For web-only users (logged in via Keycloak): NO. Django never sees the password. The user types their password on Keycloak's page, not Django's page. Django receives a token FROM Keycloak that says "this user is authenticated," and stores an **unusable password** in its database — a placeholder like `!aBcDeF12345`.

This means:

- The user CAN log in through the web browser (Keycloak handles password checking)
- The user CANNOT log in through the mobile API (no valid Django password to check against)

For mobile/API users (logged in via direct API): YES. Django receives the password from the mobile app and checks it against its database. The password is stored securely (hashed using Django's password hasher).

This means:

- The user CAN log in through the mobile API (Django checks the stored password)
- The user can ALSO log in through the web (if they also have a Keycloak account)

Key insight — A user needs TWO different credentials for web + mobile:

```
EXAMPLE: User "amina@moh.go.tz"

  KEYCLOAK PASSWORD              DJANGO PASSWORD
  (for web browser)              (for mobile app)

  password: "M0H@2024"           password: "Am!naM0bile99"

  She types on Keycloak          She types in Flutter app
  login page                     login screen

  Stored in Keycloak's           Stored in Django's database
  own database                   (hashed, never in plain text)
```

These are **SEPARATE** passwords stored in **SEPARATE** databases. Changing one does not change the other.

How does a user get a Django password?

1. When a Super Admin creates the user directly in Django (via the user management page), they set a password
2. When a PendingAccess request is approved, the admin sets a password at that time
3. The password is sent to Django, hashed, and stored in the `password` field of `authentication_customuser` table

What happens if a Keycloak-only user tries to log in via the mobile app? They fail. Django has no valid stored password for them (only an unusable placeholder). The mobile app shows "Invalid credentials." This is correct behavior — the admin must set a separate Django password if the user needs mobile access.

What if a user exists in Keycloak but NOT in our Django database?

Step 1: User goes to our login page, clicks SSO, types their Keycloak username/password correctly. Keycloak says "Yes, this person is real."

Step 2: Keycloak sends the user back to Django with a valid token. Django checks the token and verifies: "Yes, Keycloak really signed this. This user is real."

Step 3: Django looks in its OWN database for this user. If the user exists → login success. If the user does NOT exist:

```
→ Django creates a PendingAccess record
→ Django shows the user a page saying:
  "Your account is not yet registered. Contact your administrator."
→ The Super Admin sees a pending approval request
→ The Super Admin can approve or reject
→ If approved: Django creates the actual user in its database
  (with role, ministry, and a Django password for mobile access)
→ The user can now log in via web (Keycloak) AND mobile (Django password)
```

Code evidence (authentication/oidc_backend.py):

```
def create_user(self, request, user_claims):
    # This runs when Keycloak says "user is real"
    # but Django has no matching user
    ...
    PendingAccess.objects.create(
        email=email,
        keycloak_id=sub,
        first_name=first_name,
        last_name=last_name,
    )
    return None  # ← Returning None means "user not allowed yet"
```

How does someone get created in Keycloak alone? (Without Django knowing)

An IT officer at a ministry can create a user in Keycloak directly without involving Django at all:

```
IT Officer goes to http://localhost:8180 → govasset realm → Users → Add User
  → Sets username: "amina", email: "amina@moh.go.tz"
  → Sets a temporary password
  → Tells Amina: "Your account is ready. Go to the system and log in."
                                  ↓
Amina opens our web app → Clicks "Sign in with Government SSO"
  → Keycloak says: "Yes, I know you. Password is correct."
  → Keycloak sends Amina to Django with a valid token
  → Django checks its database... "I don't know this person."
  → PendingAccess record is created
  → Amina sees: "Your account is awaiting approval. Contact your administrator."
  → Super Admin sees the pending request → approves it
  → Super Admin assigns Amina's role and ministry
  → Super Admin sets Amina's Django password (for mobile access)
  → Amina can now log in via web AND mobile
```

Important: Keycloak alone is NOT enough to access our system. The user must also exist in Django with a role and ministry assignment. PendingAccess is the bridge between "Keycloak knows you" (identity) and "Django knows you" (authorization).

Can someone be created in Django alone? (Without Keycloak)

YES. A Super Admin can create a user directly in Django using the user management page. This creates the user only in Django. The user can then log in via the mobile app (API login with Django password) but NOT via the web (no Keycloak account).

If the user also needs web access, the Super Admin must ALSO create them in Keycloak. Our code in `authentication/keycloak_admin.py` helps automate this — when you create a user in Django, it can optionally create a matching account in Keycloak too.

Summary: Two creation paths

```
PATH 1: Keycloak → Django
  User created in Keycloak by IT officer
    → User tries web login
    → PendingAccess created
    → Super Admin approves
    → Django user created with unusable password
    → User can log in via web ONLY
    → (Mobile access requires admin to set separate Django password)

PATH 2: Django → Keycloak
  Super Admin creates user in Django
    → Django creates matching account in Keycloak automatically
    → User gets BOTH:
        - Django password (for mobile API login)
        - Keycloak account (for web browser SSO)
    → User can log in via web AND mobile
```

The PendingAccess flow (Path 1) is designed for the real-world scenario where an IT officer creates users in the central identity system (Keycloak) without coordinating with our team. PendingAccess ensures no user gets access to our system until a Super Admin explicitly approves them and assigns a role.

Does Django manage sessions?

YES. Django manages TWO kinds of sessions:

Kind	What it is	Where it's stored
Django session cookie	"This browser is logged in as user X"	`django_session` table (public schema)
Keycloak session	"This user has an active Keycloak login"	Inside Keycloak's database

When you log in via the web:

1. Keycloak creates a session (you are logged into Keycloak)
2. Django creates a session (you are logged into Django)
3. Both are active at the same time

When you log out:

1. Django destroys its session (you're logged out of Django)
2. Django tells Keycloak to destroy its session too (you're logged out of Keycloak)
3. Both sessions are gone

Session expiry — Why you get asked to sign in again

You noticed that if you leave the system idle for a while, or switch to another tab for a long time, you come back and it asks you to sign in again. This is called **session expiry** and it is one of your security features — not a bug.

There are actually **three separate timeout systems** running in your project simultaneously. Each one works differently depending on whether you are on the web or mobile.

System 1 — Django Session Timeout (Web Platform)

When you log into the web platform, Django creates a **session** — a record stored in your database that says "this browser is logged in as user X."

This session has a time limit. In `config/settings.py`:

```
SESSION_COOKIE_AGE = 28800        # 8 hours (in seconds)
SESSION_COOKIE_HTTPONLY = True    # JavaScript cannot steal this cookie
```

28800 seconds = 8 hours. After 8 hours of inactivity, the session expires and Django makes you log in again.

Did you build this or did it come automatically? You configured it. Django's session system exists by default, but the 8-hour limit is something you deliberately set. A typical Django default is 2 weeks — you shortened it to 8 hours because a government platform should not stay logged in for 2 weeks on an unattended computer.

How to check:

```
Open config/settings.py → search for SESSION_COOKIE_AGE
Divide by 3600 to get hours: 28800 ÷ 3600 = 8 hours
```

How to change it (e.g., to 4 hours):

```
SESSION_COOKIE_AGE = 14400   # 4 hours (4 × 60 × 60)
```

System 2 — Keycloak Session Timeout (Web Login)

Keycloak has its own separate session. When a user logs in through Keycloak's page, Keycloak creates its own session remembering "this browser authenticated as user X."

This matters for **SSO (Single Sign-On)** — if a user has a valid Keycloak session and navigates to another system connected to the same Keycloak, they don't need to type their password again. But if the Keycloak session expires, even a valid Django session would force a re-authentication through Keycloak.

Where is this configured? In the Keycloak admin panel at `http://localhost:8180`:

```
Login as admin → govasset realm → Realm Settings → Sessions tab
```

You will see settings like:

- **SSO Session Idle** — how long a session can be idle before Keycloak expires it
- **SSO Session Max** — the absolute maximum lifetime regardless of activity

Did you build this? No — Keycloak manages this entirely on its own. You only interact with it through the Keycloak admin panel.

System 3 — JWT Token Expiry (Mobile App and API)

The mobile app does NOT use Django sessions or Keycloak sessions. It uses **JWT tokens**. These tokens have their own expiry built directly into them.

In `config/settings.py`:

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME':  timedelta(minutes=30),   # Token dies after 30 min
```

```
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),        # Refresh lasts 1 day
}
```

Access token — lasts 30 minutes. After 30 minutes, any API call the mobile app makes gets a 401 Unauthorized response. The app uses the refresh token to silently get a new access token without making the user type their password again.

Refresh token — lasts 1 day. After 1 day without any app activity, the refresh token expires. The mobile app returns to the login screen and the user must type their password again.

Did you build this? You configured it. The `djangorestframework-simplejwt` library handles the token creation — but the 30-minute and 1-day limits are values you set.

The complete picture — Three timers running at once

```
WEB PLATFORM USER
─────────────────
Opens browser → logs in through Keycloak → Django creates a session

Timer 1: Django session    → 8 hours idle → must re-authenticate to Django
Timer 2: Keycloak session  → set in Keycloak admin → must re-enter password

MOBILE APP USER
───────────────
Opens app → logs in through Django API → receives JWT tokens

Timer 3: Access token      → 30 minutes → app silently refreshes
Timer 4: Refresh token     → 1 day → must log in again
```

What you are probably seeing:

- **After a few hours away** → Django session expired (Timer 1, 8 hours)
- **Asked to type password on Keycloak's page specifically** → Keycloak session expired (Timer 2)
- **Mobile app shows login screen after a day** → Refresh token expired
- **Quickly after switching tabs** → Not a timeout — likely your browser lost the cookie, or you accidentally logged out

Forgot password — How it works with Keycloak

Question: Does the "forgot password" feature still work now that Keycloak handles passwords?

Answer: Yes, but it works differently — and this is actually more secure.

Before Keycloak: Django's own "forgot password" flow would send an email with a reset link, and the user would reset the password in Django's database.

After Keycloak: The password doesn't live in Django's database anymore. Django's password field is empty (set to an unusable value) for Keycloak-authenticated users. The password lives entirely in Keycloak.

So "forgot password" now goes through Keycloak:

```
User clicks "Forgot Password" on Keycloak's login page
      ↓
Keycloak sends a reset link to their email
      ↓
User clicks the link, sets a new password in Keycloak
      ↓
Done — they can log in again with the new password
```

Django is not involved in this process. This is **more secure** because password reset logic is handled by a dedicated, tested security tool (Keycloak) rather than our own code.

Does our Django "forgot password" URL still work? It might appear to work (send an email), but it would reset a password that isn't actually used anymore. If your login page has a "forgot password" link that goes to Django's own reset page, you should remove that link or redirect it to Keycloak's login page where the real forgot-password button lives.

Wrong password lockout — Triple protection (progressive + email unlock)

Question: If someone types the wrong password 5 times, does the lockout still work with Keycloak?

Answer: You actually have THREE separate protection layers, each covering a different attack surface:

Layer 1 — Keycloak's own lockout (for web browser users):

Keycloak itself watches for wrong passwords. After 5 wrong attempts, Keycloak locks that account for 15 minutes. This happens entirely inside Keycloak — Django doesn't even know about it.

```
Web user types wrong password 5 times on Keycloak's page
         ↓
Keycloak locks the account for 15 minutes
         ↓
Web user sees on Keycloak's page: "Account temporarily disabled"
         ↓
Django was never contacted — Keycloak handled everything
```

Layer 2 — Django's 3-stage progressive lockout with email self-service unlock (for API/mobile users):

This is our NEW system. Instead of a simple counter that resets after 15 minutes, we have a **3-stage progressive lockout** that gets more severe with each failure:

The Three Stages

Stage	Attempts	What happens
WARNING	1–3 failed attempts	The user is told "Invalid credentials" with remaining attempts shown. No lock. Just counting.
COOLDOWN	4–5 failed attempts	The user is locked out for 5 minutes. On the **5th failure**, the account is also disabled and an unlock email is sent.
DISABLED	6+ failed attempts	The account is permanently locked (`is_locked = True`). A new unlock email is sent. The only way back in is via the email link or an admin.

```
Full flow:

Attempt 1:   "Invalid credentials (3 attempts remaining)"
Attempt 2:   "Invalid credentials (2 attempts remaining)"
Attempt 3:   "Invalid credentials (1 attempt remaining)"
Attempt 4:   "Account locked for 5 minutes. Try again later."
Attempt 5:   "Account disabled. Check your email for unlock instructions."
                ↓
          Email sent to console:
          "Click here to unlock: http://127.0.0.1:8000/unlock-account/<uuid>/"
                ↓
          If user tries again during cooldown: "Account locked. <X> minutes
remaining."
          If user waits for cooldown and tries again: counts as attempt #6 →
disabled again
          If user clicks unlock link → account unlocked → all LoginAttempt records
deleted → counter reset
```

Key design decisions:

Decision	Why
Counter never resets on cooldown expiry	If the user could just wait 5 minutes and try again indefinitely, the lockout is useless. Once you reach DISABLED stage, you stay there until you use the unlock link.
`is_locked` is separate from `is_active`	`is_active=False` means an admin manually deactivated the user. `is_locked=True` means the brute-force protection kicked in. An admin can unlock without affecting `is_active`, and vice versa.
Unlock token is one-time + 1-hour expiry	Even if someone intercepts the email, the link only works once and expires quickly.
Old tokens are invalidated when a new one is created	If the user requests (or an attacker triggers) multiple lockouts, only the LATEST unlock link works. All previous tokens are marked as used.
Full LoginAttempt history is DELETED on unlock	Not marked as resolved. Hard delete. The user starts with a completely clean slate.

The models:

```
# authentication/models.py

# One new field on CustomUser:
is_locked = BooleanField(default=False)
# This is distinct from is_active. A user can be:
#   is_active=True,  is_locked=False → normal (can log in)
#   is_active=True,  is_locked=True  → locked by brute force (cannot log in)
#   is_active=False, is_locked=False → deactivated by admin (cannot log in)
#   is_active=False, is_locked=True  → deactivated + locked (cannot log in)

# LoginAttempt now has a stage field:
class LoginAttempt(models.Model):
    class Stage(models.TextChoices):
        WARNING  = 'WARNING',   'Warning'    # 1–3 attempts
        COOLDOWN = 'COOLDOWN',  'Cooldown'   # 4–5 attempts (5-min lock)
        DISABLED = 'DISABLED',  'Disabled'   # 6+ attempts (permanent lock)

    username    = CharField(max_length=150)
    ip_address  = GenericIPAddressField()
    attempts    = IntegerField(default=1)
    stage       = CharField(max_length=20, choices=Stage.choices,
default=Stage.WARNING)
    locked_until = DateTimeField(null=True, blank=True)  # When cooldown expires
    created_at  = DateTimeField(auto_now_add=True)
    last_attempt = DateTimeField(auto_now=True)

# New model for email unlock:
class UnlockToken(models.Model):
    user       = ForeignKey(CustomUser, on_delete=CASCADE)
    token      = UUIDField(unique=True, default=uuid4)
    created_at = DateTimeField(auto_now_add=True)
    used       = BooleanField(default=False)

    @classmethod
    def create_for_user(cls, user):
        # Mark any existing unused tokens as used first
        cls.objects.filter(user=user, used=False).update(used=True)
        return cls.objects.create(user=user)
```

The login view method — how it decides which stage:

```
# authentication/api_views.py (simplified)
class LoginAPIView(APIView):
    def post(self, request):
        username = request.data.get('username')
        ip = _get_client_ip(request)

        # Get or create tracking record for this user+IP
        attempt, _ = LoginAttempt.objects.get_or_create(
            username=username,
```

```
            ip_address=ip,
            defaults={'attempts': 1, 'stage': Stage.WARNING}
        )

        # Check cooldown: if user is in cooldown and timer hasn't expired
        if (attempt.stage == Stage.COOLDOWN
                and attempt.locked_until
                and timezone.now() < attempt.locked_until):
            remaining = int((attempt.locked_until -
timezone.now()).total_seconds() / 60)
            return Response(
                {'error': f'Account locked. Try again in {remaining} minute(s).'},
                status=429
            )

        # If cooldown expired, let them try but keep counting
        if (attempt.locked_until and timezone.now() >= attempt.locked_until):
            attempt.locked_until = None

        # Check if permanently disabled
        if attempt.stage == Stage.DISABLED:
            return Response({
                'error': 'Account disabled due to multiple failed attempts. '
                         'Check your email for unlock instructions.'
            }, status=403)

        # Increment attempt counter
        attempt.attempts += 1
        attempt.save(update_fields=['attempts', 'last_attempt'])

        # Stage assignment based on attempt count
        if attempt.attempts <= settings.LOGIN_MAX_WARNINGS:      # 1–3
            attempt.stage = Stage.WARNING
            remaining = settings.LOGIN_MAX_WARNINGS - attempt.attempts + 1
            return Response({'error': f'Invalid credentials ({remaining} attempts
remaining)'})

        elif attempt.attempts <= settings.LOGIN_MAX_ATTEMPTS:    # 4–5
            attempt.stage = Stage.COOLDOWN
            attempt.locked_until = timezone.now() +
timedelta(minutes=cooldown_mins)
            if attempt.attempts >= settings.LOGIN_MAX_ATTEMPTS:
                # On the 5th failure: disable + send unlock email
                attempt.stage = Stage.DISABLED
                user = CustomUser.objects.filter(username=username).first()
                if user:
                    _disable_account(user, attempt)
                    _send_unlock_email(user, request)
            return Response({'error': f'Account locked for {cooldown_mins}
minute(s).'}, status=429)

        else:                                                    # 6+
            attempt.stage = Stage.DISABLED
            user = CustomUser.objects.filter(username=username).first()
```

```
            if user:
                _disable_account(user, attempt)
                _send_unlock_email(user, request)
            return Response({
                'error': 'Account disabled. Check your email for unlock
instructions.'
            }, status=403)
```

The unlock email:

```
# authentication/api_views.py — _send_unlock_email()
def _send_unlock_email(self, user, request):
    token = UnlockToken.create_for_user(user)
    unlock_url = f"{settings.PLATFORM_BASE_URL}/unlock-account/{token.token}/"
    send_mail(
        subject='Account Unlock Instructions',
        message=f'Your account has been locked due to multiple failed login
attempts.'
                f'\n\nClick the link below to unlock your account (valid for 1
hour):'
                f'\n{unlock_url}',
        from_email='noreply@govasset.go.tz',
        recipient_list=[user.email],
        fail_silently=False,
    )
```

In development, emails print to the terminal (console backend). In production, real emails are sent.

The unlock page (GET /unlock-account/uuid:token/):

```
# authentication/unlock_views.py
def account_unlock_view(request, token):
    try:
        unlock_token = UnlockToken.objects.get(token=token, used=False)
    except UnlockToken.DoesNotExist:
        return render(request, 'authentication/unlock_result.html', {
            'success': False,
            'message': 'This link is invalid or has already been used.'
        })

    # Check expiry (1 hour)
    if timezone.now() > unlock_token.created_at + timedelta(hours=1):
        return render(request, 'authentication/unlock_result.html', {
            'success': False,
            'message': 'This link has expired. Contact your administrator.'
        })

    # Unlock the account
    user = unlock_token.user
    user.is_locked = False
```

```
    user.save(update_fields=['is_locked'])

    # Delete ALL LoginAttempt records for this user (fresh start)
    LoginAttempt.objects.filter(username=user.username).delete()

    # Mark token as used
    unlock_token.used = True
    unlock_token.save(update_fields=['used'])

    # Write to audit log
    AuditLog.objects.create(...)

    return render(request, 'authentication/unlock_result.html', {
        'success': True,
        'message': 'Your account has been unlocked. You can now log in.'
    })
```

Layer 3 — SSO blocking (for Keycloak web users whose Django account is locked):

Even though Keycloak handles its own lockout, we added an EXTRA check: if a user's Django account has `is_locked=True`, they cannot log in even through Keycloak SSO. This prevents the scenario where:

1. An attacker locks the account via the mobile API (5 failures)
2. The attacker then tries Keycloak SSO via web
3. Without this check, Keycloak would say "password is correct" and let them in

Our OIDC backend checks `user.is_locked` and blocks the login:

```
# authentication/oidc_backend.py
def filter_users_by_claims(self, claims):
    users = CustomUser.objects.filter(...)
    if users.exists() and users.first().is_locked:
        # Show a lock message on the login page
        return CustomUser.objects.none()  # ← Returns empty = login fails
```

The login page then displays: "*Your account has been locked. Check your email for unlock instructions.*"

Summary — Who protects whom:

Who is trying to log in	Which protection blocks them
Web browser user (through Keycloak page)	**Layer 1** — Keycloak's own lockout (after 5 wrong passwords on Keycloak's page)
Web browser user (Keycloak password correct, but Django account locked)	**Layer 3** — OIDC backend checks `is_locked` and rejects

Who is trying to log in	Which protection blocks them
Mobile app user (through your API)	**Layer 2** — 3-stage progressive lockout (after 5 failures, permanent disable + email unlock)
Another group's system (through your API)	**Layer 2** — Same 3-stage lockout applies

You have **triple protection**. Every path is covered.

Admin user management — How locked users appear (and the bug we fixed)

Question: If a user gets locked by brute-force protection, why doesn't the user management page at `/users/` show them as disabled/locked?

Answer: This was a bug we discovered and fixed. Here's the full story:

The two separate status fields

When we designed the brute-force protection, we created `is_locked` as a **separate field** from Django's built-in `is_active`. This was intentional:

Field	What it means	Who controls it
`is_active` (Django built-in)	"Is this user allowed to log in at all?"	The admin manually toggles this from the user management page
`is_locked` (our new field)	"Was this user locked by brute-force protection?"	The system sets this automatically after 5+ failed attempts

These are independent. A user can be:

```
is_active=True,  is_locked=False → Normal user (can log in)
is_active=True,  is_locked=True  → Locked by brute force (cannot log in, needs
unlock)
is_active=False, is_locked=False → Deactivated by admin (cannot log in)
is_active=False, is_locked=True  → Deactivated by admin AND locked by system
(cannot log in)
```

The bug

The user management table at `/users/` originally only checked `is_active` to determine the status:

```
{# OLD code — only checked is_active #}
<span class="status-dot {% if u.is_active %}active{% else %}inactive{% endif %}">
</span>
{% if u.is_active %}Active{% else %}Inactive{% endif %}
```

And the toggle button only flipped `is_active`:

```
# OLD code — never touched is_locked
target_user.is_active = not target_user.is_active
target_user.save()
```

So when brute-force protection locked a user (`is_locked=True` but `is_active` stayed `True`):

- The status column showed **"Active"** (green dot) — misleading
- The toggle button showed **"Deactivate"** (red icon) — wrong action
- The row had full opacity — no visual dimming
- Clicking the toggle button would deactivate (`is_active=False`) but leave `is_locked=True`, making things worse

The fix

We made three changes:

1. Status column now shows three states:

```
User state        Display
──────────────────────────
Active + unlocked → "Active" (green dot)
Inactive          → "Inactive" (gray dot)
Locked            → "Locked" (amber/orange dot)
```

Code:

```
{% if u.is_locked %}
  <span class="status-dot" style="background:#d97706;"></span>
  <span style="color:#d97706;font-weight:600;">Locked</span>
{% elif u.is_active %}
  <span class="status-dot active"></span>
  Active
{% else %}
  <span class="status-dot inactive"></span>
  Inactive
{% endif %}
```

2. Toggle button now adapts to locked state:

User state	Button shown	Icon	Color	Confirmation
Active + unlocked	Deactivate	`person-slash`	Red outline	"Deactivate username?"
Inactive + unlocked	Activate	`person-check`	Gray outline	"Activate username?"

User state	Button shown	Icon	Color	Confirmation
Locked	**Unlock**	`unlock`	**Orange** outline	"Unlock username?"

3. Toggle view now clears `is_locked` when re-enabling:

```
target_user.is_active = not target_user.is_active
if target_user.is_active and target_user.is_locked:
    target_user.is_locked = False  # ← NEW: clear lock when activating
target_user.save()
```

So if an admin clicks the unlock button for a locked user, it:

1. Sets `is_active = True`
2. Sets `is_locked = False`
3. Syncs the active status to Keycloak via the Keycloak admin API

The user can now log in again immediately.

Why not just use `is_active` for lockout?

Question: Can we just use one field instead of two? Why do we need both `is_active` and `is_locked`?

Answer: Both are needed because they control two completely different things. Here's why you cannot remove either one:

The building analogy:

Think of your system as a secured office building with two separate security layers:

Security layer	Our system equivalent	Who controls it
The building manager's key	`is_active`	The admin (via the `/users/` page)
The automatic security lock	`is_locked`	The system (auto-locks after 5 wrong passwords)

- The **building manager** has a master key. They decide who works here. If an employee is fired, the manager disables their badge (`is_active=False`).
- The **automatic security lock** is like a door that locks itself after someone types the wrong PIN 5 times. The employee can call security (click the unlock email link) to get back in.

Now imagine what happens if we remove one of them:

Scenario A — If we only had `is_active` (no `is_locked`):

```
1. Employee forgets password → types wrong password 5 times
2. System sets is_active=False (locks them out)
3. Unlock email is sent to the employee
4. Employee clicks the unlock link
5. System sets is_active=True → employee is back in
   ✓ Good: Employee can self-service unlock

But what if:
6. Admin fires the employee → sets is_active=False
7. The fired employee goes to the login page → types wrong password 5 times
8. System sends unlock email
9. Fired employee clicks the link → is_active=True!
   ✗ BAD! A FIRED EMPLOYEE CAN UNLOCK THEMSELVES!
```

This is a security disaster. The self-service unlock should NOT override an admin's decision to deactivate someone.

Scenario B — If we only had `is_locked` (no `is_active`):

```
1. Admin fires an employee → needs to block their access
2. System sets is_locked=True
3. But is_locked is a custom field — Django's built-in auth doesn't check it
4. The employee might slip through in login pages, password resets, or API calls
5. Also, the sync to Keycloak uses Django's built-in is_active
6. Toggling the button on /users/ page would need extra custom code everywhere
```

This is unreliable. Django's core authentication system checks `is_active` in many places automatically. Our custom `is_locked` field only works where we explicitly check it.

The real distinction — What each lock protects:

Lock	Who holds the key	Can the unlock email override it?
`is_active`	Only the admin	**NO** — A fired employee cannot get back in by typing wrong passwords
`is_locked`	The user (via unlock email)	**YES** — The self-service unlock link only clears this field

Summary — Why we need both:

```
Two fields = two separate concerns.

  is_active  →  "Should this person work here at all?"
                Controlled by: Admin only
                Overrideable by: Nobody except admin

  is_locked  →  "Did someone try to break into this account?"
```

```
                    Controlled by: System (auto after 5 failures)
                    Overrideable by: User (via unlock email) OR admin
```

If we merged them, the admin's decision to fire someone could be undone by a simple unlock email. That's why they stay separate.

What about the Keycloak admin panel?

Keycloak also has its own user status (enabled/disabled). When our system locks a user (`is_locked=True`), it does NOT automatically disable the user in Keycloak. This is why:

- **Keycloak's enabled/disabled** controls whether the user can log in through Keycloak's login page
- **Django's `is_locked`** controls whether the user can log in through our API

They are separate systems. If a user is locked in Django but still enabled in Keycloak, they:

- **Cannot** log in via our mobile API (locked by Django)
- **Can** try to log in via web → Keycloak checks password → if correct, Django's OIDC backend then checks `is_locked` and blocks them (Layer 3)

When an admin toggles the user from our `/users/` page, it ALSO syncs to Keycloak — so both systems stay in sync.

The reverse sync problem — Keycloak → Django was missing

Question: If an admin disables a user in the Keycloak admin panel, why doesn't the Django `/users/` page show them as disabled?

Answer (simple): Because Django never knew about it. Only Keycloak knew. The sync was only going one direction.

Think of it like two separate offices that both have a list of employees. When you update the list in Office A (Keycloak), you have to tell Office B (Django) about the change. Before our fix, nobody was carrying the message from Office A to Office B.

What was happening before the fix:

```
An admin logs into Keycloak admin panel → finds a user → clicks "Disable"
         ↓
Keycloak disables the user (sets enabled: false)
         ↓
But nobody tells Django about this change
         ↓
Django still shows is_active=True for that user
         ↓
The /users/ page says "Active" — WRONG!
         ↓
The mobile app still lets the user log in — SECURITY RISK!
```

The fix — Two-way sync (like two offices sending each other updates):

We made the sync work in BOTH directions. Here are the three ways we did it:

Way 1 — Auto-sync when you visit the User Management page (automatic, no clicking needed)

This is the most important fix. Every time you visit `/users/` in your browser, Django automatically asks Keycloak: "Hey, what's the status of all my users?" Then it updates its own records to match.

You don't need to click anything. Just refresh the page.

```
You visit http://localhost:8000/users/
         ↓
Django calls Keycloak: "Give me ALL your users"
         ↓
Keycloak returns: [
  { "id": "abc123", "enabled": true  },
  { "id": "def456", "enabled": false },   ← This user was disabled in Keycloak!
  { "id": "ghi789", "enabled": true  },
]
         ↓
Django checks each user: "Does our is_active match Keycloak's enabled?"
         ↓
For "def456": Django had is_active=True, but Keycloak says enabled=false
         ↓
Django fixes it: sets is_active=False for that user
         ↓
The page shows "Inactive" — NOW CORRECT!
```

Why this is better than having a refresh button per user:

Imagine you have 500 users. With a refresh button per user, you'd have to click 500 buttons — that's impossible. With auto-sync on page load, you just refresh the page ONCE and all 500 users get checked and updated in the background.

The code that does this (simplified):

```
# authentication/user_views.py — user_list_view()

# Step 1: Get all users from Keycloak in ONE big request
kc = KeycloakAdminService()
all_kc_users = kc.get_all_users()  # This is ONE API call

# Step 2: Build a quick lookup table
# keycloak_id → is_enabled?
kc_status = {u['id']: u.get('enabled', True) for u in all_kc_users}

# Step 3: Check each Django user against Keycloak
for django_user in all_users_on_this_page:
    if django_user.keycloak_id in kc_status:
```

```
        kc_enabled = kc_status[django_user.keycloak_id]
        if kc_enabled != django_user.is_active:
            # Found a mismatch! Fix it.
            django_user.is_active = kc_enabled
            django_user.save()
```

This uses a new method we added to `KeycloakAdminService` that fetches ALL users from Keycloak (automatically handling pagination — if there are 300 users, it fetches them in pages of 100):

```
# authentication/keycloak_admin.py
def get_all_users(self):
    """Fetch ALL users from Keycloak. Handles pagination automatically."""
    all_users = []
    first = 0
    page_size = 100
    while True:
        batch = self.list_users_page(first=first, max=page_size)
        if not batch:
            break
        all_users.extend(batch)
        if len(batch) < page_size:
            break  # Last page — fewer results than page size
        first += page_size
    return all_users
```

Way 2 — Auto-sync when a user logs in (automatic, user doesn't notice)

Every time a user logs in through Keycloak (the "Sign in with Government SSO" button), Django checks that specific user's status in Keycloak and updates their `is_active` to match.

This handles the case where:

- An admin disables a user in Keycloak at 2 PM
- That user tries to log in at 3 PM
- Django says: "Let me check Keycloak... Oh, you're disabled. Updating my records."
- Login fails — the user sees an error message
- If the admin later visits `/users/`, they'll see "Inactive" (already synced!)

```
# authentication/oidc_backend.py — filter_users_by_claims()
if user.keycloak_id:
    kc = KeycloakAdminService()
    kc_user = kc.get_user(user.keycloak_id)
    if kc_user is not None:
        kc_enabled = kc_user.get('enabled', True)
        if kc_enabled != user.is_active:
            user.is_active = kc_enabled
            user.save(update_fields=['is_active'])
```

Way 3 — When admin toggles from Django → Keycloak (already worked, untouched)

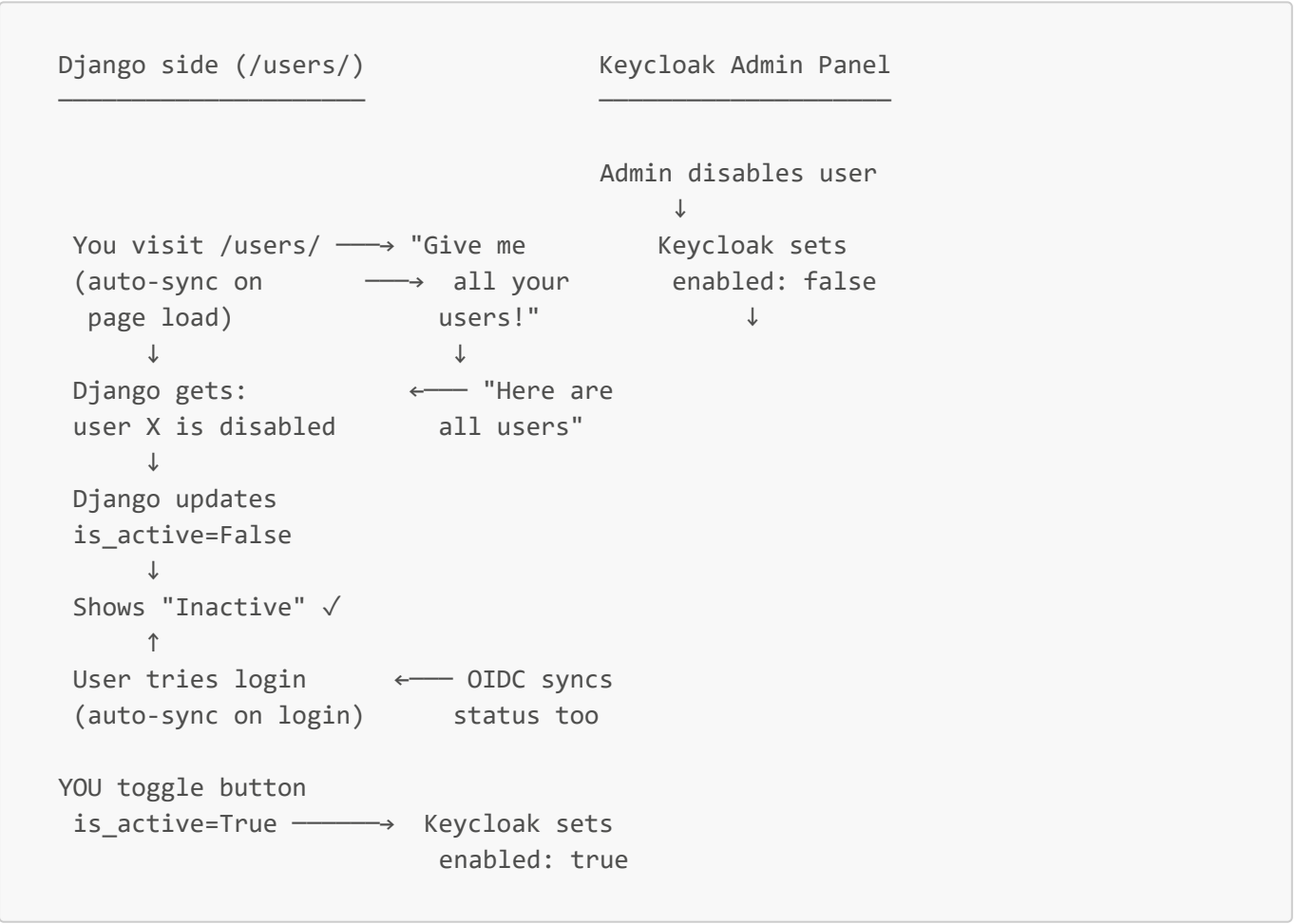
This was already working before. When you click the disable/enable button on the `/users/` page, Django:

1. Changes `is_active` in its own database
2. ALSO calls Keycloak's API to change `enabled` in Keycloak

```
# authentication/user_views.py — user_toggle_active_view()
target_user.is_active = not target_user.is_active
target_user.save()

# Also sync to Keycloak
kc = KeycloakAdminService()
kc.update_user(keycloak_id=..., is_active=target_user.is_active)
```

Complete picture — How the two-way sync works now



Direction	How it works	When it happens
Django → Keycloak	Django calls `PUT /admin/realms/{realm}/users/{id}` with `enabled: true/false`	When you click the toggle button on the `/users/` page

---

Direction	How it works	When it happens
Keycloak → Django (batch)	Django calls `GET /admin/realms/{realm}/users` to get ALL users, then updates any mismatches	Every time you visit the `/users/` page (automatic)
Keycloak → Django (single)	Django calls `GET /admin/realms/{realm}/users/{id}` to check ONE user	When that user logs in via SSO (automatic)

Result: No matter where you change a user's status — in Django or in Keycloak — the other system will find out about it. You always see the correct status on the `/users/` page. Just refresh the page, no clicking needed.

Summary — Is our system now working correctly for both locked and disabled?

Yes. Here is the complete picture of what happens in every scenario:

What happens	`is_active`	`is_locked`	User can log in?	Status shown on `/users/`
Normal user	`True`	`False`	Yes	**Active** (green)
User types wrong password 5 times (system locks)	`True`	`True`	No	**Locked** (amber)
User clicks unlock email	`True`	`False`	Yes	**Active** (green)
Admin deactivates user from `/users/` page	`False`	`False`	No	**Inactive** (gray)
Admin deactivates user from Keycloak admin	`False` (auto-synced)	`False`	No	**Inactive** (gray)
Admin unlocks user (clicks toggle button)	`True`	`False`	Yes	**Active** (green)

To confirm your system is working, test these scenarios:

1. **Brute-force lock:** Try wrong password 5 times via mobile API → user shows as "Locked" on `/users/` page
2. **Self-service unlock:** Click unlock email link → user shows as "Active" again
3. **Admin deactivation:** Click toggle button on `/users/` page → Keycloak also shows user as disabled
4. **Keycloak deactivation:** Disable user in Keycloak admin → refresh `/users/` page → Django now shows "Inactive"
5. **Re-enable:** Click toggle button again → both systems show active

All five paths work correctly because:

- `is_locked` and `is_active` are independent fields with separate purposes
- The sync is now **two-way** — changes in either system propagate to the other
- The `/users/` page checks **both** fields to show the correct status

---

PART 5: What is the difference between Keycloak and SimpleJWT?

People get confused. Here is the simple answer:

	Keycloak	**SimpleJWT**
Used for	Logging in (web browser)	API calls (mobile app)
Who handles it	Keycloak server (separate program)	Django itself
What it produces	A Django session (cookie in browser)	A JWT token (text string)
When is it used	When user clicks "Sign in"	When mobile app calls API
Password involved?	Yes, user types password on Keycloak	Yes, on first login only

Think of it this way:

- **Keycloak** = The security guard at the gate who checks your ID
- **SimpleJWT** = The visitor badge you get AFTER the guard lets you in

The mobile app (Flutter) does NOT use Keycloak. Instead:

1. Mobile app calls `POST /api/auth/login/` with username + password
2. Django checks the password against its database (same as Keycloak)
3. Django returns a JWT token (valid 30 minutes)
4. Mobile app sends this token with every API call
5. When token expires, mobile app calls `POST /api/auth/refresh/`

Why doesn't the mobile app use Keycloak?

Keycloak uses a **browser redirect flow** — it sends the user to a Keycloak login page, the user types their password there, then it redirects back to Django.

This does NOT work on a mobile app because:

```
WEB BROWSER FLOW (works with Keycloak):
  Browser → Django → redirect to Keycloak → user types password → redirect back
  ↑                                                                    ↑
  Browser can follow redirects automatically.
  The "Sign in with Google" button you see on websites works this way.

MOBILE APP FLOW (cannot use Keycloak):
  Flutter app → Django → "Go to Keycloak" → ???
  ↑
  A mobile app is not a web browser. It cannot display Keycloak's
  login page. It cannot follow web redirects.

  Option A: Open a web browser inside the app (WebView)
            → Ugly, confusing, users don't trust it
            → Keycloak session doesn't persist properly
            → Many technical problems
```

```
   Option B: Mobile app sends username + password directly to Django
             → Simple, works every time
             → Django checks the password and returns a JWT token
             → This is what we do ✓
```

The real reason: Keycloak (and all OIDC providers) were designed for web browsers. Mobile apps use a different authentication pattern called "direct login" where the app sends credentials to an API and gets back a token. This is industry standard — even Google, Facebook, and Twitter have separate "mobile login" APIs that don't use browser redirects.

Why doesn't the web browser use SimpleJWT (like the mobile app)?

Three reasons:

Reason	Explanation
1. Security	With SimpleJWT, the user types their password directly into Django's login page. If someone steals the password, they have full access. With Keycloak, Django never sees the password — Keycloak handles it securely.
2. Single Sign-On	If the web used SimpleJWT, users would need a separate login for EVERY government system (asset management, HR, budget, etc.) — 20 different passwords. With Keycloak SSO, one login works everywhere.
3. Government policy	The government mandates that ALL online services use Keycloak for authentication. We cannot build our own login page that bypasses Keycloak. This is a security policy, not a technical choice.

So why does the mobile app get an exception?

Because there is no other way. Mobile apps cannot use Keycloak's browser redirect flow (explained above). The government understands this and allows mobile apps to use direct API login. But the mobile app's login is STILL connected to Keycloak indirectly:

```
Mobile app login → Django checks password → Django stores users that
                   came FROM Keycloak originally
                   ↑
    The user first had to exist in Keycloak and be approved in Django
    via the web flow. The mobile app login is a CONVENIENCE for
    already-approved users.
```

Summary: Two authentication paths side by side

```
PATH 1: WEB BROWSER (uses Keycloak)
┌─────────┐     ┌─────────┐     ┌──────────┐
│ Browser │────→│ Django  │────→│ Keycloak │  User types password on Keycloak
│         │←────│         │←────│          │  Django never sees password
└─────────┘     └─────────┘     └──────────┘
```

```
PATH 2: MOBILE APP (uses SimpleJWT)
┌──────────┐     ┌──────────┐
│ Flutter  │────→│ Django   │  User types password on the phone's screen
│  App     │←────│          │  Django checks password against its database
└──────────┘     └──────────┘  Django returns JWT token
```

PART 6: Why 0.0.0.0:8000 instead of just `runserver`?

Normal runserver:

```
python manage.py runserver
→ Django listens ONLY on 127.0.0.1:8000
→ 127.0.0.1 = "this computer only"
→ Your phone CANNOT reach this
→ moh.localhost CANNOT reach this
```

0.0.0.0:8000:

```
python manage.py runserver 0.0.0.0:8000
→ Django listens on ALL network interfaces
→ 0.0.0.0 = "everywhere"
→ Same WiFi network? Your phone CAN reach this
→ moh.localhost on same computer CAN reach this
```

Analogy:

- `127.0.0.1` = a phone that only calls itself
- `0.0.0.0` = a phone that can receive calls from anyone

Why 127.0.0.1? — What it means when you see it

When you open your browser on your laptop and go to `http://localhost:8000`, the request NEVER leaves your machine. It travels from one program (your browser) to another program (Django) entirely inside the same computer, through what is called a **loopback interface**.

Every computer in the world reserves the address `127.0.0.1` to mean **"myself."** It is not a real network address — it is a shortcut that means "stay inside this machine."

So when Django reads `request.META['REMOTE_ADDR']` and the browser is on the same laptop, the answer is always `127.0.0.1` — because the request came from the machine itself.

When your phone sends a request over the hotspot, the request travels through a real network — from your phone's WiFi radio, through the hotspot signal, to your laptop's WiFi adapter. At that point `REMOTE_ADDR` is a real network address like `192.168.43.118` — your phone's actual address on that small private network.

The rule — Who sees what IP:

Who sent the request	What IP you see in request.META
Browser on the same laptop as Django	`127.0.0.1` (loopback — never leaves the machine)
Phone over hotspot	`192.168.43.x` (phone's real network address)
Another laptop on the same WiFi	That laptop's IP
Real internet user (in production)	Their real public IP

Why this matters for security: In a real government deployment, every audit log entry would show the real public IP of whoever performed the action. Investigators can trace a public IP back to a physical location or an organization if needed. During development, you see `127.0.0.1` which is correct — the request is from your own machine.

Do you need 0.0.0.0 in production (cloud)?

NO. In production, you use a proper web server like Nginx or Apache. The server listens on `0.0.0.0:80` (that's what web servers do by default). Django sits behind the web server and only listens on localhost (127.0.0.1:8000) — the web server forwards requests to it.

So: `0.0.0.0:8000` is ONLY for development when you need your phone or tablet to test.

What if you don't use 0.0.0.0?

Then your mobile phone connected to the same WiFi cannot access the website. You type `http://192.168.100.18:8000` on your phone and nothing happens, because Django only listens on `127.0.0.1` (which means itself only).

PART 7: All Your Code Components — Explained Simply

What is `mozilla_django_oidc`?

It is a Django library that does the Keycloak handshake for us. Without it, we would have to write hundreds of lines of code to handle the OIDC protocol (redirects, code exchange, token verification). The library provides:

- `/oidc/authenticate/` — Redirects user to Keycloak login page
- `/oidc/callback/` — Handles the return from Keycloak, exchanges code for tokens
- `/oidc/logout/` — Logs user out of both Django and Keycloak

What is `rest_framework` (DRF)?

Django REST Framework is a library that makes it easy to build APIs (Application Programming Interfaces). An API is how the Flutter mobile app talks to our Django backend.

Without DRF: You would have to manually parse JSON from requests, manually convert database data to JSON, manually handle errors, etc.

With DRF: You define a serializer (a class that says "here are the fields") and a view (a class that says "what happens on GET, POST, etc."), and DRF handles everything.

What is `corsheaders`?

CORS = Cross-Origin Resource Sharing.

Problem: By default, a web browser on `flutterapp.com` cannot call an API on `moh.localhost:8000`. The browser blocks it for security.

Solution: CORS headers. We tell Django: "Hey, it's OK if the Flutter app calls our API from a different address."

Our settings:

```
CORS_ALLOW_ALL_ORIGINS = True  # Allow Flutter from any address
CORS_ALLOW_CREDENTIALS = True   # Allow cookies to be sent
```

What is `drf_yasg`?

It automatically creates a Swagger documentation page at `/api/docs/`. This is a nice web page that shows ALL your API endpoints, with buttons to test them. Other groups (Group 2-10) use this page to understand how to call our API.

You don't write documentation — drf_yasg reads your code and generates it.

What is `python-decouple`?

It reads secret values from a `.env` file. This keeps passwords out of your code.

```
# Instead of writing the password in settings.py:
SECRET_KEY = 'my-secret-key-here'  # BAD — visible in code

# We do this:
SECRET_KEY = config('SECRET_KEY')  # GOOD — read from .env file
```

Your `.env` file has:

```
SECRET_KEY=my-secret-key-here
DB_PASSWORD=my-db-password
```

The `.env` file is NEVER committed to GitHub (it's in `.gitignore`).

What is `psycopg2-binary`?

It is the driver that lets Python talk to PostgreSQL. Think of it like a translator — Django speaks Python, PostgreSQL speaks SQL, psycopg2 translates between them.

What is Pillow?

A library that handles image files. When someone uploads a photo of an asset (like a laptop), Pillow processes the image — resizing, format conversion, etc.

What are DECORATORS?

A decorator is a wrapper around a function. It runs code BEFORE your function runs.

Analogy: A security guard at the door of a building. Before you enter (before your function runs), the guard checks your ID.

```
@login_required_custom     # ← This is a decorator (security guard)
@role_required('SUPER_ADMIN')  # ← Another security guard
def ministry_list_view(request):
    # This code only runs if BOTH guards say OK
    ...
```

Our custom decorators:

Decorator	What it does	Where it's used
`@login_required_custom`	Blocks unauthenticated users → redirect to login	ALL views
`@role_required('MINISTRY_ADMIN')`	Blocks wrong roles → redirect to dashboard	Admin views
`@ministry_isolation_check`	Blocks users without a ministry schema	Ministry views

Code (authentication/decorators.py):

```
def login_required_custom(view_func):
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
            messages.warning(request, "Please log in.")
            return redirect('login')
        return view_func(request, *args, **kwargs)
    return wrapper
```

What are SERIALIZERS?

A serializer converts between two formats:

1. **Python objects** → **JSON** (for the API response)
2. **JSON** → **Python objects** (from the API request)

Example: When Flutter asks for an asset, Django:

1. Gets the Asset object from the database (Python object)
2. Passes it through AssetSerializer
3. DRF converts it to JSON automatically
4. Flutter receives: `{"id": 1, "name": "Dell Laptop", "asset_number": "MOH-ICT-0001"}`

Our serializers (authentication/api_serializers.py):

- `AssetSerializer` — Converts assets to JSON for the mobile app
- `AuditLogSerializer` — Converts audit logs
- `OrgUnitSerializer` — Converts organization units
- `UserProfileSerializer` — Converts user profiles (never exposes passwords!)
- `CustomTokenObtainPairSerializer` — Adds role, ministry_schema to JWT tokens

What is MIDDLEWARE?

Middleware is code that runs on EVERY request, BEFORE and AFTER your view.

Think of it as a pipeline:

```
Request comes in
  → Middleware 1 (TenantMainMiddleware: sets schema)
  → Middleware 2 (SecurityMiddleware: security headers)
  → Middleware 3 (SessionMiddleware: loads session)
  → Middleware 4 (CorsMiddleware: handles CORS)
  → Middleware 5 (AuthMiddleware: loads logged-in user)
  → Middleware 6 (SchemaMiddleware: OUR custom one)
  → YOUR VIEW FUNCTION
  → Back through all middlewares
Response goes out
```

Our custom middleware (authentication/middleware.py):

```python
class SchemaMiddleware(MiddlewareMixin):
    def process_request(self, request):
        # Skip public pages
        # For authenticated users, save their schema name on the request
        if hasattr(request, 'user') and request.user.is_authenticated:
            request.schema_name = request.user.ministry_schema or 'public'
            request.user_role = request.user.role
```

This means ANY view can access `request.schema_name` — no need to look it up from the database each time.

What is the EXCEPTION HANDLER?

When an API error happens (e.g., user not authenticated, permission denied, not found), DRF returns errors in different formats. The Flutter app needs a CONSISTENT format.

Our custom handler (authentication/api_exception_handler.py) catches all errors and returns them in one shape:

```
{
    "error": True,
    "message": "Authentication credentials were not provided.",
    "code": "authentication_required",
    "status": 401
}
```

Flutter always knows: check `error`, read `message`, handle by `code`.

What is PAGINATION?

When you have 1000 assets and the user views the list, you don't send all 1000 at once — you send 20 at a time (one page).

Our pagination helper (authentication/pagination.py):

```
def paginate_queryset(queryset, request, per_page=20):
    paginator = Paginator(queryset, per_page)
    page = paginator.page(request.GET.get('page', 1))
    return page, paginator
```

Usage in views:

```
page, paginator = paginate_queryset(assets, request, per_page=25)
# page.object_list = the 25 items for this page
# paginator.count = total number of items
# paginator.num_pages = how many pages total
```

What are PERMISSION CLASSES?

DRF has a built-in system for checking who can access API endpoints. We created custom permission classes:

Where (authentication/api_permissions.py):

```
class HasMinistrySchema(BasePermission):
    """Block users who aren't assigned to any ministry."""
    def has_permission(self, request, view):
        if request.user.role == 'SUPER_ADMIN':
            return True
        return bool(request.user.ministry_schema)

class CanManageAssets(BasePermission):
    """Auditors can read only. Others can write."""
```

```
    def has_permission(self, request, view):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True  # Anyone can read
        return request.user.role in ['SUPER_ADMIN', 'MINISTRY_ADMIN', ...]
```

Usage in API views:

```
class AssetListCreateAPIView(APIView):
    permission_classes = [
        IsAuthenticated,    # Must be logged in
        HasMinistrySchema,  # Must have a ministry
        CanManageAssets,    # Must have permission
    ]
```

What are HELPER FUNCTIONS?

The most important thing to know: A helper function is NOT a special kind of function. It is just a normal Python function. Python treats it exactly the same as any other function. The name "helper" only describes its **purpose** — it helps other parts of your code do their job.

So what makes a function a "helper"?

A helper function is a small function that does ONE small, reusable job. Think of it as a **tool** you keep in your toolbox and grab whenever you need it.

	Main function	**Helper function**
Job	Does the main task (e.g., login a user, create an asset)	Does a small supporting task (e.g., get the user's IP)
Size	Can be 20-100+ lines	Usually 3-15 lines
Used by	The browser/API calls it directly	Other functions call it
Reused?	Usually called once per URL	Called from many places

Analogy:

- Main function = a chef cooking a full meal
- Helper function = a knife that the chef uses to chop, slice, and dice across many meals

The rule — When to create a helper function:

If you write the **same code more than once** in different places, and that code performs **one clear task**, make it a helper function.

For example, instead of writing this in 3 different views:

```
x_forwarded_for = request.META.get("HTTP_X_FORWARDED_FOR")
if x_forwarded_for:
```

```
    ip = x_forwarded_for.split(",")[0].strip()
else:
    ip = request.META.get("REMOTE_ADDR")
```

You write it ONCE as a helper and call it from everywhere:

```
ip = _get_client_ip(request)  # One line — clean!
```

Why does this matter? If you later need to change how the IP is determined (e.g., add support for a new header), you update ONE function instead of searching through many files. This is called **Don't Repeat Yourself (DRY)** — a core principle of good code.

Where is `request.META['REMOTE_ADDR']` in our project?

The answer is: **It's not in our project.** It is provided by **Django** automatically.

When a browser sends a request, Django creates an `HttpRequest` object for you:

```
def my_view(request):    # ← Django CREATES this 'request' object
    print(request)       #    You never created it yourself
```

Inside that object, Django stores a dictionary called `META` containing everything about the request — the sender's IP, what browser they used, what domain they visited, etc. Your code just reads from it:

```
request.META['REMOTE_ADDR']    # Returns: "192.168.1.100"
```

You never create `request.META` — Django does it for you automatically on every request.

Example in our code — `_get_client_ip()`:

```
def _get_client_ip(request):
    """Get the real IP address of the user making the request."""
    x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
    if x_forwarded_for:
        ip = x_forwarded_for.split(',')[0].strip()
    else:
        ip = request.META.get('REMOTE_ADDR')
    return ip
```

Why make this a helper function? Because 3 different places need the user's IP:

1. The audit log (records who did what from what IP)
2. The brute force lockout (counts failed logins per IP)
3. The dashboard (shows login history)

Without a helper, you write the same 6 lines in 3 places. With a helper, you write it ONCE and call it.

Why does it start with an underscore `_`?

The underscore is a Python **convention** (not a rule enforced by Python). It tells other programmers:

> "This function is for internal use only. Don't use it outside this file unless you have a good reason."

Python does NOT stop you from calling `_get_client_ip(request)` from anywhere. The underscore is just a polite warning to other developers.

How to spot a helper function in code:

- It has a descriptive name like `_get_client_ip`, `paginate_queryset`, `format_date`
- It is usually small (3-15 lines)
- It does ONE thing and does it well
- It often starts with underscore `_` (internal use convention)
- It is defined near the top of a file, right after the imports

Another example in our code — `paginate_queryset()`:

```
def paginate_queryset(queryset, request, per_page=20):
    paginator = Paginator(queryset, per_page)
    page = paginator.page(request.GET.get('page', 1))
    return page, paginator
```

Every view that shows a list (assets, users, audit logs) calls this instead of writing pagination logic from scratch. That's the whole point of a helper — write once, use everywhere.

PART 8: SECURITY FEATURES

Here is EVERY security feature in the system and where to find it:

1. Database-level isolation

Users of one ministry CANNOT see another ministry's data — even if there is a bug in our code. PostgreSQL schemas enforce this.

- **Evidence:** `DATABASE_ENGINE = django_tenants.postgresql_backend`

2. Role-based access control

Each user has a role (SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK, AUDITOR). Views check roles before running.

- **Evidence:** `@role_required('MINISTRY_ADMIN')` on every sensitive view

3. API permission classes

Same as roles but for API endpoints. Different levels: IsSuperAdmin, IsMinistryAdmin, CanManageAssets, CanDeleteAssets, CanViewAuditLogs.

- **Evidence:** `authentication/api_permissions.py`

4. Brute force protection — 3-stage progressive lockout with email unlock

A 3-stage system that gets progressively more severe with each failed login attempt:

- **Stage 1 — WARNING** (attempts 1–3): Counter only. User sees remaining attempts.
- **Stage 2 — COOLDOWN** (attempts 4–5): 5-minute lockout. On the 5th failure, account is disabled + unlock email is sent.
- **Stage 3 — DISABLED** (attempts 6+): Permanent lock. `is_locked=True`. Only an unlock link or admin can restore.
- **Self-service unlock:** Email link valid 1 hour, one-time use. Deletes all `LoginAttempt` records on unlock.
- **`is_locked` is separate from `is_active`** so admin deactivation and brute-force lockout don't interfere.
- **Also blocks Keycloak SSO:** `filter_users_by_claims()` in `oidc_backend.py` checks `is_locked` and rejects locked users.
- **Evidence:** `authentication/api_views.py` — `LoginAPIView.post()` with `_handle_failed_attempt()`, `_disable_account()`, `_send_unlock_email()`, `_check_cooldown()`
- **Evidence:** `authentication/models.py` — `LoginAttempt.stage`, `CustomUser.is_locked`, `UnlockToken`
- **Evidence:** `authentication/unlock_views.py` — `account_unlock_view()`
- **Admin display:** User management table at `/users/` shows "Locked" (amber badge) for locked users. Toggle button changes to "Unlock" (orange icon). View `user_views.py:user_toggle_active_view()` clears `is_locked` when re-enabling.
- **Auto-sync from Keycloak:** `user_list_view()` fetches ALL Keycloak users in one batch call on page load, syncs `is_active` for any mismatches. Also syncs during OIDC login. See `keycloak_admin.py:get_all_users()` + `oidc_backend.py:filter_users_by_claims()`.

5. Tamper-proof audit log

Once an audit record is created, it cannot be edited or deleted. This is enforced by overridden `save()` and `delete()` methods.

- **Evidence:** `organizations/models.py` — lines 180-192

Every audit record captures the user's IP address. This is how we know not just WHO did something, but FROM WHERE.

When a user performs an action (create asset, edit user, delete record), the view captures the IP:

```
# Inside every view that creates an audit log:
ip = _get_client_ip(request)  # ← Helper function (explained in Part 7)
AuditLog.objects.create(
    performed_by=request.user,
    action='CREATE',
    model_name='Asset',
    object_id=asset.id,
    ip_address=ip,             # ← User's real IP is saved here
```

```
    ...
)
```

How does Django know the user's IP? You might think "where is `request.META['REMOTE_ADDR']` defined in our code?" The answer is: **nowhere.** Django provides it automatically.

When a browser sends a request to your website, the operating system and web server know who connected and from what IP address. Django takes that information and places it inside `request.META` — a dictionary that contains everything about the incoming request:

```
request.META == {
    "REMOTE_ADDR": "192.168.1.100",      # ← The user's IP (Django adds this)
    "HTTP_HOST": "moh.localhost:8000",    # ← The domain they visited
    "HTTP_USER_AGENT": "Mozilla/5.0...",  # ← Their browser type
    "REQUEST_METHOD": "GET",              # ← GET, POST, etc.
    ...
}
```

You never create this dictionary. Django creates it for you automatically on every single request. Your code only needs to READ from it:

```
request.META['REMOTE_ADDR']    # Returns: "192.168.1.100" (the user's IP)
```

Important — proxy awareness: When your site is behind a proxy (like Nginx or Cloudflare), `REMOTE_ADDR` shows the proxy's IP, not the user's. The real user IP is in a header called `HTTP_X_FORWARDED_FOR`. Our `_get_client_ip()` helper checks this header first, then falls back to `REMOTE_ADDR`. This ensures the audit log always captures the real user's IP whether or not a proxy is present. (See Part 22 for a full explanation of proxies.)

Why does audit log use numbers for user IDs (looks like numbers)?

You will notice the audit log stores `performed_by_id` as a number, not a name:

```
# organizations/models.py
performed_by_id = models.IntegerField()  # ← Just a number, not a ForeignKey
```

This is NOT a mistake. It is a deliberate security design:

Why not store the name?	Because...
Names can change	User "John" might change their name to "Jonathan". The audit log would show the wrong name

Why not store the name?	Because...
Users can be deleted	If a user is deleted, a ForeignKey would break. An IntegerField never breaks — the number stays forever
PostgreSQL limitation	Foreign keys cannot point to tables in DIFFERENT schemas. Users are in `public` schema, audit records are in each ministry's schema
Integrity	The number `5` will always mean "user with ID 5" — even if that user is later deleted, the record still shows who did it

Think of it like a prison ID number: The inmate's name might change (new surname), but their ID number never changes. The audit log uses the ID number because it is permanent.

What if you need to know the name? You can look it up:

```
from authentication.models import CustomUser
user = CustomUser.objects.get(id=5)  # The "5" from the audit log
print(user.get_full_name())          # Returns: "Amina Hassan"
```

Code evidence for tamper-proof design:

```
# organizations/models.py
class AuditLog(models.Model):
    performed_by_id = models.IntegerField()     # User ID (never changes)
    performed_by_name = models.CharField()      # Name at time of action
(snapshot)
    action = models.CharField()                 # CREATE, UPDATE, DELETE
    model_name = models.CharField()             # Which model was changed
    object_id = models.CharField()              # Which record was changed
    object_repr = models.CharField()            # Human-readable description
    old_value = models.JSONField(null=True)     # Before values (for updates)
    new_value = models.JSONField(null=True)     # After values (for
creates/updates)
    ip_address = models.GenericIPAddressField() # User's IP at time of action
    timestamp = models.DateTimeField(auto_now_add=True)  # When it happened

    def save(self, *args, **kwargs):
        raise PermissionError("Audit log is read-only!")  # ← Cannot edit

    def delete(self, *args, **kwargs):
        raise PermissionError("Audit log cannot be deleted!")  # ← Cannot delete
```

The `save()` and `delete()` methods are overridden to RAISE AN ERROR if anyone tries to modify or delete an audit record. This includes the Super Admin — NO ONE can change the audit log.

6. Session security

Session cookie expires when browser closes, cannot be read by JavaScript, and only sent over HTTPS in production.

- **Evidence:** `config/settings.py` — `SESSION_COOKIE_HTTPONLY = True`, `SESSION_EXPIRE_AT_BROWSER_CLOSE = True`

7. JWT token security

Short expiry (30 minutes), rotated refresh tokens, old tokens blacklisted.

- **Evidence:** `config/settings.py` — `SIMPLE_JWT` settings

8. Clickjacking protection

Our pages cannot be loaded inside an iframe on another website.

- **Evidence:** `X_FRAME_OPTIONS = 'DENY'`

9. Content sniffing protection

Browsers cannot guess content types — prevents certain types of attacks.

- **Evidence:** `SECURE_CONTENT_TYPE_NOSNIFF = True`

10. Cross-schema protection

User IDs are stored as IntegerField instead of ForeignKey because PostgreSQL cannot create foreign keys across schemas. This is a security-conscious design decision.

- **Evidence:** `performed_by_id = models.IntegerField()`

11. Pending access (approval workflow)

New users cannot just log in — an admin must approve them first.

- **Evidence:** `authentication/oidc_backend.py` — `create_user()` returns None

How PendingAccess works in detail:

This solves a real problem: what happens when someone has a Keycloak account (can log in) but doesn't have a profile in Django (no role, no ministry)?

```
Step 1: User logs in through Keycloak successfully
Step 2: Keycloak redirects to Django with a valid token
Step 3: Django looks for the user in its OWN database
Step 4: If NOT found → PendingAccess record is created
Step 5: User sees: "Your account is awaiting approval"
Step 6: Super Admin sees the request and approves/rejects it
Step 7: If approved → Django creates the actual user profile
Step 8: User can now log in successfully
```

Without PendingAccess, the user would just get "access denied" with no explanation.

Code evidence (authentication/oidc_backend.py):

```
try:
    user = CustomUser.objects.get(username=keycloak_username)
    return user  # Found them — let them in

except CustomUser.DoesNotExist:
    # Keycloak says they're real, but Django doesn't know them
    PendingAccess.objects.get_or_create(
        keycloak_id=keycloak_id,
        defaults={
            'username': keycloak_username,
            'email': keycloak_email,
        }
    )
    return None  # User sees "waiting for approval" message
```

13. Password validation

Users must have passwords at least 8 characters, cannot be common passwords, cannot be similar to username.

- **Evidence:** `AUTH_PASSWORD_VALIDATORS` in settings.py

14. Warning for default secret key

If production is running with a weak secret key, Django prints a warning.

- **Evidence:** Security checks in settings.py

Things that might look like issues (but are actually correct)

The panel may notice these things and think they are security problems. Here is the honest explanation for each:

Thing that might look like an issue	The real situation
Django's password fields are empty/null for Keycloak users	This is correct — passwords live in Keycloak, not Django. Django's password is set to an unusable value
PendingAccess records exist with no role assigned	These are users Keycloak knows about but Django hasn't approved yet. Normal and expected
LoginAttempt records keep growing	This is normal — they track every failed attempt. When a user successfully unlocks via email, ALL their LoginAttempt records are DELETED (clean slate). Records for users who never unlock remain as a permanent security record

Thing that might look like an issue	The real situation
Web "forgot password" might still point to Django's reset page	This is a real issue to fix — redirect it to Keycloak's forgot-password page instead
`/api/auth/login/` can be called without a token	This is correct — you obviously cannot require a token BEFORE logging in
Some endpoints return 403 (Forbidden) instead of 404	This is intentional — returning 404 would reveal whether the URL exists; 403 is more secure
Multiple failed login attempts from the same IP	Keycloak blocks after 5 failures (web). Django blocks with a 3-stage progressive system: WARNING (1–3), COOLDOWN (4–5, 5-min lock), DISABLED (6+, permanent lock with email unlock). Triple protection covers all paths

---

PART 9: DATABASE DESIGN — How our tables are organized

> **This part covers:** A quick overview of which tables live where.
> For the full deep dive with every column explained, see **Part 21: Database Deep Dive**.

Public schema (shared across ALL ministries):

```
tenants_ministry     → List of all ministries (MOH, MOF, etc.)
tenants_domain       → Domain to ministry mapping (moh.localhost → MOH)
authentication_customuser  → All users from all ministries
django_session       → Login sessions
...
```

Each ministry schema (one per ministry):

```
assets_asset           → Assets belonging to this ministry
assets_assetcategory   → Asset categories (ICT, Furniture, etc.)
organizations_orgunit  → Org hierarchy (Ministry → Agency → Facility)
organizations_masterdata → Dropdown options (funding sources, etc.)
organizations_auditlog → Audit trail for this ministry
...
```

Why this design?

Question: "Why not just have one table with a `ministry_id` column?"

Answer: Two reasons:

1. **Security** — If a programmer forgets to filter by `ministry_id`, they accidentally see ALL ministries' data. With schemas, they can ONLY see their own ministry's data. The database enforces this, not the code.
2. **Performance** — Each schema's tables are smaller. Querying 1,000 assets in MOH's table is faster than querying 100,000 assets in one big table and filtering by ministry_id.

PART 10: OTHER TECHNOLOGIES — What they are and where we use them

Django REST Framework (DRF)

What: A library that makes building APIs easy. **Where:** Every `api_views.py` file uses DRF's `APIView`, `Response`, and `status` classes. **Why:** Writing JSON APIs by hand is tedious. DRF handles JSON conversion, error handling, authentication, and permissions automatically.

django-cors-headers

What: Allows the Flutter mobile app (running on a phone) to call our Django API (running on a laptop). **Where:** `MIDDLEWARE` includes `corsheaders.middleware.CorsMiddleware`. Settings include `CORS_ALLOW_ALL_ORIGINS`. **Why:** Without CORS, the mobile browser would block API calls because they come from different addresses.

mozilla-django-oidc

What: Handles the Keycloak OIDC handshake (redirects, code exchange, token verification). **Where:** `urlpatterns` includes `path('oidc/', include('mozilla_django_oidc.urls'))`. Settings include `OIDC_OP_*_ENDPOINT` URLs. **Why:** Writing OIDC from scratch is extremely complex and error-prone. Mozilla (the Firefox company) maintains this library.

drf-yasg

What: Generates Swagger API documentation automatically. **Where:** `/api/docs/` in the browser shows the documentation page. **Why:** Other groups (Group 2-10) need to understand our API. Instead of writing documentation manually, drf-yasg reads our code and creates an interactive page.

djangorestframework-simplejwt

What: Creates and verifies JWT tokens for the mobile app. **Where:** `SIMPLE_JWT` settings in settings.py. Used in `LoginAPIView`, `RefreshTokenAPIView`, etc. **Why:** JWT is the standard way for mobile apps to authenticate. SimpleJWT handles token creation, refresh, blacklisting, and verification.

python-decouple

What: Reads secrets from `.env` file. **Where:** `SECRET_KEY = config('SECRET_KEY')` in settings.py. **Why:** You never hardcode passwords or secret keys in your code. If someone views your code on GitHub, they can't steal your secrets.

requests

What: A library for making HTTP calls from Python. **Where:** Used to call Keycloak Admin API (creating users in Keycloak from Django). **Why:** Django needs to communicate with Keycloak server-to-server.

pytest-django

What: A testing framework. **Where:** Currently not used extensively (test files are placeholders). **Why:** Allows writing automated tests that verify the system works correctly.

PART 11: DEPLOYMENT TO CLOUD — What, where, how

What is "the cloud"?

The cloud = someone else's computer that runs 24/7.

Instead of running `python manage.py runserver` on your laptop (which you turn off at night), you rent a server from a company like:

- **AWS** (Amazon Web Services)
- **DigitalOcean** ($5/month basic server)
- **Linode** or **Vultr**

What you need for production:

```
[User's Browser]
       ↓
[Domain Name: moh.mof.go.tz]
       ↓
[DNS — points domain to server IP]
       ↓
[NGINX or Apache — web server]
  (handles HTTPS certificates, static files, security)
       ↓
[Gunicorn or uWSGI — runs Django]
       ↓
[PostgreSQL Database]
```

Do you use 0.0.0.0 in production?

NO. In production:

- NGINX listens on `0.0.0.0:80` (HTTP) and `0.0.0.0:443` (HTTPS with SSL)
- Django runs behind NGINX, listening ONLY on `127.0.0.1:8000`
- NGINX forwards (proxies) requests from the internet to Django

The 0.0.0.0:8000 thing is ONLY for your development testing when you access Django from your phone.

What about domains?

In production:

```
https://moh.mof.go.tz    → Routes to Ministry of Health's schema
https://mof.mof.go.tz    → Routes to Ministry of Finance's schema
https://admin.mof.go.tz  → Routes to public schema (Super Admin)
```

Each domain gets an SSL certificate (for HTTPS — the padlock icon).

PART 12: JWT TOKENS — Access token, refresh token, how they work

What is a JWT token?

JWT stands for **JSON Web Token**. It is a string of text that proves who you are. Think of it like a **digital ID card** that the mobile app carries around.

The mobile app logs in once, gets this ID card, and shows it with every request so Django knows: "Oh, this is user X, they are a Ministry Admin for MOH."

What does a JWT look like?

```
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxfQ.s5xLm8vF9kZn6w
├──────────────────┘ ├──────────────────┘ ├─────────────┘
   PART 1               PART 2               PART 3
   HEADER               PAYLOAD              SIGNATURE
```

Three parts separated by dots:

Part	Name	What it contains	Can you read it?
First part	**Header**	What type of token and how it was signed (e.g., HS256)	Yes (base64 decoded)
Second part	**Payload**	The actual data: user_id, role, ministry_schema, expiration time	Yes (base64 decoded)
Third part	**Signature**	A cryptographic seal that proves the token was issued by Django and not forged	No (requires secret key)

Important: Parts 1 and 2 are just encoded (base64), NOT encrypted. Anyone can decode them and see the contents. But they CANNOT change them because the signature (part 3) would break.

How to SEE inside a JWT token:

Go to jwt.io and paste your token. It will show you:

```
// HEADER
{
  "alg": "HS256",
```

```
  "typ": "JWT"
}

// PAYLOAD
{
  "user_id": 1,
  "role": "MINISTRY_ADMIN",
  "ministry_schema": "moh_schema",
  "exp": 1712345678,     // ← Expiration time (Unix timestamp)
  "iat": 1712343878      // ← Issued at time
}
```

Access token vs Refresh token

We have TWO tokens, not one:

```
ACCESS TOKEN (short life)
  Purpose: Proves who you are for API calls
  Valid for: 30 minutes
  Used in: Every API request header
  What if stolen? Only useful for 30 minutes

REFRESH TOKEN (longer life)
  Purpose: Gets you a NEW access token when the old one expires
  Valid for: 1 day
  Used in: POST /api/auth/refresh/
  What if stolen? Can be blacklisted (invalidated)
```

The flow:

```
Mobile app logs in
  → Gets access token (30 min) + refresh token (1 day)
  → Uses access token for 30 minutes of API calls
  → Token expires (30 min passed)
  → Mobile app calls: POST /api/auth/refresh/ with refresh token
  → Django gives a NEW access token (another 30 min)
  → Sometimes also gives a NEW refresh token (ROTATE_REFRESH_TOKENS = True)
  → Old refresh token is blacklisted (cannot be used again)
  → Cycle continues until 1 day passes
  → After 1 day, user must log in again
```

Code evidence — Where we configure token lifetimes:

```
# config/settings.py — LINE 235-248
from datetime import timedelta

SIMPLE_JWT = {
```

```
    'ACCESS_TOKEN_LIFETIME':  timedelta(minutes=30),   # ← 30 minutes
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),        # ← 1 day
    'ROTATE_REFRESH_TOKENS':  True,                     # ← Give new refresh token
each time
    'BLACKLIST_AFTER_ROTATION': True,                   # ← Old one stops working
    'ALGORITHM': 'HS256',                               # ← How we sign it
    'SIGNING_KEY': SECRET_KEY,                          # ← Uses Django's secret
key
    'AUTH_HEADER_TYPES': ('Bearer',),                   # ← Token prefix in header
}
```

How to change the token duration (if panel asks):

```
# Change 30 minutes to 2 hours:
'ACCESS_TOKEN_LIFETIME': timedelta(hours=2)

# Change 1 day to 7 days:
'REFRESH_TOKEN_LIFETIME': timedelta(days=7)
```

Why 30 minutes? Why not 1 year?

Short access tokens = more secure. If someone steals your token, they can only use it for 30 minutes. After that, it's useless. The refresh token lasts longer but can be blacklisted if stolen.

Code evidence — Where the token is created with extra data:

```
# authentication/api_serializers.py — LINE 7-18
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # These extra fields are embedded INSIDE the token
        token['role'] = user.role
        token['ministry_schema'] = user.ministry_schema or ''
        token['full_name'] = user.get_full_name() or user.username
        return token
```

This means when the mobile app decodes the token, it can read the user's role and ministry without making an extra API call.

Code evidence — Mobile app login endpoint:

```
# authentication/api_urls.py — LINE 18
path('auth/login/', LoginAPIView.as_view(), name='api_login'),
# POST /api/auth/login/  → returns { access, refresh, user }
```

Code evidence — How the login API creates the token

When the mobile app or other group calls `POST /api/auth/login/`, this code runs:

```
# authentication/api_views.py
class LoginAPIView(APIView):
    permission_classes = [AllowAny]  # No token needed to log in

    def post(self, request):
        username = request.data.get('username')
        password = request.data.get('password')

        user = authenticate(username=username, password=password)
        if user:
            # Create the token with extra info baked in
            refresh = RefreshToken.for_user(user)
            refresh['role'] = user.role                # Role goes INTO the token
            refresh['ministry_schema'] = user.ministry_schema  # Ministry too

            return Response({
                'access': str(refresh.access_token),
                'refresh': str(refresh),
                'user': {
                    'username': user.username,
                    'role': user.role,
                    'ministry_schema': user.ministry_schema
                }
            })
        return Response({'error': 'Invalid credentials'}, status=401)
```

What this means: When a user logs in, Django:

1. Checks the username/password (authenticate)
2. Creates a JWT token with `RefreshToken.for_user(user)`
3. Adds `role` and `ministry_schema` as extra fields INSIDE the token
4. Returns the token AND the user info in one response

The Flutter app or other group never needs to make a second call to find out the user's role — it's already in the token.

PART 13: INTEGRATION WITH OTHER GROUPS — How they use our API

Our role

We are Group 1. Groups 2-10 depend on us. They need to call our API to:

- Verify users (is this token valid? what role do they have?)
- Get asset data for their systems
- Get organization data

What do we share with them?

We share our **API endpoint URLs** and a **sample token**. We do NOT share our source code or database.

Scenario 1: They just need to check if a user is real

Their system receives a token from a mobile user. They call our API to verify it.

What we give them:

```
API Endpoint: POST http://moh.platform.go.tz/api/auth/verify-token/

Headers:
  Authorization: Bearer <the_token_their_user_provided>
  Content-Type: application/json

Response (if valid):
{
  "valid": true,
  "role": "MINISTRY_ADMIN",
  "ministry_schema": "moh_schema",
  "full_name": "Amina Hassan",
  "email": "amina@moh.go.tz"
}

Response (if invalid/expired):
{
  "valid": false,
  "message": "Token is invalid or expired"
}
```

They don't need our database. They don't need Django. They just need to make an HTTP request to our URL.

Scenario 2: They want asset data for reporting

What we give them:

```
API Endpoint: GET http://moh.platform.go.tz/api/assets/?status=ACTIVE

Headers:
  Authorization: Bearer <valid_token>

Response:
{
  "count": 150,
  "results": [
    {"id": 1, "asset_number": "MOH-ICT-0001", "name": "Dell Laptop", ...},
    ...
```

```
  ]
}
```

Do we give them a token or do they get their own?

They get their own token. Here's how:

```
1. We create a user account for their system (like "group2_service_account")
2. We give them: username + password
3. Their system calls: POST /api/auth/login/ with those credentials
4. Our API returns: { access_token, refresh_token, user_info }
5. They use the access_token in all subsequent API calls
6. When token expires, they call /api/auth/refresh/ to get a new one
```

The simple analogy:

> The **API** is the door. The **token** is the key to that door. You share the **door** (API URL) with other groups. They get their **own key** (token) by logging in themselves.

You do NOT hand them a token directly. You never say "here is your token." That would be like giving someone your house key — they'd have YOUR identity, not their own.

Here is exactly what you give them (one time, via WhatsApp or email):

```
1. Our server address:      http://172.16.20.232:8000
2. Our API docs link:       http://172.16.20.232:8000/api/docs/
3. A test account:          username: moh_admin
                            password: Admin@123
```

And here is what THEY do with it:

```
Step 1 — They call your login API themselves:
  POST http://172.16.20.232:8000/api/auth/login/
  {"username": "moh_admin", "password": "Admin@123"}

Step 2 — Your system gives THEM their own token:
  {"access": "eyJ...their token...", "user": {"role": "MINISTRY_ADMIN"}}

Step 3 — They use that token on every request:
  GET http://172.16.20.232:8000/api/assets/
  Header: Authorization: Bearer eyJ...their token...
```

So the flow is: You give them the door address → they knock (login) → the system gives them their own key → they use that key on every future knock.

Do other groups go to the Keycloak login page?

It depends on the group. There are two types of integration:

Type A — Groups WITHOUT their own login page (Group 2, 3, 4, etc.) These groups built their own backend and database but **did not build a login system**. Their users log in through **our Keycloak SSO page** — the same government-branded login page our own web users see:

```
User → opens Group 2's app → redirected to OUR Keycloak login page
  → types password on Keycloak → redirected back to Group 2's app
  → Group 2's app calls our API to get the user's role + ministry
```

The user leaves Group 2's site, authenticates on ours, then returns. This is the SSO flow.

Type B — Groups WITH their own login page (Group 5, 6, etc.) These groups built their own login form. Their backend calls our **API login endpoint** directly:

```
User → types password on Group 5's login form
  → Group 5's backend calls POST /api/auth/login/ with our credentials
  → Our API returns JWT token → Group 5 lets the user in
```

Our Keycloak page never appears. This is the API flow.

Both approaches work because every user is created in our system. The difference is only whether the user sees our Keycloak login page (Type A) or the other group's own form (Type B).

Important — How SSO works across multiple groups:

If ALL groups use the SSO approach (Type A), the user only logs in **once** in their browser, then can access every group's system without typing their password again:

```
Browser Tab 1: User logs in on Group 2's system
  → Redirected to Keycloak → logs in → redirected back to Group 2

Browser Tab 2: User opens Group 5's system (same browser)
  → Redirected to Keycloak → Keycloak sees "already logged in"
  → Immediately redirects back to Group 5 — NO password prompt

Browser Tab 3: User opens Group 10's system
  → Same — instant access, no login needed
```

This works because Keycloak stores a session cookie in the browser. Every group's system redirects to Keycloak, Keycloak checks the cookie, sees an active session, and sends the user straight back with a new token for that group.

This is the whole purpose of SSO. One username and password. Access to all 10 systems.

What if they want a public API (no token required)?

You already have this ability. Look at your views — some use `AllowAny`, some use `IsAuthenticated`:

```
# PUBLIC — no token needed (currently only the login endpoint):
class LoginAPIView(APIView):
    permission_classes = [AllowAny]

# PROTECTED — token required:
class AssetListCreateAPIView(APIView):
    permission_classes = [IsAuthenticated]
```

If another group asks you to make a specific endpoint public — for example, a read-only list of asset categories — you change one line:

```
class AssetCategoryListAPIView(APIView):
    permission_classes = [AllowAny]  # ← Change this line
```

Should you make things public? Only read-only, non-sensitive data. Never user data, never audit logs, never financial data. The rule is: if the data could embarrass the government or reveal security information, it must stay protected.

What if they use different technology?

It doesn't matter. Our API speaks HTTP + JSON. Every programming language can do this:

Their technology	How they call our API
Python	`requests.get(url, headers={'Authorization': 'Bearer ...'})`
PHP	`curl` or `file_get_contents`
Java	`HttpURLConnection` or `OkHttp`
JavaScript	`fetch(url, {headers: {'Authorization': 'Bearer ...'}})`
C# .NET	`HttpClient`
Any language	HTTP is universal — every language supports it

Sample Python code they would write:

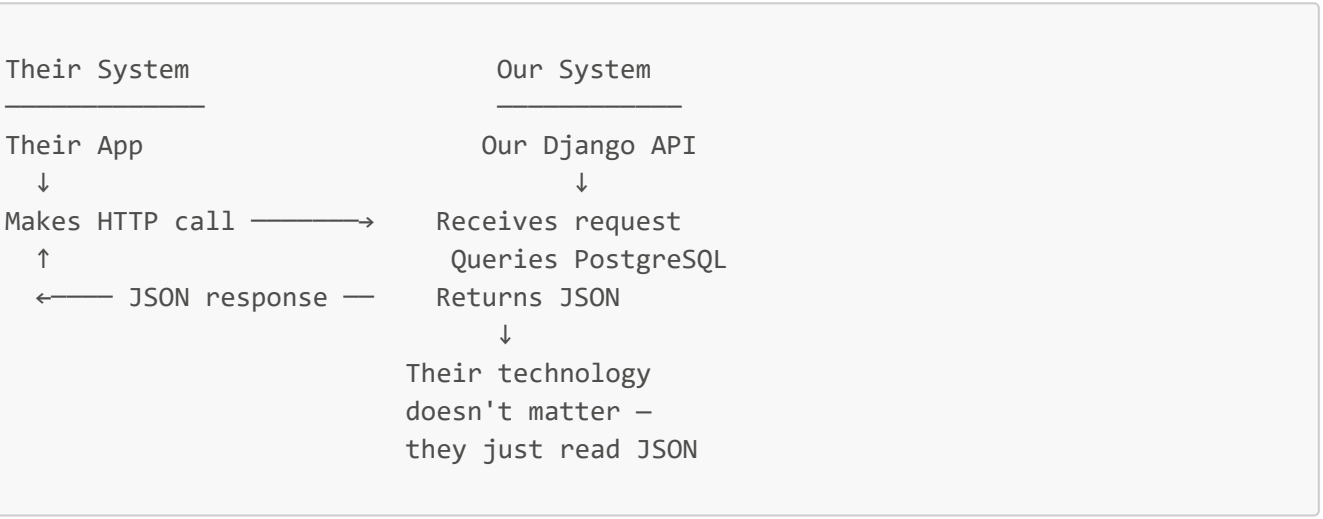
```
import requests

# Step 1: Login to get a token
login_response = requests.post(
    'http://moh.platform.go.tz/api/auth/login/',
    json={'username': 'group2_user', 'password': 'given_password'}
)
token = login_response.json()['access']
```

```
# Step 2: Use the token to get asset data
headers = {'Authorization': f'Bearer {token}'}
assets_response = requests.get(
    'http://moh.platform.go.tz/api/assets/',
    headers=headers
)
assets = assets_response.json()
print(f"Total assets: {assets['count']}")
```

What if they don't use PostgreSQL?

They don't need to. Our database is inside our system. They never connect to our database. They only connect to our API (which is an HTTP endpoint). Our API queries our PostgreSQL database and returns JSON. They receive JSON. Their database can be anything — MySQL, MongoDB, SQL Server, or even Excel.



What if they need a new feature or data?

They tell us: "We need to get all assets that expired this month." We add a new API endpoint or a filter parameter. We update the Swagger documentation at `/api/docs/`. They check the documentation and start using the new endpoint.

What happens if we create a user but they don't add it?

They will call `/api/auth/verify-token/` with that user's token and get back: `{"valid": false, "message": "User not found"}`. They will know the user doesn't exist in our system.

What happens if they add a user but we don't?

The user will authenticate at Keycloak successfully, but when Keycloak redirects to Django, Django will say: "This user is not registered in our system." A PendingAccess record will be created and the user will see: "Your account is not yet registered. Contact your administrator."

Where is our API documentation?

```
http://localhost:8000/api/docs/
```

This is a Swagger UI page that shows EVERY endpoint with:

- The URL path (e.g., /api/assets/)
- What HTTP method to use (GET, POST, etc.)
- What data to send (JSON format)
- What you get back (JSON format)
- A "Try it out" button to test the API live

We tell other groups: "Go to this URL. All our APIs are documented there."

How is this page automatically created? In your `settings.py` and `config/urls.py`, you have `drf-yasg` installed, which reads your code and builds the documentation page:

```
# config/settings.py — drf_yasg is in INSTALLED_APPS
INSTALLED_APPS = [
    ...
    'drf_yasg',  # ← This is what creates the automatic documentation
]

# config/urls.py — The Swagger page configuration
from drf_yasg.views import get_schema_view
from drf_yasg import openapi

schema_view = get_schema_view(
    openapi.Info(
        title="GovAsset Platform API",
        default_version='v1',
        description="REST API for the Government Asset Management Platform",
    ),
    public=True,  # Anyone can view the docs (no login required)
)

urlpatterns += [
    path('api/docs/', schema_view.with_ui('swagger')),
]
```

Every time you add a new API endpoint, Swagger automatically adds it to the documentation page. You never write documentation manually.

This means: If another group asks "Can you add an API for X?" you:

1. Add the new endpoint to `api_urls.py` and `api_views.py`
2. Restart the server
3. The new endpoint appears on the Swagger page automatically
4. Tell them: "Check the docs again, it's there now"

What credentials do we give other groups for testing?

```
API Base URL: http://192.168.100.18:8000/api/
Username: test_group_user
Password: Test@123
```

They use these to log in and get a token. Simple.

Who gives the token to other groups? Django or Keycloak?

Django gives the token. Here is why:

```
Token type: JWT (JSON Web Token)
Issued by: Django (using SimpleJWT library)
Token contains: user_id, role, ministry_schema, expiration time
Signed with: Django's SECRET_KEY

Keycloak also issues tokens, but those Keycloak tokens are ONLY for
web browser login. The tokens we give other groups for API access
are Django tokens, created by SimpleJWT.
```

Important: Keycloak tokens and Django JWT tokens are DIFFERENT. They have different formats, different signatures, and different purposes:

```
KEYCLOAK TOKEN (for web login)
  — Used between: Browser → Django → Keycloak
  — Validates: "This user logged in through Keycloak"
  — Format: OIDC id_token (longer, more complex)

DJANGO JWT TOKEN (for API access)
  — Used between: Mobile app → Django, Other groups → Django
  — Validates: "This user has permission to call our API"
  — Format: SimpleJWT access_token (shorter, simpler)
```

What does an actual API endpoint look like? (Real example from our code)

When another group's developer asks "What URL do I call?" — here is exactly what we give them:

```
BASE URL (during development):
  http://192.168.100.18:8000/api/

BASE URL (in production):
  https://moh.platform.go.tz/api/

ENDPOINT: Get list of assets
  GET  /api/assets/

ENDPOINT: Get one specific asset
```

```
  GET  /api/assets/{id}/
  Example: GET /api/assets/5/

ENDPOINT: Create a new asset
  POST  /api/assets/

ENDPOINT: Update an asset
  PUT  /api/assets/{id}/

ENDPOINT: Delete an asset
  DELETE  /api/assets/{id}/
```

But we don't give them all endpoints individually! We give them ONE URL:

> **"Go to http://192.168.100.18:8000/api/docs/ — all our APIs are listed there with a 'Try it out' button."**

The Swagger page (`/api/docs/`) automatically lists EVERY endpoint that exists. If we add a new endpoint tomorrow, it appears on the Swagger page automatically. The other group does NOT need us to send them a new document.

What attributes/fields do other groups need to send? (How do they know?)

Again, Swagger tells them. Here is what they see on the Swagger page for "Create Asset":

```
POST /api/assets/

Request body (JSON):
{
  "name": "Dell Latitude 5420",           // Required — the asset name
  "asset_number": "MOH-ICT-0042",         // Required — unique ID
  "category": 3,                          // Required — category ID from dropdown
  "status": "ACTIVE",                     // Required — one of: ACTIVE,
UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
  "description": "Laptop for ICT officer", // Optional
  "serial_number": "SN123456789",         // Optional
  "purchase_date": "2024-01-15",          // Optional — format: YYYY-MM-DD
  "purchase_cost": "2500000.00",          // Optional — decimal number as string
  "location": "MOH HQ, Room 305",         // Optional
  "assigned_to": "amina@moh.go.tz",       // Optional — user email
  "org_unit": 12                          // Optional — org unit ID
}
```

The Swagger page also shows:

- Which fields are required vs optional
- What data type each field is (string, number, date)
- What the valid values are for dropdown fields
- Example values
- The exact JSON format

How do other groups access our API during development?

During development, we are all on the same college/local WiFi. Here is the setup:

```
OUR LAPTOP (Group 1):
  IP: 192.168.100.18
  Running: python manage.py runserver 0.0.0.0:8000
  API available at: http://192.168.100.18:8000/api/

OTHER GROUP'S LAPTOP (Group 2-10):
  Also on the same WiFi
  They open their browser/postman/terminal
  They call: GET http://192.168.100.18:8000/api/assets/
  They get back: JSON data
```

What if our IP changes? We tell them: "My IP changed, here is the new one: http://192.168.100.XXX:8000/api/"

What if we are on different WiFi? We cannot access each other. We would need to either:

1. Both connect to the same WiFi (college network)
2. Use a tool like ngrok (gives us a public URL that forwards to our laptop)
3. Deploy to a cloud server (production mode)

Complete example: A group wants to integrate with us from scratch

Here is the ENTIRE process from start to finish:

```
STEP 1: We meet with the other group
  Them: "We want to display our ministry's assets in our dashboard."
  Us: "OK. Here is our API base URL and your test credentials."

STEP 2: We give them:
  API Base URL: http://192.168.100.18:8000/api/
  Username: group2_service_account
  Password: Test@123
  Swagger Docs: http://192.168.100.18:8000/api/docs/

STEP 3: They test the API manually (in their browser or Postman)
  They open: http://192.168.100.18:8000/api/docs/
  They click "Try it out" on GET /api/assets/
  They see the JSON response with actual data
  They understand the format

STEP 4: They write code in THEIR system
  Let's say they use PHP (their system is built in PHP).

  They write this code:
  ```php
  <?php
```
```

```
// Step 1: Login to get token
$login = http_post('http://192.168.100.18:8000/api/auth/login/', [
    'username' => 'group2_service_account',
    'password' => 'Test@123'
]);
$token = json_decode($login)->access;

// Step 2: Get assets
$assets = http_get('http://192.168.100.18:8000/api/assets/', [
    'Authorization: Bearer ' . $token
]);
$data = json_decode($assets);

// Step 3: Display in their dashboard
foreach ($data->results as $asset) {
    echo $asset->name . " - " . $asset->status;
}
?>
```

STEP 5: They test. It works. They are integrated.

STEP 6 (later): We deploy to production. We tell them: "The new URL is https://moh.platform.go.tz/api/" They update one line in their code. Everything still works.

```
### Do we also give them access to Keycloak?

**Depends on the group type:**

**For groups WITHOUT their own login (Type A):**
Yes — their users will log in through our Keycloak SSO page. We register their app
as an OIDC client in Keycloak and give them a Client ID + Client Secret. Their
backend never handles passwords.

**For groups WITH their own login (Type B):**
No — they call our API login endpoint directly with username + password. Keycloak
is never involved. We give them test credentials + the API base URL.
```

Type A (no login): User → Keycloak SSO page → token → Group 2's app Type B (has login): User → Group 5's form → API call → our JWT → Group 5's app

Both types: the user is authenticated by US. The difference is only the UI they see.

```
### What if our system is down? How does it affect other groups?
```

Their System → calls our API → no response (our server is down) → Their dashboard shows "Service unavailable" → Their users see a timeout error → They cannot get fresh data until our system is back up → They CAN use their last cached data (if they saved it)

```
This is why we need production deployment (24/7 server). During development, we
only run when we are testing.

### How does the mobile (Flutter) app use our API?

The Flutter app is ALSO an "other group" — it just happens to be built by our
team, not another group. The Flutter app:
```

1. Has our API base URL hardcoded: http://192.168.100.18:8000/api/
2. Shows a login screen: username + password fields
3. Calls POST /api/auth/login/ with those credentials
4. Gets back a JWT token (access + refresh)
5. Stores the token on the phone (secure storage)
6. Adds Authorization: Bearer to EVERY request
7. When token expires, silently refreshes it
8. If refresh fails, shows login screen again

```
The Flutter app NEVER opens a browser to Keycloak. It uses the direct API login
(same as other groups would).

### Complete list of ALL API endpoints other groups can use

These are defined in `authentication/api_urls.py` and `assets/api_urls.py`:

| Method | URL | Purpose |
|---|---|---|
| POST | `/api/auth/login/` | Get JWT token (username + password) |
| POST | `/api/auth/refresh/` | Get new access token (using refresh token) |
| POST | `/api/auth/logout/` | Blacklist token (log out) |
| POST | `/api/auth/verify-token/` | Check if a token is valid |
| GET | `/api/auth/profile/` | Get current user's profile |
| GET/POST | `/api/assets/` | List all assets / Create new asset |
| GET/PUT/DELETE | `/api/assets/{id}/` | Get / Update / Delete one asset |
| GET | `/api/org-units/` | List organization units |
| GET | `/api/dashboard/stats/` | Dashboard statistics |
| GET | `/api/audit-log/` | View audit log entries |

All these are documented at `/api/docs/` with example requests and responses.

### Where is the API configuration in our code?

The API routing is configured in TWO files:
```

````
**File 1 — `authentication/api_urls.py` (lists every API endpoint):**

```python
# This file maps URLs to their handler views
urlpatterns = [
    # Authentication
    path('auth/login/', LoginAPIView.as_view()),
    path('auth/logout/', LogoutAPIView.as_view()),
    path('auth/refresh/', RefreshTokenAPIView.as_view()),
    path('auth/verify-token/', VerifyTokenAPIView.as_view()),
    path('auth/me/', MeAPIView.as_view()),

    # Assets
    path('assets/', AssetListCreateAPIView.as_view()),
    path('assets/<int:asset_id>/', AssetDetailAPIView.as_view()),
    path('assets/categories/', AssetCategoryListAPIView.as_view()),

    # Organization
    path('org-units/', OrgUnitListAPIView.as_view()),

    # Audit logs
    path('audit-logs/', AuditLogListAPIView.as_view()),

    # Dashboard statistics
    path('dashboard/stats/', DashboardStatsAPIView.as_view()),
]
```
````

File 2 — `config/urls.py` (connects the API to the main project):

```
urlpatterns = [
    # ... all the web page URLs ...

    # This ONE line attaches ALL API endpoints:
    path('api/', include('authentication.api_urls')),

    # Swagger documentation
    path('api/docs/', schema_view.with_ui('swagger')),
]
```

The line `path('api/', include(...))` puts `api/` in front of every URL in `api_urls.py`. So `auth/login/` becomes `/api/auth/login/`. That is why all your APIs start with `/api/`.

How other groups see this: They never look at these files. They just go to `/api/docs/` and see everything listed automatically.

NEW SECTION: Step-by-Step — How to Share Your API with Other Groups RIGHT NOW

This is the practical guide for contacting groups 2-10 and getting them connected to your API.

Step 1: Start your server (make it accessible from other devices)

```
# IMPORTANT: 0.0.0.0 makes Django listen on ALL network interfaces
# Without this, other computers cannot reach you
python manage.py runserver 0.0.0.0:8000
```

Step 2: Deploy to Railway (recommended — permanent URLs, no ngrok needed)

Railway hosts both Django and Keycloak with permanent URLs that never change. Group 2 sets up once and is done. Both services run 24/7 — your laptop can be off.

What you deploy:

```
Railway Project "govasset"
  ├── Service 1: Django API    → https://govasset-api.up.railway.app   (permanent)
  ├── Service 2: PostgreSQL    → Railway managed (shared between Django +
Keycloak)
  └── Service 3: Keycloak      → https://govasset-keycloak.up.railway.app
(permanent)
```

Deploy PostgreSQL (do this first)

Both Django and Keycloak need a database. Railway provides PostgreSQL as a plugin.

1. In Railway dashboard → **New** → **Database** → **Add PostgreSQL**
2. Wait for it to provision. Copy the `DATABASE_URL` connection string (looks like `postgresql://...`)
3. You'll use this for both Django and Keycloak

Deploy Django

1. Push your code to GitHub
2. Railway → **New Project** → **Deploy from GitHub repo** → select your repo
3. Railway auto-detects Django. Set these in the **Settings** tab:
   - **Build command:** `pip install -r requirements.txt && python manage.py collectstatic --noinput && python manage.py migrate`
   - **Start command:** `gunicorn config.wsgi`
4. In the **Variables** tab, add these environment variables:

Variable	Value	Why
`DATABASE_URL`	The Railway PostgreSQL URL from above	Connect Django to the database

Variable	Value	Why
SECRET_KEY	Generate a random 50-char key	Django's cryptographic signing
DJANGO_SETTINGS_MODULE	config.settings	Tell Django which settings to use
DEBUG	False	Never run DEBUG in production
ALLOWED_HOSTS	.up.railway.app	Allow Railway's domain
CSRF_TRUSTED_ORIGINS	https://govasset-api.up.railway.app	Allow POST requests from Railway
KEYCLOAK_SERVER_URL	https://govasset-keycloak.up.railway.app	Django talks to Keycloak
KEYCLOAK_CLIENT_ID	govasset-django	Your OIDC client in Keycloak
KEYCLOAK_CLIENT_SECRET	(the secret from Keycloak)	Your OIDC client secret
PLATFORM_BASE_URL	https://govasset-api.up.railway.app	For emails, redirects

5. Railway deploys automatically. Your Django URL will be: https://govasset-api.up.railway.app

If you get SSL errors when Django tries to contact Keycloak: Add this variable too:

- OIDC_OP_JWKS_ENDPOINT = https://govasset-keycloak.up.railway.app/realms/govasset/protocol/openid-connect/certs

---

Deploy Keycloak

Keycloak runs as a Docker container on Railway.

1. In the same Railway project → **New** → **Start a new service** → **Docker**
2. For image, paste: quay.io/keycloak/keycloak:26.1
3. In the **Settings** tab, set the **Port** to 8080 (Keycloak's default)
4. In the **Variables** tab, add these environment variables:

Variable	Value	Why
KC_DB	postgres	Tell Keycloak to use PostgreSQL
KC_DB_URL	jdbc:postgresql://... (convert Railway's DATABASE_URL to JDBC format)	Keycloak's database connection
KC_DB_USERNAME	(from Railway PostgreSQL, usually postgres)	Database user
KC_DB_PASSWORD	(from Railway PostgreSQL)	Database password

Variable	Value	Why
KC_HOSTNAME	https://govasset-keycloak.up.railway.app	Keycloak's public URL
KC_PROXY	edge	Required behind Railway's proxy — without this, redirects break
KC_HTTP_ENABLED	true	Allow HTTP internally (Railway handles HTTPS at edge)
KEYCLOAK_ADMIN	admin	Admin username for Keycloak admin panel
KEYCLOAK_ADMIN_PASSWORD	Admin@123 (or a strong password)	Admin password

5. Deploy Keycloak. Your Keycloak URL: https://govasset-keycloak.up.railway.app
6. Open the admin panel at https://govasset-keycloak.up.railway.app/admin and log in with admin / Admin@123

Configure both services after deployment

Step A — Import your Keycloak realm

Since your Keycloak locally has the govasset realm with clients and users, you have two options:

Option 1 — Export from local, import to Railway (best):

1. On your local Keycloak: go to **Realm Settings** → **Export** → download JSON
2. On Railway Keycloak admin: go to **Add Realm** → upload the JSON file
3. All your clients (govasset-django, group2-app) and users are imported

Option 2 — Recreate manually (if export fails):

1. Create the govasset realm
2. Create the govasset-django client (Client authentication: ON, Standard flow: ON)
3. Copy the Client Secret — you need this for Django's KEYCLOAK_CLIENT_SECRET env var
4. Create the group2-app client for Group 2's SSO
5. Create test users (moh_admin, mof_admin, etc.) in the realm

Step B — Update Django's env var with Keycloak secret

After fixing the Client Secret in Keycloak admin, update Django's environment variable:

- KEYCLOAK_CLIENT_SECRET = (the copied secret)

Railway will restart Django automatically when you change the variable.

Step C — Configure Keycloak redirect URIs

For each client in Keycloak:

Client	Setting	Value
`govasset-django`	Valid Redirect URIs	`https://govasset-api.up.railway.app/*`
`govasset-django`	Valid Post Logout Redirect URIs	`https://govasset-api.up.railway.app/*`
`govasset-django`	Web Origins	`https://govasset-api.up.railway.app`
`group2-app`	Valid Redirect URIs	`https://group2-callback-url.com/*`
`group2-app`	Valid Post Logout Redirect URIs	`https://group2-callback-url.com/*`

Verify everything works

1. **Keycloak is up:** Visit `https://govasset-keycloak.up.railway.app/realms/govasset/` → should show realm info
2. **Django is up:** Visit `https://govasset-api.up.railway.app/api/docs/` → should show Swagger
3. **Login works:** Visit `https://govasset-api.up.railway.app/login/` → should redirect to Keycloak
4. **API works:** Call `POST https://govasset-api.up.railway.app/api/auth/login/` with test credentials → should return token

Cost: Free tier — all three services run 24/7. Your laptop can be off.

Fallback — ngrok for quick local testing (before Railway deployment)

If you haven't deployed to Railway yet and need to test immediately, use ngrok. Note that **free ngrok only supports one tunnel** — you need two different tunneling tools for both services.

Option A — ngrok for Keycloak + bore/localtunnel for Django:

```
Terminal 1: python manage.py runserver 0.0.0.0:8000
Terminal 2: kc.bat start-dev --http-port=8180
Terminal 3: ngrok http 8180                    → https://keycloak.ngrok-free.app
Terminal 4: bore local 8000 --to bore.pub      → https://bore.pub:12345 (Django)
```

Install bore: download exe from https://github.com/ekzhang/bore/releases

Option B — Two separate ngrok binaries (different accounts):

```
Terminal 3: ngrok http 8000   (MSIX store version)
Terminal 4: C:\ngrok-cli\ngrok.exe http 8180  (standalone, 2nd account)
```

Limitations of ngrok:

- URLs change every restart → must re-share with Group 2

- Only one tunnel per ngrok instance on free plan
- Your laptop must stay on for anyone to access

Hotspot (if groups are nearby):

```
python manage.py runserver 0.0.0.0:8000
ipconfig  # Find your IP: 192.168.x.x
# Share: http://192.168.x.x:8000/api/docs
```

Step 3: Share this exact message on WhatsApp / Email

Copy and send this to your groups 2-10 group:

```
GOVASSET PLATFORM — API Integration Guide

Dear Groups 2-10,

Our authentication system is now live and accessible 24/7. Here are our permanent
URLs:

Our Django API:   https://govasset-api.up.railway.app
Our Keycloak SSO: https://govasset-keycloak.up.railway.app

API Documentation:
https://govasset-api.up.railway.app/api/docs/

Test Accounts (all passwords: Admin@123):
  superadmin  — Super Admin (sees all ministries)
  moh_admin   — Ministry Admin (Ministry of Health)
  mnh_manager — Agency Manager (Ministry of Health)
  rad_clerk   — Facility Clerk (Ministry of Health)
  moh_auditor — Auditor (Ministry of Health)
  mof_admin   — Ministry Admin (Ministry of Finance)

How to Integrate:

Option A — If your group built a login page (Type B): Call our API directly
  1. POST /api/auth/login/ with username + password
  2. Save the 'access' token
  3. GET /api/auth/verify-token/ to get user role + ministry

Option B — If your group has NO login page (Type A): Use SSO
  1. We register your app in Keycloak, give you Client ID + Secret
  2. You redirect users to our Keycloak login page at: https://govasset-
keycloak.up.railway.app/realms/govasset/protocol/openid-connect/auth
  3. Keycloak sends them back to your callback URL with a token
  4. You call GET /api/auth/verify-token/ at https://govasset-
api.up.railway.app/api/auth/verify-token/ to get user role + ministry

Choose whichever fits your group's architecture.
```

```
Response you get from verify-token (both options):
{
  "valid": true,
  "user": {
    "username": "moh_admin",
    "role": "MINISTRY_ADMIN",
    "ministry_schema": "moh_schema",
    "ministry": "Ministry of Health"
  }
}

Use "role" to control permissions in your system.
Use "ministry_schema" to filter data to their ministry.
Token expires in 30 minutes → call /api/auth/refresh/ to extend.

If you need additional fields in the response, let me know.
```

Step 4: Handle their questions

Q: "How do I call your API from my language?" Every language can make HTTP requests. Here is the Python example (they translate to PHP/Java/C#/JavaScript):

```python
import requests

# Step 1: Login
resp = requests.post('https://a1b2c3d4.ngrok-free.app/api/auth/login/', json={
    'username': 'moh_admin',
    'password': 'Admin@123'
})
token = resp.json()['access']

# Step 2: Verify token
resp = requests.get('https://a1b2c3d4.ngrok-free.app/api/auth/verify-token/',
headers={
    'Authorization': f'Bearer {token}'
})
user = resp.json()['user']
print(user['role'], user['ministry_schema'])
```

Q: "Do I need to install Keycloak?" No. You never install Keycloak. If your group has no login page (Type A), your users will log in through OUR Keycloak page — we give you the redirect URL. If your group has its own login (Type B), you call our API directly — Keycloak is invisible to you.

Q: "What database should I use?" Any database — MySQL, PostgreSQL, MongoDB, SQL Server. Your database is separate from ours. We only communicate via HTTP.

Q: "How do I handle users from different ministries?" Every user response includes `ministry_schema`. Add this column to your tables and filter by it:

```
SELECT * FROM your_table WHERE ministry_schema = 'moh_schema'
```

Q: "What if the token expires?" Access tokens expire after 30 minutes. Call `POST /api/auth/refresh/` with your refresh token to get a new pair.

Q: "What if I need more user information (phone, department, etc.)?" Ask us and we add it to the verify-token response. Takes 2 minutes if the field exists in our database.

Q: "Will my system appear in your sidebar?" No. Each group has their own separate application with their own sidebar and pages. You can add a link to our system:

```
<a href="http://localhost:8000/dashboard/">GovAsset Platform</a>
```

Q: "Does your audit trail track my users' actions?" No. Our audit trail tracks actions within our platform (login, user management, asset changes). You build your own audit trail for your own system.

Q: "Do I need to create users in my own database?" Yes and no. You use OUR users (via verify-token) for authentication. But if you need extra fields (like "employee department" or "phone extension"), store those in your own database linked by `username`.

Q: "What if your system is down?" Contact us. During development, our server runs on our laptop — it goes offline when we close it. In production, it runs 24/7 on a cloud server.

Hosting — How Group 2 Accesses Your System

Group 2 cannot access `localhost:8000` or `localhost:8180` from their computer. You need to make your system accessible to them. Here are your options:

Option	Cost	Setup Time	URL Changes?	Best For
Railway	Free tier	2-3 hours	**No — permanent**	Recommended for integration
ngrok	Free	10 minutes	Yes — every restart	Quick local testing only
DigitalOcean / Linode	$6-12/month	3-4 hours	No — permanent	Full production demo

Option 1 — Railway (RECOMMENDED — permanent URLs, 24/7 uptime)

Deploy both Django + Keycloak to Railway. Both get permanent URLs that never change. Group 2 configures once and never updates. Your laptop can be off. Full step-by-step instructions are in the "Step-by-Step" section above.

Pros: Permanent URLs, 24/7 uptime, free tier, laptop can be off, PostgreSQL included. **Cons:** 30-45 minute setup time (one time only).

Option 2 — ngrok (fallback for quick local testing only)

ngrok creates temporary public URLs that tunnel to your localhost. Free plan only supports one tunnel at a time. Documented in the "Step 2 Fallback" section above.

Pros: Works immediately, no hosting setup. **Cons:** URLs change every restart. Laptop must stay on.

Option 3 — VPS (if you want full control)

Rent a small Linux server ($6/month on DigitalOcean or Linode):

```
Server: 1 CPU, 1GB RAM, 25GB SSD
  └── Install: Docker + Docker Compose
      ├── Container 1: Django + Gunicorn
      ├── Container 2: PostgreSQL
      └── Container 3: Keycloak
```

Pros: Full control, everything runs on one machine, Docker Compose makes setup repeatable. **Cons:** Costs money, requires Linux command line knowledge.

My recommendation for your presentation timeline:

1. **Today/Tomorrow:** Deploy to Railway (permanent URLs), then share with Group 2
2. **This week:** Deploy to Railway or a VPS for a permanent URL
3. **Before panel:** Have the permanent URL ready so you demo from the cloud, not localhost

Once hosted, update these in your documentation and share with Group 2:

What to update	Old value	New value
Keycloak Auth URL	`http://localhost:8180/...`	`https://your-app.up.railway.app/...`
Django API URL	`http://localhost:8000/...`	`https://your-api.up.railway.app/...`
Client redirect URI	`http://localhost:3000/callback`	`https://group2-system.com/callback`
WhatsApp message template	localhost URLs	live URLs

PART 14: Why does each ministry need its own domain?

Question: "Why can't we use one domain like `platform.go.tz/ministry/1/`?"

Answer: Because django-tenants works by reading the DOMAIN from the URL, not the PATH.

How django-tenants identifies the tenant:

```
URL: https://moh.platform.go.tz/assets/
                      ↑
            django-tenants reads this part
            Looks up "moh.platform.go.tz" in Domain table
            Finds: this belongs to Ministry of Health
            Switches database to moh_schema
```

Why not use URL path instead? (e.g., /ministry/1/)

```
# This would NOT work with django-tenants:
URL: https://platform.go.tz/ministry/1/assets/
                          ↑
               django-tenants doesn't look here
               It ONLY looks at the domain name
```

django-tenants is designed specifically for the "each tenant has its own domain" model. This is actually the standard for SaaS products:

- `company1.salesforce.com`
- `company2.slack.com`
- `moh.github.com`

But we also handle IP access:

For mobile API access, users hit our server by IP address (not domain). We set:

```
SHOW_PUBLIC_IF_NO_TENANT_FOUND = True
```

When Django can't find a matching domain (because the user connected via IP), it falls back to the public schema. This is why the mobile app works even without domains.

PART 15: PRESENTATION TIPS — How to present well

Before the panel:

1. **Run the project** — Make sure `python manage.py runserver 0.0.0.0:8000` works. Show it running.
2. **Have test data** — Create a test ministry, test user, test assets so you can click through the UI.
3. **Open Keycloak admin** — Show Keycloak's admin page (`http://localhost:8180`) with users listed there.
4. **Open PostgreSQL** — Show the database schemas: `\dn` in psql shows moh_schema, mof_schema, etc.

Common panel questions and how to answer:

Q: "What problem does your system solve?"

> "Currently, government ministries track assets on paper or Excel. There is no central system. When an asset moves between ministries, there is no record. Our system gives each ministry its own secure space to track assets, with a shared super admin who can see everything."

Q: "Why PostgreSQL and not MySQL?"

> "Because only PostgreSQL supports schemas. We need schemas to give each ministry its own private set of tables. MySQL doesn't have this feature."

Q: "How is data kept separate between ministries?"

> "Each ministry has its own PostgreSQL schema — basically its own set of tables. When a user from Ministry of Health logs in, the system only connects to MOH's schema. Even if there's a bug in the code, the database won't show another ministry's data."

Q: "What if one ministry has millions of assets?"

> "Each schema is independent. MOH having 2 million assets doesn't slow down MOF's queries. Each ministry's tables only contain their own data."

Q: "How does the mobile app authenticate?"

> "The mobile app calls our API with username and password. Django returns a JWT token that expires in 30 minutes. The mobile app sends this token with every request. When it expires, they use a refresh token to get a new one."

Q: "What happens if Keycloak is down?"

> "Users who are already logged in can continue. New users cannot log in. The mobile app's existing tokens continue to work for 30 minutes. After that, they cannot refresh."

Q: "What is the most important security feature?"

> "Database-level isolation. Each ministry's data lives in a separate PostgreSQL schema. Even if someone finds a bug in our code, the database itself prevents cross-ministry access."

Q: "How do you know who changed an asset?"

> "Every action — create, update, delete, login, logout — is recorded in the AuditLog. The record cannot be modified or deleted. It captures who did it, what they changed, the old values, and the new values."

Q: "How do you add a new ministry?"

> "The Super Admin fills a form with the ministry name, schema name, and domain. When they click submit, Django creates a new PostgreSQL schema, runs all migrations inside it, creates the root organization unit, and sets up the domain mapping. The whole process takes about 2 seconds."

If you don't know an answer:

> "That's a good question. I have not tested that scenario yet. Based on how the system is designed, I believe it would work like this: [your best guess]. But I would need to verify that before giving a

> confident answer."

This is honest and shows understanding without lying.

For the demo:

1. Start with the **login page** — show the SSO button
2. Log in as **Super Admin** — show ministry list
3. Show **onboarding a new ministry** — explain schemas
4. Show **asset list** — demonstrate search, filter, pagination
5. Show **audit log** — demonstrate tamper-proof records
6. Show **Swagger API docs** (/api/docs/) — demonstrate the API
7. Show **PostgreSQL** — run `\dn` to show schemas

Browser cache issue — CSS not loading (important!)

What happened to you: Your browser saved (cached) the CSS file locally from a previous visit. Even after restarting your PC, the browser still used the SAVED CSS from its cache instead of fetching the fresh CSS from the server. This is why Ctrl+F5 (hard refresh) fixed it — it tells the browser to IGNORE the cache and download everything fresh.

Why restarting the PC didn't work: The browser cache is stored on your hard drive. Restarting the PC does NOT clear the browser cache. Only clearing the cache or a hard refresh removes it.

How to fix it during a presentation (if CSS breaks):

1. **Quick fix:** Press **Ctrl+Shift+R** (Windows) or **Cmd+Shift+R** (Mac) on the page
2. **Alternative:** Open your browser's Developer Tools (F12) → right-click the refresh button → "Empty Cache and Hard Reload"
3. **Nuclear option:** Chrome → Settings → Privacy and security → Clear browsing data → Check "Cached images and files" → Clear data

Why this happens more with CSS than HTML:

- HTML pages are usually served with `no-cache` headers (always check for updates)
- CSS files are often served with cache headers that say "keep this for days/weeks"
- When you change the CSS, the browser doesn't know — it keeps using the old saved version

To prevent this in the future: When you update the CSS file, you can also restart the Django server (Ctrl+C then `python manage.py runserver`). Django's development server automatically adds cache-busting headers in debug mode, which helps — but sometimes the browser's cache still wins.

PART 16: QUICK REFERENCE — Key files and what they do

File	What it does	Why it exists
`config/settings.py`	All configuration — apps, database, security, auth	The brain of the project

File	What it does	Why it exists
config/urls.py	Maps URLs to view functions	The roadmap — matches URLs to code
tenants/models.py	Ministry and Domain models	Defines what a tenant (ministry) is
tenants/views.py	Web pages for managing ministries	Super Admin manages ministries here
authentication/models.py	CustomUser, LoginAttempt, PendingAccess	User accounts beyond Django's defaults
authentication/views.py	Login/logout pages	The web login page
authentication/api_views.py	API login, refresh, verify token	Mobile app authentication
authentication/api_urls.py	API URL routes	Maps API URLs to API views
authentication/api_serializers.py	Converts data to JSON for API	Translates database to API format
authentication/api_permissions.py	Who can access which API endpoint	Security for APIs
authentication/api_exception_handler.py	Consistent error format for API	Makes errors predictable
authentication/decorators.py	Role checking for web views	Security for web pages
authentication/middleware.py	Sets schema on every request	Ensures correct data isolation
authentication/oidc_backend.py	Bridges Keycloak to Django users	The Keycloak connection
authentication/pagination.py	Splits lists into pages	Handles "next page" logic
assets/models.py	Asset and AssetCategory models	The core data — what is an asset?
assets/api_views.py	Asset CRUD API endpoints	Mobile app manages assets here
assets/views.py	Asset web pages	Browser manages assets here
organizations/models.py	OrgUnit, MasterData, AuditLog	Org hierarchy, dropdowns, audit trail

File	What it does	Why it exists
organizations/views.py	Org hierarchy web pages	Browser views org tree
organizations/api_views.py	Org tree, audit log, dashboard API	Mobile app reads org data
organizations/master_data_views.py	Dropdown management pages	Admin configures dropdown options
templates/shared/base.html	Main layout with sidebar	Every page uses this template

PART 17: COMPLETE REQUEST-TO-RESPONSE FLOW

Web browser flow (full example):

```
1. User types: http://moh.localhost:8000/assets/
                                        ↑
2. Operating system asks DNS: "What is moh.localhost?"
   → /etc/hosts file says: 127.0.0.1 moh.localhost
   → So the request goes to your own computer (localhost)

3. Django's runserver receives the request on port 8000

4. Django goes through MIDDLEWARE in order:

   a) TenantMainMiddleware:
      "The domain is moh.localhost. Let me find the tenant."
      → Queries tenancy_domain table: "Who owns moh.localhost?"
      → Finds: Ministry of Health (moh_schema)
      → Sets connection to moh_schema

   b) SecurityMiddleware:
      Adds security headers to the response later

   c) SessionMiddleware:
      Loads the session for this browser

   d) CorsMiddleware:
      Checks if this is a cross-origin request

   e) CommonMiddleware:
      Handles URL redirects (e.g., /assets → /assets/)

   f) AuthenticationMiddleware:
      "Is this user logged in?"
      Loads the user from the session

   g) Our SchemaMiddleware:
      "What is this user's ministry_schema?"
      Sets request.schema_name = 'moh_schema'
```

```
   h) SessionRefresh (mozilla-django-oidc):
      Checks if Keycloak session is still valid

5. URL dispatch:
   Django matches /assets/ against urlpatterns:
   → Finds: path('assets/', asset_list_view, name='asset_list')

6. Decorators check (before view runs):
   @login_required_custom: "Is user logged in? Yes → continue"
   @role_required('MINISTRY_ADMIN'): "What is user's role? Yes → continue"

7. View function runs:
   with schema_context('moh_schema'):
       assets = Asset.objects.all()  → Only MOH's assets
   Returns HTML page with list of assets

8. Response goes back through middleware (reverse order)

9. Browser receives HTML and renders it
```

Mobile API flow (full example):

```
1. Flutter app sends: GET http://192.168.100.18:8000/api/assets/
   Header: Authorization: Bearer eyJhbGciOiJI...

2. Django's middleware processes the request (same as above)

3. URL dispatch matches: /api/assets/
   → Finds: AssetListCreateAPIView

4. Permission classes check:
   IsAuthenticated: "Is the JWT token valid? Yes → continue"
   HasMinistrySchema: "Does user have a ministry? Yes → continue"
   CanManageAssets: "GET is read-only, all roles can read → continue"

5. View runs:
   with schema_context('moh_schema'):
       assets = Asset.objects.all()
   serializer = AssetSerializer(assets, many=True)
   return Response(serializer.data)  → JSON response

6. Flutter receives JSON and shows it in the mobile app
```

PART 18: GLOSSARY — Simple definitions

Term	Simple meaning
Schema	A private folder inside the PostgreSQL database. Each ministry has one.

Term	Simple meaning
Tenant	One customer in a multi-tenant system. For us, one ministry = one tenant.
OIDC	OpenID Connect — a standard way for one website to let another website handle login. Google, Facebook, and Microsoft use this.
JWT	JSON Web Token — a string of text that proves who you are. Like a digital ID card with an expiration date.
SSO	Single Sign-On — log in once, access many systems. Keycloak is our SSO provider.
DRF	Django REST Framework — makes building APIs easy.
CORS	Cross-Origin Resource Sharing — tells browsers it's OK to let the Flutter app call our API.
Middleware	Code that runs on every request before your view. Like airport security — everyone goes through it.
Decorator	A wrapper around a function that checks something first (like "is the user logged in?").
Serializer	Converts database data to JSON (for API) and JSON back to database data.
Mixin	A pre-built class you add to your model to give it extra features automatically. TenantMixin gives us schema_name for free.
REST API	A way for programs to talk to each other over the internet. Our Flutter app uses the REST API to get and update data.
Claims	Information inside a JWT token about the user (username, role, email, etc.).
Realm	In Keycloak: a collection of users, clients, and configurations. Ours is called "govasset".
Search path	PostgreSQL setting that tells it which schemas to look in when running queries. django-tenants sets this automatically.

PART 19: REMEMBER THIS FOR THE PANEL

The panel wants to know:

1. **You understand your own project** — Not just how to use it, but WHY you made each choice
2. **You can explain technical concepts simply** — If your supervisor asks "what is multi-tenancy?", you can explain it without reading from a script
3. **You know the weaknesses** — Be honest about what's not perfect (no tests, no rate limiting)
4. **You can demo it live** — Practice clicking through the UI without freezing

Key sentences to memorize:

> "Each ministry gets its own PostgreSQL schema — a private set of database tables. This means Ministry of Health data is physically separate from Ministry of Finance data, enforced by the database, not just by our code."

> "Keycloak handles the password checking. Our Django app never sees the user's password. Keycloak tells us: 'This user is who they say they are.' Then we check our database: 'Does this user exist in our

> system?'"

> "JWT tokens expire after 30 minutes with a 1-day refresh token. This limits damage if a token is stolen — the thief can only use it for 30 minutes."

> "Our audit log is append-only. Once a record is created, it cannot be changed or deleted. This is required for government compliance and asset tracking accountability."

The one-paragraph answer — If the panel asks "How does your security work end to end?"

Memorize this paragraph. It covers authentication, tokens, multi-tenancy, and audit in one confident answer:

> *"Our system supports two authentication paths. Web browser users log in through Keycloak SSO, which handles password checking and brute-force lockout. Other groups' users either go through the same Keycloak SSO (if their group has no login page) or authenticate via our API endpoint (if their group has their own login form). Both paths produce a JWT token containing the user's identity, role, and ministry. Every subsequent request attaches that token, and our permission classes check it before any data is touched. The multi-tenancy middleware switches the database to the correct ministry's private schema before any query runs. Every action is permanently recorded in an audit log that cannot be modified or deleted by anyone, including the Super Admin. This is all configured in settings.py, oidc_backend.py, api_views.py, and the models in tenants and organizations."*

This answer covers Keycloak, JWT, brute-force, multi-tenancy, and audit — everything they are likely to ask about.

PART 20: CSS QUICK REFERENCE — What to change when the panel asks

The panel may ask: "Change this text to red" or "Make the background blue". Here is how to do it fast.

Quickest way: Inline CSS (highest priority)

Inline CSS beats any file. Add `style="..."` to any HTML tag:

```
<!-- Change text color -->
<h1 style="color:red;">This text will be red</h1>
<p style="color:blue;">Blue text</p>
<span style="color:green;">Green text</span>

<!-- Change background color -->
<div style="background-color:yellow;">Yellow background</div>
<td style="background:#fff3cd;">Light yellow cell</td>

<!-- Change text size -->
<p style="font-size:18px;">Bigger text</p>
<p style="font-size:14px;">Smaller text</p>
<h1 style="font-size:32px;">Custom heading size</h1>

<!-- Change font weight (boldness) -->
<p style="font-weight:bold;">Bold text</p>
<p style="font-weight:normal;">Normal weight</p>
```

```
<!-- Change padding (space inside) -->
<div style="padding:20px;">More space inside</div>
<div style="padding:10px 15px;">10 top/bottom, 15 left/right</div>

<!-- Change margin (space outside) -->
<div style="margin-bottom:20px;">Push things below down</div>

<!-- Change border -->
<div style="border:2px solid red;">Red border</div>
<div style="border:1px dashed #ccc;">Dashed grey border</div>

<!-- Change width -->
<div style="width:50%;">Half width</div>
<div style="max-width:800px;">Max 800px wide</div>

<!-- Multiple styles together -->
<div style="color:red; background:yellow; font-size:18px; padding:15px; border:1px
solid orange;">
  All at once
</div>
```

Our project's CSS variable names (used everywhere)

We defined these in `static/css/style.css`. Use them in inline styles:

```
<!-- Text colors -->
style="color:var(--accent);"       /* Blue accent color */
style="color:var(--danger);"       /* Red for errors */
style="color:var(--success);"      /* Green for success */
style="color:var(--warning);"      /* Orange/amber for warnings */
style="color:var(--text-muted);"   /* Grey for secondary text */
style="color:var(--text-primary);" /* Default text color */

<!-- Background colors -->
style="background:var(--bg-page);"            /* Page background */
style="background:var(--success-bg);"         /* Light green bg */
style="background:var(--danger-bg);"          /* Light red bg */
style="background:var(--warning-bg);"         /* Light yellow bg */
style="background:var(--accent);color:#fff;"  /* Blue bg + white text */

<!-- Border colors -->
style="border-color:var(--danger-border);"
style="border-color:var(--success-border);"

<!-- Border radius (rounded corners) -->
style="border-radius:var(--radius-sm);"  /* Small rounding */
style="border-radius:var(--radius-md);"  /* Medium rounding */
style="border-radius:var(--radius-lg);"  /* Large rounding */
style="border-radius:var(--radius-xl);"  /* Extra large */
```

```
<!-- Shadows -->
style="box-shadow:var(--shadow-sm);"  /* Small shadow */
style="box-shadow:var(--shadow-md);"  /* Medium shadow */
style="box-shadow:var(--shadow-lg);"  /* Large shadow */
```

Common color names vs our project colors

Panel says...	Use this inline CSS
"Make this red"	`style="color:red;"` or `style="color:var(--danger);"`
"Make background yellow"	`style="background:yellow;"` or `style="background:var(--warning-bg);"`
"Make this green"	`style="color:green;"` or `style="color:var(--success);"`
"Make this blue"	`style="color:blue;"` or `style="color:var(--accent);"`
"Make text bigger"	`style="font-size:20px;"`
"Make text smaller"	`style="font-size:12px;"`
"Make this bold"	`style="font-weight:bold;"`
"Add spacing"	`style="padding:15px;"` or `style="margin:10px 0;"`
"Add border"	`style="border:1px solid #ccc;"`

Custom CSS classes we have (defined in style.css)

Do NOT use Bootstrap classes like `text-danger` or `bg-primary` — Bootstrap CSS is NOT loaded. Instead, use our custom classes:

Class	What it does
`.btn`	Makes a link/button look like a styled button
`.btn-primary`	Blue button
`.btn-outline`	Button with border (no fill)
`.btn-danger-outline`	Red outlined button
`.card`	White box with shadow and rounded corners
`.badge`	Small colored label
`.badge-active`	Green badge (asset is active)
`.badge-maint`	Orange badge (under maintenance)
`.badge-decomm`	Grey badge (decommissioned)
`.data-table`	Full-width table with borders
`.alert-danger`	Red message box

Class	What it does
`.alert-success`	Green message box
`.alert-warning`	Yellow message box
`.alert-info`	Blue message box
`.form-control`	Input field styling
`.form-select`	Dropdown styling
`.page-header`	Title area at top of page
`.filter-bar`	Search/filter row
`.empty-state`	"No data" message box
`.stat-card`	Dashboard statistic box
`.sidebar`	Left navigation panel

Bootstrap Icons (yes, we have these)

We load Bootstrap Icons (not Bootstrap CSS). Use them like this:

```
<i class="bi bi-plus-circle"></i>    <!-- Plus icon -->
<i class="bi bi-pencil"></i>         <!-- Edit/pencil icon -->
<i class="bi bi-trash"></i>          <!-- Delete/trash icon -->
<i class="bi bi-search"></i>         <!-- Search icon -->
<i class="bi bi-download"></i>       <!-- Download icon -->
<i class="bi bi-arrow-left"></i>     <!-- Left arrow -->
<i class="bi bi-check-circle"></i>   <!-- Check mark -->
<i class="bi bi-exclamation-triangle"></i>  <!-- Warning triangle -->
<i class="bi bi-gear"></i>           <!-- Settings gear -->
<i class="bi bi-person"></i>         <!-- User icon -->
<i class="bi bi-box"></i>            <!-- Box/asset icon -->
<i class="bi bi-building"></i>       <!-- Building/ministry icon -->
<i class="bi bi-shield-lock"></i>    <!-- Security/lock icon -->
```

Where files are located

What you want to change	File to edit
Global styles (all pages)	`static/css/style.css`
A single page's layout	The `.html` template file in `templates/`
A single element	Use inline `style="..."` on that HTML tag

If they ask you to find what CSS class controls something:

Open the page in a browser → Right-click the element → Inspect (or F12) → Look at the "Styles" tab → It shows you which CSS file and line number controls it.

If they ask you to add a new page:

1. Create `.html` file in the correct `templates/` folder
2. Use existing classes: `class="page-header"`, `class="card"`, `class="btn btn-primary"`
3. Use inline styles for unique styling
4. All templates extend `base.html` which loads style.css automatically

---

PART 21: DATABASE DEEP DIVE — Tables, schemas, shared vs private

Why do we never start PostgreSQL?

PostgreSQL runs as a **Windows service** — it starts automatically when Windows starts. You don't need to run it manually.

```
Check it: Open "Services" (Windows search → type "Services")
  → Look for "postgresql-x64-16" (or similar name)
  → Status should be "Running"
  → Startup type: "Automatic" (starts when Windows boots)

If it's not running: Right-click → Start
```

When you run `python manage.py runserver`, Django automatically connects to PostgreSQL using the settings in `config/settings.py`. You never need to type `pg_ctl start` or anything like that.

How many tables do we have? And their purpose?

We have **10 main tables** across our system:

#	Table name	Schema	Purpose
1	`tenants_ministry`	Public	List of all ministries (MOH, MOF, MOE...)
2	`tenants_domain`	Public	Maps domains to ministries (moh.localhost → MOH)
3	`authentication_customuser`	Public	All users from ALL ministries
4	`authentication_loginattempt`	Public	Failed login records (brute force tracking)
5	`authentication_pendingaccess`	Public	Users waiting for admin approval
6	`django_session`	Public	Browser session storage
7	`assets_asset`	Per-ministry	All assets for that ministry

#	Table name	Schema	Purpose
8	assets_assetcategory	Per-ministry	Asset categories (ICT, Furniture, Vehicles...)
9	organizations_orgunit	Per-ministry	Organization hierarchy (Ministry → Agency → Facility)
10	organizations_auditlog	Per-ministry	Every action recorded (cannot be deleted)
+	organizations_masterdata	Per-ministry	Dropdown options (funding sources, etc.)

Wait — tables 1-6 are in the PUBLIC schema (one copy for everyone) Tables 7-10+ are in EACH MINISTRY'S PRIVATE schema (separate copy per ministry)

If we have 5 ministries, we have: 6 public tables + (5 × 4 private tables) = 26 tables total.

What columns does each table have?

1. tenants_ministry (who are our ministries?)

```
id          → Auto number (1, 2, 3...)
name        → "Ministry of Health"
schema_name → "moh_schema" (used by PostgreSQL)
created_at  → When this ministry was added
```

2. tenants_domain (how URLs map to ministries)

```
id          → Auto number
domain      → "moh.localhost" (the URL people type)
tenant_id   → Points to which Ministry this domain belongs to
is_primary  → Is this the main domain for this ministry?
```

3. authentication_customuser (all users in the system)

```
id              → Auto number
username        → Login name (e.g., "moh_admin")
email           → Email address
password        → Hashed password (never the real password!)
role            → SUPER_ADMIN, MINISTRY_ADMIN, AGENCY_MANAGER, FACILITY_CLERK,
AUDITOR
ministry_schema → "moh_schema" (which ministry they belong to)
keycloak_id     → Their ID in Keycloak (links the two systems)
is_active       → True/False (can they log in?)
date_joined     → When they registered
```

4. assets_asset (the main data — what we track)

```
id              → Auto number
name            → "Dell Latitude 5420"
asset_number    → "MOH-ICT-0042" (unique barcode number)
category        → Which category (points to assetcategory table)
status          → ACTIVE, UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
description     → "Laptop for ICT officer"
serial_number   → Manufacturer's serial number
purchase_date   → When it was bought
purchase_cost   → How much it cost
location        → Where it is physically
assigned_to     → Who is using it (email)
org_unit        → Which organization unit it belongs to
created_at      → When this record was added
updated_at      → When it was last changed
```

Why do some tables live in the public schema and others in private schemas?

This is the most important database design decision. Here is the simple rule:

```
PUBLIC SCHEMA = Things that are shared across ALL ministries
  — Users (a user belongs to one ministry but the user table is shared)
  — Ministries (the list of all ministries)
  — Domains (the URL-to-ministry mapping)
  — Login attempts (tracking failed logins globally)
  — Pending access (approval queue for new users)
  — Sessions (browser login sessions)

PRIVATE SCHEMA = Things that belong to ONE ministry only
  — Assets (MOH's laptops are not MOF's laptops)
  — Categories (each ministry has their own categories)
  — Org units (each ministry has their own structure)
  — Audit logs (each ministry's actions are private)
```

The key reason: Data isolation is enforced by the database.

```
WRONG APPROACH (one big table with ministry_id):
  assets_asset table:
    id  name             ministry_id
    1   Dell Laptop      1  (MOH)
    2   Office Chair     2  (MOF)
    3   Hospital Bed     1  (MOH)

  Query: Asset.objects.all()
  → Returns ALL 3 rows from all ministries
  → BUG! A programmer forgot to filter by ministry_id
  → MOH sees MOF's data. Security breach.
```

```
OUR APPROACH (separate schemas):
  MOH schema → assets_asset table:
    id  name
    1   Dell Laptop
    3   Hospital Bed

  MOF schema → assets_asset table:
    id  name
    2   Office Chair

  Query: Asset.objects.all() (while in MOH schema)
  → Returns only 2 rows (MOH's data)
  → Cannot see MOF's data even if we forget to filter
  → The DATABASE ITSELF prevents cross-ministry access
```

Analogy: Think of the public schema as the building's LOBBY — everyone shares it (entrance, directory, security desk). Each ministry's private schema is their OWN OFFICE — other ministries cannot walk in.

Why is the users table in the public schema?

Because a user's identity crosses ministries:

```
Super Admin → manages ALL ministries → needs to see everything
Ministry Admin → manages ONE ministry → sees only their schema
But ALL users are listed in one table so we can:
  1. Check "does this username already exist?"
  2. Super Admin can see all users across all ministries
  3. Prevent duplicate accounts
```

If users were in private schemas, two ministries could accidentally create the user "admin" — and we wouldn't know.

But wait — if users are in the public schema, can MOH users see MOF users?

No. The `CustomUser` model has a `ministry_schema` field. When we query users, we filter by schema:

```
# In code, when listing users for a ministry admin:
users = CustomUser.objects.filter(ministry_schema='moh_schema')
# This only returns MOH users, even though the table is shared
```

This is called **application-level filtering** (the code filters). It is less secure than schema-level isolation, which is why we only use it for the users table (which needs to be shared).

PART 22: MAKING THE SYSTEM LIVE — From development to production

What needs to happen to make our system live?

Right now, our system only runs on our laptop. To make it live (accessible 24/7 to real users), we need:

```
┌──────────────────────────────────────────────────────────┐
│ WHAT WE HAVE NOW (development):                          │
│                                                          │
│  Our laptop → python manage.py runserver 0.0.0.0:8000    │
│  → Only works when laptop is on                          │
│  → Only works on our WiFi                                │
│  → Only we can access it                                 │
│  → No proper URL (just IP address)                       │
│  → No HTTPS (no padlock icon)                            │
│  → Laptop might overheat if running 24/7                 │
└──────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────┐
│ WHAT WE NEED (production):                               │
│                                                          │
│  A cloud server (AWS / DigitalOcean / Vultr)             │
│  Running: Ubuntu Linux + PostgreSQL + Django             │
│  Accessible at: https://moh.platform.go.tz               │
│  Running: 24/7, never stops                              │
│  Secure: HTTPS padlock, firewall, automatic backups      │
│  Fast: Good internet connection, proper server           │
└──────────────────────────────────────────────────────────┘
```

What happens if we don't use proper domains?

If we only use `127.0.0.1:8000` or `192.168.x.x:8000`:

```
Problem 1: django-tenants cannot identify the ministry
  → django-tenants reads the domain from the URL
  → If URL is 127.0.0.1:8000 → no domain found
  → Falls back to public schema
  → All users see the public schema (not their ministry's data)
  → The whole multi-tenancy system breaks!

Problem 2: No real SSL/HTTPS
  → Modern mobile apps REQUIRE HTTPS for API calls
  → Without HTTPS, the Flutter app will refuse to connect
  → Browsers show "Not Secure" warning

Problem 3: No proper URL
  → Users cannot remember "192.168.100.18:8000"
  → They expect "moh.go.tz" or "govasset.go.tz"
  → Looks unprofessional for a government system
```

What code changes are needed for production?

1. ALLOWED_HOSTS in settings.py:

```
# Currently (development):
ALLOWED_HOSTS = ['192.168.100.18', '172.16.20.232', '10.187.165.150',
                 'localhost', '127.0.0.1', '.localhost']

# Change to (production):
ALLOWED_HOSTS = ['.platform.go.tz',     # All subdomains
                 'platform.go.tz',       # Main domain
                 '192.168.100.18']       # Keep IP for testing
```

2. Add real domain entries in the Domain table:

```
# Currently: Domain.objects.create(domain='moh.localhost', tenant=moh)
# Change to: Domain.objects.create(domain='moh.platform.go.tz', tenant=moh)
```

3. Set DEBUG = False:

```
# Currently:
DEBUG = True

# Change to (production):
DEBUG = False
```

4. Configure HTTPS/SSL:

```
# Add to settings.py for production:
SECURE_SSL_REDIRECT = True     # Force HTTPS
SESSION_COOKIE_SECURE = True   # Only send session cookies over HTTPS
CSRF_COOKIE_SECURE = True      # Only send CSRF cookies over HTTPS
```

5. Set up a proper web server (NGINX or Apache):

```
NGINX configuration (simplified):
_____________________________________

server {
    listen 443 ssl;
    server_name moh.platform.go.tz;

    ssl_certificate /etc/ssl/certs/platform.go.tz.crt;
    ssl_certificate_key /etc/ssl/private/platform.go.tz.key;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
    }
```

```
    location /static/ {
        alias /var/www/govasset/static/;
    }
}
```

How will other groups access our system in production?

```
Other Group → Their Browser → https://moh.platform.go.tz/
                                    ↓
                                DNS server
                                (maps domain to server IP)
                                    ↓
                                Cloud Server (AWS/DigitalOcean)
                                    ↓
                                NGINX (handles HTTPS)
                                    ↓
                                Gunicorn (runs Django)
                                    ↓
                                PostgreSQL Database
```

They type: `https://moh.platform.go.tz/` in their browser **They see:** Our application, with proper URL, proper SSL, professional look

For API access: Same thing, but use:

```
https://moh.platform.go.tz/api/assets/
Instead of:
http://192.168.100.18:8000/api/assets/
```

What about our own `etc/hosts` file?

Currently, we use the `etc/hosts` file to make `moh.localhost` work:

```
127.0.0.1  moh.localhost
127.0.0.1  mof.localhost
127.0.0.1  admin.localhost
```

In production, we don't need this because real DNS servers handle the mapping:

```
moh.platform.go.tz → DNS → 104.28.45.12 (the server's public IP)
```

The `etc/hosts` file is ONLY for development (since we don't own real domains yet).

How to see what domain a user is using (code evidence):

```
# tenants/views.py — The tenant detection logic
from django_tenants.utils import get_tenant_model

def get_tenant_from_request(request):
    domain = request.get_host()  # Gets: "moh.localhost" or "192.168.100.18:8000"
    domain_model = get_tenant_model()
    try:
        tenant = domain_model.objects.get(domains__domain=domain.split(':')[0])
        return tenant
    except domain_model.DoesNotExist:
        return None  # No matching domain → public schema
```

What does django-tenants do when it finds a domain?

```
# django-tenants internally does this:
# 1. Reads: request.get_host() → "moh.localhost"
# 2. Queries: Domain.objects.get(domain="moh.localhost")
# 3. Gets: tenant = moh_ministry (schema_name = "moh_schema")
# 4. Runs: connection.set_schema("moh_schema")
# 5. Now ALL queries use: SET search_path TO moh_schema, public;
```

Proxies and load balancers — What they are and why they matter

What is a proxy?

A proxy is a server that sits BETWEEN the user and your Django application. Instead of the user talking directly to Django, the user talks to the proxy first, and the proxy forwards the request to Django.

```
User (203.0.113.5)
    │
    ▼
Proxy (198.51.100.10)   ← The proxy receives the request
    │
    ▼
Django Server            ← The proxy forwards it to Django
```

Why use a proxy?

- **Security** — Hides your Django server from the public internet
- **HTTPS** — Handles SSL certificates (the padlock in the browser)
- **Caching** — Saves copies of pages to make the site faster
- **Load balancing** — Spreads traffic across multiple Django servers

What is a load balancer?

Imagine your website becomes very popular. Instead of one Django server, you have three. A load balancer decides which server handles each request:

```
                ┌──────────────► Django Server 1
User            │
  │             ├──────────────► Django Server 2
  ▼             │
Load Balancer   └──────────────► Django Server 3
```

If 3,000 people visit at the same time, one server would be overloaded. Three servers share the work. The load balancer spreads requests among them.

The IP address problem (important!)

Without a proxy, Django sees the user's real IP:

```
User (203.0.113.5) ──────► Django
REMOTE_ADDR = "203.0.113.5"  ✓ Correct!
```

With a proxy, Django sees the PROXY's IP, not the user's:

```
User (203.0.113.5) ──────► Proxy (198.51.100.10) ──────► Django
REMOTE_ADDR = "198.51.100.10"  ✗ Wrong — that's the proxy!
```

Why does the proxy change the IP? The proxy doesn't "change" anything. It creates a NEW connection to Django. From Django's point of view, the proxy IS the client. Django can only see the computer that connected directly to it.

Analogy: You call a pizza shop. Without a proxy, you call directly — the shop sees YOUR number. With a proxy, you call your friend, and your friend calls the shop for you. The shop sees YOUR FRIEND'S number because your friend is the one making the call.

The solution — X-Forwarded-For header

To solve this, the proxy adds a special header called `X-Forwarded-For` that contains the ORIGINAL user's IP:

```
User (203.0.113.5) ──────► Proxy ──────► Django
                                          Header: X-Forwarded-For: 203.0.113.5
```

Django can then read:

```
request.META['HTTP_X_FORWARDED_FOR']  # Returns: "203.0.113.5" (real user)
request.META['REMOTE_ADDR']            # Returns: "198.51.100.10" (proxy's IP)
```

Our helper function handles both scenarios:

```
def _get_client_ip(request):
    x_forwarded_for = request.META.get('HTTP_X_FORWARDED_FOR')
    if x_forwarded_for:
        # Proxy is present — use the FIRST IP in the list
        ip = x_forwarded_for.split(',')[0].strip()
    else:
        # No proxy — use the direct connection IP
        ip = request.META.get('REMOTE_ADDR')
    return ip
```

Do we have a proxy NOW (in development)?

No. During development with `python manage.py runserver`, there is no proxy. The helper function checks for `HTTP_X_FORWARDED_FOR`, doesn't find it, and simply returns `REMOTE_ADDR` (which is correct).

Why write this now? You are preparing for production where a proxy like Nginx or Cloudflare is very common. By writing this helper now, your code works correctly in BOTH situations:

```
DEVELOPMENT (no proxy):      PRODUCTION (with proxy):
  REMOTE_ADDR is correct       X-Forwarded-For has real IP
  returned as-is               helper reads it automatically
```

Your audit log, brute-force protection, and any security features that rely on IP addresses will work correctly without changes when you deploy.

PART 23: DEVELOPMENT SETUP — How web + mobile work together

How did the web developer and mobile developer access the same data?

This is a common panel question. Here is how we worked together:

```
SCENARIO: Two developers on different laptops, different roles

Web Developer (Group 1A):                Mobile Developer (Group 1B):
┌──────────────────────────┐             ┌──────────────────────────┐
│ Laptop A                 │             │ Laptop B                 │
│ IP: 192.168.100.18       │             │ IP: 192.168.100.45       │
│ Runs: Django + PostgreSQL│             │ Runs: Flutter + Emulator │
│ Port: 8000               │             │ No database              │
│                          │             │                          │
```

```
  │ His job:                    │          │ His job:                    │
  │ Build web pages             │          │ Build mobile screens        │
  │ Build APIs                  │          │ Call existing APIs          │
  │ Manage database             │          │ Display data on phone       │
  └─────────────────────────────┘          └─────────────────────────────┘
```

The mobile developer does NOT run Django or PostgreSQL. The mobile developer only runs Flutter (mobile app framework). To test, the mobile developer:

```
Step 1: Connect to the same WiFi (both laptops on college network)
Step 2: Web developer runs: python manage.py runserver 0.0.0.0:8000
Step 3: Web developer tells mobile: "My IP is 192.168.100.18"
Step 4: Mobile developer puts this URL in the Flutter app code:

        const API_BASE_URL = 'http://192.168.100.18:8000/api/';

Step 5: Mobile developer runs the Flutter app on his phone emulator
Step 6: The emulator calls: GET http://192.168.100.18:8000/api/assets/
Step 7: Web developer's Django receives the request and returns JSON
Step 8: Mobile developer's Flutter app displays the data
```

What if the web developer's IP changes?

```
Web developer runs: ipconfig
  → Sees new IP: 192.168.100.55
  → Tells mobile developer: "My IP changed to 192.168.55"
  → Mobile developer updates one line in Flutter code
  → Everything works again
```

What if they are on different WiFi?

```
→ They cannot connect directly
→ Solutions:
  a) Both join the same WiFi (college network, cafe, etc.)
  b) Web developer uses ngrok (creates a public URL):
       ngrok http 8000
       → Gives: https://abc123.ngrok.io
       → Mobile uses that URL instead
  c) Deploy Django to a temporary cloud server
```

Does the mobile developer need to install Django?

No. The mobile developer only needs:

```
Flutter SDK (to build the mobile app)
VS Code or Android Studio (to write code)
A phone or emulator (to test)
An internet connection (to reach the web developer's laptop)
```

The mobile developer NEVER needs:

```
Python or Django
PostgreSQL
Keycloak
Our project's source code (Django part)
```

The mobile developer only needs the API URL and the documentation.

How does the mobile developer know what API endpoints exist?

1. Web developer runs Django with Swagger enabled
2. Mobile developer opens in browser: `http://192.168.100.18:8000/api/docs/`
3. Mobile developer sees ALL endpoints with descriptions
4. Mobile developer clicks "Try it out" to test each endpoint
5. Mobile developer sees exactly what JSON is returned
6. Mobile developer writes Flutter code that parses that JSON

This is why Swagger is important! Without Swagger, the mobile developer would need to ask: "What URL do I call for assets? What fields are returned? What format is the date?" With Swagger, they find everything themselves.

What if the mobile developer makes changes that need new API features?

```
Mobile developer: "I need an API that returns only assets that expired this
month."
Web developer: "OK, I'll add it."
  → Web developer adds a filter: /api/assets/?expired_this_month=true
  → Swagger updates automatically
  → Mobile developer refreshes Swagger page
  → Mobile developer sees the new filter
  → Mobile developer starts using it
```

Why can't the mobile developer run everything locally?

The mobile developer COULD run Django locally, but:

1. Setting up PostgreSQL + Keycloak is complex
2. They would need all the demo data
3. Their laptop might be slow (Flutter + Django + PostgreSQL)

4. Any data they create would be on their laptop, not shared
5. It's easier to use one central API that everyone shares

This is the same model used in real companies:

```
Frontend team (mobile/web) → calls APIs from → Backend team (server)
Backend team maintains the server and database
Frontend teams just consume the APIs
```

What does the Flutter code look like for calling our API?

```dart
// Flutter code (simplified) — This is what the mobile developer writes

import 'package:http/http.dart' as http;
import 'dart:convert';

class ApiService {
  final String baseUrl = 'http://192.168.100.18:8000/api/';
  String? token;

  // Step 1: Login
  Future<void> login(String username, String password) async {
    final response = await http.post(
      Uri.parse('${baseUrl}auth/login/'),
      body: {'username': username, 'password': password},
    );
    final data = json.decode(response.body);
    token = data['access'];  // Save the JWT token
  }

  // Step 2: Get assets
  Future<List<Asset>> getAssets() async {
    final response = await http.get(
      Uri.parse('${baseUrl}assets/'),
      headers: {'Authorization': 'Bearer $token'},
    );
    final data = json.decode(response.body);
    return data['results'].map((json) => Asset.fromJson(json)).toList();
  }
}
```

The Flutter developer's job is to:

1. Call the API URL
2. Receive the JSON response
3. Parse it and display it beautifully on the phone screen
4. Handle errors (no internet, token expired, etc.)

The Flutter developer does NOT need to know:

- What database we use
- How multi-tenancy works
- What Keycloak is
- How Python/Django works

They just need the API URL and the JSON format.

How do OTHER groups (Group 2-10) work with us?

Same model as the mobile developer:

```
Other Group's Developer:
  → Gets our API URL + test credentials
  → Opens Swagger: http://192.168.100.18:8000/api/docs/
  → Tests endpoints
  → Writes code in their language (PHP, Java, C#, etc.)
  → Calls our API during development (same WiFi)
  → When we deploy to production, switches to production URL
```

The API is our contract with the world. As long as the API keeps working the same way, other groups don't care what technology we use behind it.

Do our fixes and changes affect the mobile app too?

This is an important question. The answer depends on WHAT we change:

Type of fix	Affects mobile?	Why
Changes to `AuditLog.objects.create()` or any model/database code	✅ Yes automatically	Same Django code runs for both web and mobile requests
Changes to API views in `api_views.py`	✅ Yes	Mobile calls these directly — new endpoints appear immediately
Adding new API filters (e.g., `?expired_this_month=true`)	✅ Yes	Mobile can start using them right away
Fixed IP address capture (`_get_client_ip()`)	✅ Yes	Django doesn't care if request is from browser or Flutter
Changes to web views in `views.py`	❌ No	These render HTML that mobile never sees
Changes to HTML templates (`.html` files)	❌ No	Flutter has its own completely separate screens
New sidebar links in `base.html`	❌ No	Flutter sidebar is built separately in Dart code
Changes to `settings.py`	✅ Depends	Database, JWT, CORS settings affect all clients

The general rule: Every database change and API change automatically helps mobile. Every HTML/template change is web-only. This is the clean separation between your two clients (web browser and Flutter app).

Example — IP fix we added: When a field clerk on their phone creates an asset, the request travels from the phone over the hotspot to Django. Django reads the IP address from that request using the same `_get_client_ip()` function — the phone's hotspot IP like `192.168.43.118` — and saves it to the audit log. You do NOT need to do anything in the Flutter code. It works automatically.

Why the audit trail is web-only (important for panel questions): The audit trail is a governance and accountability feature designed for office-based administrators and auditors. Nobody performs a serious audit investigation from a phone screen. The mobile app is for field work — registering assets, checking stock, quick lookups. The detailed audit trail belongs on the web platform.

If the panel asks "why is the detailed audit trail not on mobile?", say:

> *"The audit trail is a governance feature for office-based administrators and auditors. The mobile application is designed for field officers doing operational work. Putting a full audit investigation interface on a small phone screen would be poor UX design and is outside the field officer's job scope. The same audit data is available to anyone who needs it through the web platform."*

PART 24: COMPLETE FILE-BY-FILE EXPLANATION — Every folder and every file

This part explains the ENTIRE project, folder by folder, file by file. After reading this, if someone mentions any file, you will know exactly what it does.

Project structure overview

```
govasset_platform/
├── manage.py                          # Entry point — everything starts here
├── config/                            # Central settings, URLs, server entry
│   ├── settings.py                    # ALL configuration in one file
│   ├── urls.py                        # URL routing map
│   ├── asgi.py                        # For async deployment
│   └── wsgi.py                        # For production deployment
├── authentication/                    # Users, login, API, security
│   ├── models.py                      # CustomUser, PendingAccess, LoginAttempt
│   ├── views.py                       # Login/logout web pages
│   ├── user_views.py                  # User CRUD web pages
│   ├── dashboard_views.py             # Dashboard stats
│   ├── pending_access_views.py        # Approve/reject new users
│   ├── api_views.py                   # Mobile app login/refresh/verify API
│   ├── api_serializers.py             # Convert data to/from JSON
│   ├── api_permissions.py             # Who can access each API
│   ├── api_exception_handler.py       # Standard error format
│   ├── api_urls.py                    # API endpoint routing
│   ├── decorators.py                  # Security guards for web pages
│   ├── middleware.py                  # Sets schema on every request
│   ├── oidc_backend.py                # Keycloak ↔ Django bridge
│   ├── keycloak_admin.py              # Talk to Keycloak admin API
```

```
│   └── pagination.py                # Split lists into pages
├── assets/                          # Asset register app
│   ├── models.py                    # Asset and AssetCategory models
│   ├── views.py                     # Asset web pages (list, create, detail,
edit, delete)
│   └── api_views.py                 # Asset API for mobile app
├── organizations/                   # Org hierarchy, master data, audit
│   ├── models.py                    # OrgUnit, MasterData, AuditLog
│   ├── views.py                     # Org unit web pages
│   ├── master_data_views.py         # Dropdown options management
│   └── api_views.py                 # Org + dashboard API
├── tenants/                         # Multi-tenancy management
│   ├── models.py                    # Ministry and Domain models
│   └── views.py                     # Ministry onboarding web pages
├── templates/                       # All HTML pages (26 files)
│   ├── shared/                      # Base layout (sidebar, topbar)
│   ├── authentication/              # Login, user management pages
│   ├── assets/                      # Asset pages (list, form, detail)
│   ├── tenants/                     # Ministry management pages
│   ├── dashboard/                   # Dashboard page
│   └── organizations/               # Org unit, master data pages
└── static/                          # CSS files
    └── css/
        └── style.css                # All custom styles (758 lines)
```

FOLDER 1: `config/` — The project brain

This folder contains EVERYTHING that makes the project work: settings, URLs, server entry points. Without this folder, Django wouldn't know what to do.

`config/__init__.py`

Just tells Python "this folder is a package." Empty file.

`config/settings.py` — The MOST important file (390 lines)

Purpose: ONE file that controls EVERYTHING — which apps are installed, which database to use, how authentication works, what middleware runs, how logging works, security settings.

Think of it as: The control panel of an airplane. Every switch, every setting, every configuration lives here.

Key sections explained simply:

```
# Lines 28-70: Which apps are shared vs per-ministry
SHARED_APPS = [
    'django_tenants',      # The multi-tenancy library itself
    'tenants',             # Ministry model (public schema)
    'authentication',      # Users, auth (public schema — one user table for all)
    'rest_framework',      # DRF — makes APIs easy
    'corsheaders',         # Lets Flutter call our API from a different address
```

```
    'mozilla_django_oidc', # Handles Keycloak handshake
    'drf_yasg',            # Creates Swagger docs automatically
]

TENANT_APPS = [
    'organizations',       # Org hierarchy, audit logs (per-ministry)
    'assets',              # Asset register (per-ministry)
]
```

```
# Lines 135-143: Database connection
DATABASES = {
    'default': {
        'ENGINE': 'django_tenants.postgresql_backend',
        # ↑ This is the magic! django-tenants adds SET search_path before every
query
    }
}
```

```
# Lines 217-235: JWT token settings
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=30),     # Token dies in 30 min
    'REFRESH_TOKEN_LIFETIME': timedelta(days=1),        # Refresh lasts 1 day
    'ROTATE_REFRESH_TOKENS': True,                      # New refresh each time
    'BLACKLIST_AFTER_ROTATION': True,                   # Old refresh stops
working
}
```

```
# Lines 461-510: Logging
LOGGING = {
    # Writes to TWO files:
    # django.log — normal messages
    # security.log — IMPORTANT: security events (logins, failures, access denied)
}
```

`config/urls.py` **— The roadmap (169 lines)**

Purpose: Maps every URL to the code that handles it. When someone types a URL, Django checks this file to find which function to run.

Think of it as: A restaurant menu — each dish (URL) has a kitchen (function) that prepares it.

```
# Example — how URL routing works:
urlpatterns = [
    path('login/', login_view, name='login'),
```

```
    # When user types: http://moh.localhost:8000/login/
    # Django calls: login_view() function

    path('assets/', asset_list_view, name='asset_list'),
    # When user types: http://moh.localhost:8000/assets/
    # Django calls: asset_list_view() function

    path('api/', include('authentication.api_urls')),
    # All API URLs start with /api/... and are defined in api_urls.py
]
```

Main URL groups:

URL pattern	What it does	Who uses it
`/login/`, `/logout/`	Web login/logout pages	Browser users
`/dashboard/`	Main dashboard	Logged-in users
`/assets/...`	Asset CRUD pages	Ministry admins, clerks
`/users/...`	User management	Admins
`/ministries/...`	Ministry onboarding	Super Admin
`/api/...`	All REST API endpoints	Mobile app, other groups
`/api/docs/`	Swagger documentation	Everyone
`/oidc/...`	Keycloak SSO handshake	Browser users

`config/asgi.py` and `config/wsgi.py`

Purpose: Entry points for running Django on a server. ASGI is for modern async servers, WSGI is for traditional servers.

When you use them: NEVER in development. Only when deploying to production (cloud server). The cloud server (like Gunicorn) reads one of these files to start Django.

Simplified: When you type `python manage.py runserver`, Django ignores these files. When you type `gunicorn config.wsgi:application`, it uses wsgi.py.

FOLDER 2: `authentication/` — The most important folder

This folder handles EVERYTHING related to users, login, security, and APIs. It is the biggest folder (20+ files). If you understand this folder, you understand 70% of the project.

`authentication/__init__.py`

Empty package marker.

`authentication/apps.py`

Purpose: Tells Django this folder is an app named "authentication".

`authentication/models.py` **— User database design (199 lines)**

Purpose: Defines THREE database tables: CustomUser, PendingAccess, LoginAttempt.

1. CustomUser (the main user table):

```
class CustomUser(AbstractUser):
    # Django's default user has: username, password, email, first_name, last_name
    # We ADDED these extra fields:
    role = models.CharField(
        choices=[   # A user can only be ONE of these:
            'SUPER_ADMIN',      # Top-level — manages all ministries
            'MINISTRY_ADMIN',   # Manages ONE ministry
            'AGENCY_MANAGER',   # Manages one agency within a ministry
            'FACILITY_CLERK',   # Manages assets at one facility
            'AUDITOR',          # Can ONLY read, cannot create/edit/delete
        ]
    )
    ministry_schema = models.CharField()  # "moh_schema" — ties user to their
ministry
    keycloak_id = models.CharField()       # Links to Keycloak account
    phone = models.CharField()             # Phone number
```

2. PendingAccess (users waiting for approval):

```
class PendingAccess(models.Model):
    # Created when someone tries to log in via Keycloak
    # but doesn't have a Django account yet
    email = models.EmailField()
    keycloak_id = models.CharField()
    status = models.CharField()  # PENDING → APPROVED or REJECTED
    # Admin must approve before they can log in
```

3. LoginAttempt (brute-force protection):

```
class LoginAttempt(models.Model):
    MAX_ATTEMPTS = 5           # After 5 failures...
    LOCKOUT_MINUTES = 15       # ...locked for 15 minutes

    username = models.CharField()
    ip_address = models.CharField()
    attempts = models.IntegerField(default=0)
    locked_until = models.DateTimeField(null=True)

    def is_locked(self):
```

```
        # Returns: "Is this user currently locked out?"
        return self.locked_until and self.locked_until > now()
```

`authentication/views.py` **— Login/logout pages (145 lines)**

Purpose: Shows the login page and handles logging out.

login_view: Shows a page with a "Sign in with Government SSO" button. When clicked, it redirects to Keycloak. Also displays "Your account is pending approval" messages.

logout_view: When user clicks logout:

1. Writes audit log: "User X logged out"
2. Destroys Django session
3. Redirects to Keycloak logout page (so Keycloak also forgets them)

Key code — how we detect failed logins:

```
def _is_locked_out(username, ip_address):
    attempt = LoginAttempt.objects.filter(
        username=username, ip_address=ip_address
    ).first()
    if attempt and attempt.is_locked:
        return attempt.minutes_remaining  # "Locked for 12 more minutes"
    return None
```

`authentication/user_views.py` **— User management (403 lines)**

Purpose: Web pages for creating, editing, activating/deactivating, and resetting passwords for users.

Views and what they do:

View	What it does	Who can access
`user_list_view`	Shows all users, filtered by ministry	Admins
`user_create_view`	Creates user in Keycloak + Django simultaneously	Admins
`user_edit_view`	Edits user details, syncs to Keycloak	Admins
`user_toggle_active_view`	Enables/disables a user account	Admins
`user_reset_password_view`	Resets password in Django + Keycloak	Admins

Key code — creating a user in BOTH systems:

```
def user_create_view(request):
    if request.method == 'POST':
        # Step 1: Create in Keycloak first
        keycloak = KeycloakAdminService()
```

```
        kc_id = keycloak.create_user(form_data)  # Returns Keycloak user ID

        # Step 2: Create in Django
        user = CustomUser.objects.create(
            username=form_data['username'],
            keycloak_id=kc_id,  # ← Links Django user to Keycloak user
            role=form_data['role'],
            ministry_schema=form_data['ministry_schema'],
        )
        # If Keycloak fails → Django does NOT create the user (rollback)
```

`authentication/dashboard_views.py` **— Dashboard stats (145 lines)**

Purpose: Shows the main dashboard with statistics and expiry warnings.

dashboard_view:

- Super Admin sees: "Total ministries: 5, Total users: 120, Total assets: 5,000"
- Ministry users see: "Your assets: 500, Expired: 12, Expiring soon: 25"

```
def _get_ministry_stats(ministry_schema):
    with schema_context(ministry_schema):
        # This ONLY queries their ministry's schema!
        total_assets = Asset.objects.count()
        expired = Asset.objects.filter(asset_expiry_date__lt=date.today())
        expiring_30 = Asset.objects.filter(asset_expiry_date__range=[today,
30_days])
    return {'total': total_assets, 'expired': expired.count()}
```

`authentication/pending_access_views.py` **— Access approvals (261 lines)**

Purpose: Super Admin reviews users who tried to log in but weren't registered yet.

View	What it does
`pending_access_list_view`	Shows list of all pending/approved/rejected requests
`pending_access_review_view`	Approve or reject a specific request
`pending_access_clear_view`	Delete old rejected requests (30+ days)

`authentication/api_views.py` **— Mobile app authentication (473 lines)**

Purpose: REST API endpoints that the Flutter mobile app and other groups call. This is the MOST important file for API authentication.

Classes (each handles one API endpoint):

Class	URL	What it does
`LoginAPIView`	POST `/api/auth/login/`	Takes username + password → returns JWT tokens
`RefreshTokenAPIView`	POST `/api/auth/refresh/`	Takes old refresh token → returns new tokens
`MeAPIView`	GET `/api/auth/me/`	Returns current user's profile
`VerifyTokenAPIView`	GET/POST `/api/auth/verify-token/`	Checks if a token is valid
`LogoutAPIView`	POST `/api/auth/logout/`	Blacklists refresh token

Key code — Login with brute force protection:

```
class LoginAPIView(TokenObtainPairView):
    def post(self, request):
        username = request.data.get('username')
        ip = get_client_ip(request)

        # Check 1: Is this user locked out?
        locked = _is_locked_out(username, ip)
        if locked:
            return Response({'error': f'Locked {locked} minutes'}, status=429)

        # Check 2: Is the password correct?
        serializer = self.get_serializer(data=request.data)
        if serializer.is_valid():
            _clear_failed_attempts(username, ip)  # Success → reset counter
            return Response(serializer.validated_data)
        else:
            _record_failed_attempt(username, ip)   # Failure → increment counter
            return Response({'error': 'Invalid credentials'}, status=401)
```

`authentication/api_serializers.py` **— Data formatters (222 lines)**

Purpose: Converts between Python objects and JSON format. The FLutter app sends/recieves JSON — serializers handle the conversion.

Key classes:

Serializer	What it converts
`CustomTokenObtainPairSerializer`	Adds role, ministry_schema INSIDE the JWT token
`UserProfileSerializer`	User data → JSON for /api/auth/me/
`AssetSerializer`	Asset data → JSON for /api/assets/
`AuditLogSerializer`	Audit log → JSON

Key code — Embedding extra data in JWT:

```
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        # These fields are embedded INSIDE the token
        # So the mobile app can read them WITHOUT making an extra API call
        token['role'] = user.role
        token['ministry_schema'] = user.ministry_schema
        token['full_name'] = user.get_full_name()
        return token
```

`authentication/api_permissions.py` **— API bouncers (117 lines)**

Purpose: Checks "Does this user have permission to call this API?" before the API code runs.

Classes:

Class	Who it lets through
`IsSuperAdmin`	Only SUPER_ADMIN role
`IsMinistryAdmin`	SUPER_ADMIN + MINISTRY_ADMIN
`CanManageAssets`	Everyone can READ (GET). Only admins can WRITE (POST/PUT/DELETE)
`CanDeleteAssets`	Only SUPER_ADMIN + MINISTRY_ADMIN can delete
`CanViewAuditLogs`	SUPER_ADMIN, MINISTRY_ADMIN, AUDITOR
`HasMinistrySchema`	Blocks users without a ministry (Super Admin is exempt)

```
class CanManageAssets(BasePermission):
    def has_permission(self, request, view):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True  # Anyone can READ
        # Only certain roles can WRITE
        return request.user.role in ['SUPER_ADMIN', 'MINISTRY_ADMIN',
'AGENCY_MANAGER']
```

`authentication/api_exception_handler.py` **— Error formatter (36 lines)**

Purpose: When an API error happens, DRF returns different error formats for different errors. This file makes ALL errors the same format so the Flutter app can handle them consistently.

```
def custom_exception_handler(exc, context):
    response = exception_handler(exc, context)
    if response is not None:
```

```
        response.data = {
            'error': True,
            'message': str(response.data.get('detail', 'An error occurred')),
            'code': getattr(exc, 'default_code', 'error'),
            'status': response.status_code,
        }
    return response
```

Without this, errors look different each time:

```
Error 1: {"detail": "Authentication credentials were not provided."}
Error 2: {"username": ["This field is required."]}
Error 3: {"non_field_errors": ["Unable to log in"]}

**With this, ALL errors look the same:**
{"error": true, "message": "...", "code": "...", "status": 401}
```

`authentication/api_urls.py` **— API routing (89 lines)**

Purpose: Maps all API URLs to their view classes.

```
urlpatterns = [
    path('auth/login/', LoginAPIView.as_view(), name='api_login'),
    path('auth/refresh/', RefreshTokenAPIView.as_view(), name='api_refresh'),
    path('auth/me/', MeAPIView.as_view(), name='api_me'),
    path('auth/verify-token/', VerifyTokenAPIView.as_view(), name='api_verify'),
    path('auth/logout/', LogoutAPIView.as_view(), name='api_logout'),
    path('assets/', AssetListCreateAPIView.as_view(), name='api_asset_list'),
    path('assets/<int:id>/', AssetDetailAPIView.as_view(),
name='api_asset_detail'),
    path('assets/categories/', AssetCategoryListAPIView.as_view()),
    path('org-units/', OrgUnitListAPIView.as_view()),
    path('audit-logs/', AuditLogListAPIView.as_view()),
    path('dashboard/stats/', DashboardStatsAPIView.as_view()),
]
```

Result: Flutter calls `POST /api/auth/login/` → Django runs `LoginAPIView`

`authentication/decorators.py` **— Web page bouncers (73 lines)**

Purpose: Checks permissions BEFORE a web page loads. These are for browser pages (not APIs).

```
def login_required_custom(view_func):
    # "Guard" for web pages
    @wraps(view_func)
    def wrapper(request, *args, **kwargs):
        if not request.user.is_authenticated:
```

```
            messages.warning(request, "Please log in first.")
            return redirect('login')
        return view_func(request, *args, **kwargs)
    return wrapper

def role_required(*allowed_roles):
    # "Guard" that checks user's role
    def decorator(view_func):
        @wraps(view_func)
        def wrapper(request, *args, **kwargs):
            if request.user.role not in allowed_roles:
                messages.error(request, "You don't have permission.")
                return redirect('dashboard')
            return view_func(request, *args, **kwargs)
        return wrapper
    return decorator
```

Usage:

```
@login_required_custom          # First guard: "Are you logged in?"
@role_required('SUPER_ADMIN')    # Second guard: "Are you a Super Admin?"
def ministry_list_view(request):
    # This code ONLY runs if both guards say OK
    ...
```

`authentication/middleware.py` **— Schema setter (51 lines)**

Purpose: On EVERY request, this sets `request.schema_name` so that all views know which ministry's data to use.

```
class SchemaMiddleware(MiddlewareMixin):
    def process_request(self, request):
        if request.user.is_authenticated:
            request.schema_name = request.user.ministry_schema
            request.user_role = request.user.role
```

Why this is needed: Without this, every view would need to look up the user's ministry schema from the database. This makes it available automatically.

`authentication/oidc_backend.py` **— Keycloak bridge (146 lines)**

Purpose: The MIDDLEMAN between Keycloak and Django. When Keycloak says "this user is real", this file checks if the user exists in Django.

```
class GovAssetOIDCBackend(OIDCAuthenticationBackend):
    def filter_users_by_claims(self, claims):
```

```
        # Keycloak says: "This user is authenticated"
        # Django says: "Let me find this user in MY database"
        keycloak_id = claims.get('sub')
        try:
            return CustomUser.objects.get(keycloak_id=keycloak_id)
        except CustomUser.DoesNotExist:
            return self.UserModel.objects.none()

    def create_user(self, request, user_claims):
        # Keycloak says: "This user is real but not in Django"
        # We do NOT auto-create. We create a PendingAccess request.
        PendingAccess.objects.create(
            email=user_claims.get('email'),
            keycloak_id=user_claims.get('sub'),
        )
        return None  # ← "User NOT allowed yet. Admin must approve."
```

`authentication/keycloak_admin.py` **— Keycloak API (295 lines)**

Purpose: Talks directly to Keycloak's admin API to create/update/delete users. When an admin creates a user in Django, this file also creates them in Keycloak.

```
class KeycloakAdminService:
    def _get_admin_token(self):
        # Logs into Keycloak as admin to get permission to create users
        response = requests.post(
            'http://localhost:8180/realms/master/protocol/openid-connect/token',
            data={
                'client_id': 'admin-cli',
                'username': settings.KEYCLOAK_ADMIN_USERNAME,
                'password': settings.KEYCLOAK_ADMIN_PASSWORD,
                'grant_type': 'password',
            }
        )
        return response.json()['access_token']

    def create_user(self, data):
        # Creates user in Keycloak with role + ministry attributes
        response = requests.post(
            f'{KEYCLOAK_URL}/admin/realms/govasset/users',
            headers={'Authorization': f'Bearer {self.token}'},
            json={
                'username': data['username'],
                'email': data['email'],
                'enabled': True,
                'attributes': {
                    'role': [data['role']],          # ← Synced to Keycloak
                    'ministry_schema': [data['ministry_schema']],  # ← Synced!
                }
```

```
            }
        )
```

`authentication/pagination.py` **— Page splitter (36 lines)**

Purpose: When you have 1000 assets, you don't show all 1000 on one page. This splits them into pages of 20.

```python
def paginate_queryset(queryset, request, per_page=20):
    paginator = Paginator(queryset, per_page)
    page_number = request.GET.get('page', 1)
    page = paginator.get_page(page_number)
    return page, paginator
```

FOLDER 3: `assets/` — The asset register

This folder handles the MAIN DATA of the system — the assets themselves (laptops, vehicles, hospital beds, etc.).

`assets/models.py` **— Asset database design (245 lines)**

Purpose: Defines what an "asset" is and what "categories" exist.

AssetCategory: Categories like ICT, Furniture, Vehicles, Medical Equipment.

```python
class AssetCategory(models.Model):
    name = models.CharField()    # "ICT Equipment"
    code = models.CharField()    # "ICT" — used in asset numbers: MOH-ICT-0001
    is_active = models.BooleanField(default=True)
```

Asset (30+ fields): The main model with everything about an asset.

```
IDENTIFICATION
  asset_number → "MOH-ICT-2025-0042" (auto-generated, unique barcode)
  name         → "Dell Latitude 5420"
  serial_number → "SN123456789" (manufacturer's serial)

CATEGORY AND STATUS
  category     → Points to AssetCategory (e.g., ICT)
  status       → ACTIVE, UNDER_MAINTENANCE, DECOMMISSIONED, DISPOSED
  condition    → EXCELLENT, GOOD, FAIR, POOR, CRITICAL

LOCATION
  org_unit     → Which department/section
  location_description → "3rd Floor, Room 305"
```

```
DATES
  acquisition_date     → When bought
  warranty_expiry_date → When warranty ends
  asset_expiry_date    → When it should be replaced
  useful_life_years    → Expected lifespan

FINANCIAL
  acquisition_cost  → How much it cost
  current_value     → Depreciated value
  funding_source    → Where the money came from

TRACKING
  registered_by → Who created this record
  created_at    → When created
  updated_at    → When last modified
```

Key computed properties (automatically calculated):

```python
@property
def is_expired(self):
    """Has this asset passed its expiry date?"""
    return self.asset_expiry_date and self.asset_expiry_date < date.today()

@property
def expires_soon(self):
    """Will this asset expire within 90 days?"""
    if self.asset_expiry_date:
        return 0 < (self.asset_expiry_date - date.today()).days <= 90
    return False
```

`assets/views.py` **— Asset web pages (577 lines)**

Purpose: Web pages for browser-based asset management.

View	URL	What it does
`asset_list_view`	`/assets/`	Shows paginated list with search + filters
`asset_create_view`	`/assets/create/`	Form to add new asset, auto-generates number
`asset_detail_view`	`/assets/<id>/`	Full asset detail with expiry warnings
`asset_edit_view`	`/assets/<id>/edit/`	Edit form, captures old/new values for audit
`asset_delete_view`	`/assets/<id>/delete/`	Delete (MINISTRY_ADMIN only)

Key code — auto-generating asset numbers:

```python
def generate_asset_number(category_code, schema_name):
    # Result example: "MOH-ICT-2025-0042"
```

```
    prefix = schema_name[:3].upper()     # "MOH"
    year = str(date.today().year)        # "2025"
    last_asset = Asset.objects.filter(
        asset_number__startswith=f'{prefix}-{category_code}-{year}'
    ).order_by('-asset_number').first()
    next_num = int(last_asset.asset_number.split('-')[-1]) + 1 if last_asset else
1
    return f'{prefix}-{category_code}-{year}-{next_num:04d}'
```

`assets/api_views.py` **— Asset API (628 lines)**

Purpose: API endpoints that the Flutter mobile app uses to manage assets.

Class	URL	What it does
`AssetListCreateAPIView`	GET/POST `/api/assets/`	List assets or create new one
`AssetDetailAPIView`	GET/PUT/DELETE `/api/assets/<id>/`	Get/update/delete one asset
`AssetCategoryListAPIView`	GET `/api/assets/categories/`	Get list of categories for dropdowns

Key code — creating an asset via API with automatic audit logging:

```
class AssetListCreateAPIView(APIView):
    permission_classes = [IsAuthenticated, HasMinistrySchema, CanManageAssets]

    def post(self, request):
        serializer = AssetSerializer(data=request.data)
        if serializer.is_valid():
            with schema_context(request.user.ministry_schema):
                asset = serializer.save(
                    registered_by_id=request.user.id,
                    asset_number=generate_asset_number(...)
                )
                # Auto-record audit log
                AuditLog.objects.create(
                    action='CREATE',
                    model_name='Asset',
                    object_id=asset.id,
                    new_value=serializer.data,
                )
                return Response(serializer.data, status=201)
```

FOLDER 4: `organizations/` — Org hierarchy, master data, audit

`organizations/models.py` **— Three important models (230 lines)**

1. OrgUnit (Organization Units):

```
class OrgUnit(models.Model):
    # Three-level hierarchy
    UNIT_TYPES = [
        ('MINISTRY', 'Ministry'),    # Top level (e.g., Ministry of Health)
        ('AGENCY', 'Agency'),        # Middle level (e.g., Tanzania Medicines
Authority)
        ('FACILITY', 'Facility'),    # Bottom level (e.g., Muhimbili Hospital)
    ]
    name = models.CharField()        # "Muhimbili National Hospital"
    code = models.CharField()        # "MNH"
    unit_type = models.CharField(choices=UNIT_TYPES)
    parent = models.ForeignKey('self', null=True)  # Points to parent unit
```

2. MasterData (dropdown options):

```
class MasterData(models.Model):
    # Configurable reference data per ministry
    CATEGORIES = [
        ('FUNDING_SOURCE', 'Funding Source'),     # "Government Budget", "Donor
Funded"
        ('ACQUISITION_METHOD', 'Acquisition Method'),  # "Purchase", "Donation"
        ('DISPOSAL_METHOD', 'Disposal Method'),   # "Auction", "Transfer",
"Destroyed"
        ('COST_CENTRE', 'Cost Centre'),           # "ICT Department", "HR
Division"
    ]
    category = models.CharField(choices=CATEGORIES)
    value = models.CharField()        # "GOV_BUDGET"
    label = models.CharField()        # "Government Budget"
    is_active = models.BooleanField(default=True)
```

3. AuditLog (tamper-proof records):

```
class AuditLog(models.Model):
    # IMMUTABLE — once created, cannot be changed or deleted
    performed_by_id = models.IntegerField()    # Who did it (Integer, not
ForeignKey!)
    performed_by_name = models.CharField()      # Their name at that time
    action = models.CharField()                 # CREATE, UPDATE, DELETE, LOGIN,
LOGOUT
    model_name = models.CharField()             # "Asset", "User", "OrgUnit"
    object_id = models.CharField()              # Which record was affected
    old_value = models.JSONField(null=True)     # What it WAS before
    new_value = models.JSONField(null=True)     # What it IS now
    timestamp = models.DateTimeField(auto_now_add=True)
    ip_address = models.CharField()
```

```
    def save(self, *args, **kwargs):
        if self.pk:  # If this record ALREADY exists
            raise Exception("AuditLog cannot be modified!")
        super().save(*args, **kwargs)

    def delete(self, *args, **kwargs):
        raise Exception("AuditLog cannot be deleted!")
```

Why IntegerField for user ID instead of ForeignKey? Because the users table is in the PUBLIC schema and the audit log is in each MINISTRY'S schema. PostgreSQL cannot create a foreign key across schemas.

`organizations/views.py` **— Org web pages (403 lines)**

View	What it does
`org_unit_list_view`	Shows org tree (Ministry → Agency → Facility)
`org_unit_create_view`	Creates Agency or Facility under a parent
`org_unit_edit_view`	Edits org unit name/code
`org_unit_delete_view`	Deletes org unit (cannot delete if has children)
`audit_log_view`	Shows audit trail with action filter

`organizations/master_data_views.py` **— Dropdown management (589 lines)**

View	What it does
`master_data_list_view`	Lists all dropdown options grouped by category
`master_data_create_view`	Adds new dropdown option
`master_data_edit_view`	Edits dropdown option
`master_data_delete_view`	Deletes dropdown option
`master_data_seed_view`	Loads default options (funding sources, methods, etc.)
`asset_category_list_view`	Lists asset categories
`asset_category_create_view`	Creates asset category
`asset_category_edit_view`	Edits asset category

`organizations/api_views.py` **— Org + dashboard API (424 lines)**

Class	URL	What it does
`OrgUnitListAPIView`	GET `/api/org-units/`	Returns org tree + flat facilities list
`AuditLogListAPIView`	GET `/api/audit-logs/`	Paginated audit trail with filters

Class	URL	What it does
`DashboardStatsAPIView`	GET `/api/dashboard/stats/`	Dashboard statistics

FOLDER 5: `tenants/` — Multi-tenancy management

`tenants/models.py` — Ministry and Domain (47 lines)

Purpose: Defines what a "ministry" (tenant) is and how domains map to ministries.

```
class Ministry(TenantMixin):
    # TenantMixin gives us: schema_name (e.g., "moh_schema")
    # auto_create_schema = True means when we save a Ministry,
    # PostgreSQL creates a NEW SCHEMA automatically

    name = models.CharField()       # "Ministry of Health"
    short_name = models.CharField() # "MOH"
    is_active = models.BooleanField(default=True)

class Domain(DomainMixin):
    domain = models.CharField()     # "moh.localhost" or "moh.platform.go.tz"
    is_primary = models.BooleanField(default=True)
    tenant = models.ForeignKey(Ministry)  # This domain belongs to MOH
```

The magic of TenantMixin: When you create a `Ministry(name="Health")`, django-tenants automatically:

1. Runs `CREATE SCHEMA moh_schema` in PostgreSQL
2. Creates ALL tenant tables inside it (assets_asset, organizations_orgunit, etc.)
3. Sets up the domain mapping

`tenants/views.py` — Ministry management (261 lines)

View	What it does	Who can access
`ministry_list_view`	Lists all ministries with stats	SUPER_ADMIN
`ministry_create_view`	Creates new ministry + schema + domain + root org unit	SUPER_ADMIN
`ministry_detail_view`	Shows ministry details: users, org units, recent assets	SUPER_ADMIN
`ministry_toggle_active_view`	Activate/deactivate a ministry	SUPER_ADMIN

Key code — onboarding a new ministry:

```
def ministry_create_view(request):
    if request.method == 'POST':
```

```
        # Step 1: Create Ministry (this auto-creates the PostgreSQL schema!)
        ministry = Ministry.objects.create(
            name=form.cleaned_data['name'],
            schema_name=form.cleaned_data['schema_name'],  # e.g., "moh_schema"
        )
        # Step 2: Create domain mapping
        Domain.objects.create(
            domain=form.cleaned_data['domain'],  # e.g., "moh.localhost"
            tenant=ministry,
            is_primary=True,
        )
        # Step 3: Create root org unit
        with schema_context(ministry.schema_name):
            OrgUnit.objects.create(
                name=ministry.name,
                unit_type='MINISTRY',
            )
```

FOLDER 6: `templates/` — All HTML pages (26 files)

`templates/shared/base.html`

Purpose: The MAIN LAYOUT that every page extends. Contains:

- Left sidebar with navigation links
- Top bar with search and user menu
- Message area for success/error messages
- Footer with user info

All other templates just fill in the content area. They "extend" base.html:

```
{% extends 'shared/base.html' %}
{% block content %}
    <!-- Page-specific content goes here -->
{% endblock %}
```

`templates/authentication/login.html`

Purpose: Two-panel login page. Left side: branding. Right side: "Sign in with Government SSO" button.

`templates/authentication/user_list.html`

Purpose: User table with avatar, name, email, role badge, status, actions.

`templates/authentication/user_form.html` and `user_edit.html`

Purpose: Create/edit user forms with role and ministry selection.

`templates/authentication/pending_access_list.html` **and** `pending_access_review.html`

Purpose: List and approve/reject pending access requests.

`templates/dashboard/dashboard.html`

Purpose: Main dashboard with stat cards and expiry warning tables.

`templates/assets/asset_list.html`

Purpose: Asset register table with search, filters, pagination, and action buttons.

`templates/assets/asset_form.html`

Purpose: Create/edit asset form (two-column layout).

`templates/assets/asset_detail.html`

Purpose: Full asset detail view with all fields displayed.

`templates/tenants/ministry_list.html`, `ministry_form.html`, `ministry_detail.html`

Purpose: Ministry management pages (list, create, detail).

`templates/organizations/` **(8 files)**

Purpose: Org unit management, master data management, audit log viewer, asset category management.

`templates/shared/pagination.html`

Purpose: Reusable pagination component (Previous, Page 1, Page 2, Next...). Included by any page that has lists.

FOLDER 7: `static/` — CSS and images

`static/css/style.css` **(758 lines)**

Purpose: ALL styling for the entire application. One file controls everything visual.

Key sections:

```
Lines 4-58:    CSS variables (colors, sizes, shadows) — THE DESIGN SYSTEM
Lines 73-212:  Sidebar layout and navigation
Lines 214-259: Main content area layout
Lines 261-283: Card styling
Lines 285-331: Dashboard stat cards
Lines 333-371: Table styling
Lines 373-403: Badges (status labels)
Lines 430-469: Buttons
```

```
Lines 471-539: Forms and inputs
Lines 541-557: Alert messages
Lines 657-758: Login page styling
```

ROOT FILE: `manage.py`

Purpose: The ENTRY POINT for EVERYTHING. Every command goes through this file.

```python
#!/usr/bin/env python
import os
import sys

def main():
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'config.settings')
    # ↑ Tells Django where to find settings
    try:
        from django.core.management import execute_from_command_line
    except ImportError:
        raise ImportError("Django is not installed.")
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```

Commands you pass to it:

```
python manage.py runserver            # Start development server
python manage.py migrate              # Create/update database tables
python manage.py migrate_schemas      # Create/update tables in ALL schemas
python manage.py createsuperuser      # Create admin account
python manage.py setup_demo_data      # Seed demo data for presentation
python manage.py shell                # Open Python command line
python manage.py test                 # Run tests
```

Summary: What to remember

If they ask about...	Go to this file/folder
Login process	`authentication/views.py` and `oidc_backend.py`
User accounts	`authentication/models.py` (CustomUser)
User management	`authentication/user_views.py` and `keycloak_admin.py`
API authentication	`authentication/api_views.py`

If they ask about...	Go to this file/folder
JWT tokens	`authentication/api_serializers.py` and `config/settings.py` (SIMPLE_JWT)
Permissions (API)	`authentication/api_permissions.py`
Permissions (web)	`authentication/decorators.py`
Assets (web)	`assets/views.py`
Assets (API)	`assets/api_views.py`
Assets (database)	`assets/models.py`
Organization units	`organizations/models.py` and `views.py`
Audit log	`organizations/models.py` (AuditLog)
Master data/dropdowns	`organizations/master_data_views.py`
Multi-tenancy	`tenants/models.py`
Onboard a ministry	`tenants/views.py`
All settings	`config/settings.py`
URL routing	`config/urls.py`
API URL routing	`authentication/api_urls.py`
Swagger docs	`http://localhost:8000/api/docs/`
Login page	`templates/authentication/login.html`
Dashboard	`templates/dashboard/dashboard.html`
All styling	`static/css/style.css`
All commands & setup	Part 25 below

PART 25: IMPORTANT COMMANDS & SETUP REFERENCE — Everything you need to run the system

> **In this part:**
>
>

> - Correct startup order

Step 0 — PostgreSQL (Do Nothing)

PostgreSQL starts automatically when Windows boots. You never need to touch it. If you want to confirm it is running, press **Win + R**, type `services.msc`, find **postgresql-x64-17** (or similar name) — it should say **Running**. That is all.

Step 1 — Find Your IP Address (Do This First, Every Time)

Open any terminal and run:

```
ipconfig
```

Look for **IPv4 Address** under your active adapter. If you are using a phone hotspot, it will look like `192.168.43.x`. Write down that number — you will use it in Steps 3 and 4.

Step 2 — Start Keycloak

Open a terminal, navigate to the Keycloak bin folder, then start it:

```
cd C:\keycloak\keycloak-26.6.2\bin
kc.bat start-dev --http-port=8180
```

Why `start-dev`: the normal `start` command requires SSL certificates and a production setup. `start-dev` skips all that and runs Keycloak in development mode — perfect for your project and presentations.

Why `--http-port=8180`: Keycloak's default port is 8080, which would clash with Django or other services. Port 8180 keeps them separate.

Wait until you see this line before continuing:

```
Keycloak 26.6.2 on JVM started
```

Keycloak is slow — give it about 30–60 seconds. Do not close this terminal window.

Step 3 — Start Django

Open a **second, separate** terminal window (keep Keycloak running in the first one):

```
cd D:\government_asset_platform
```

```
venv\Scripts\activate
```

You should see `(venv)` appear at the start of your terminal line. That means the virtual environment is active. If you accidentally use the wrong venv, nothing will work — always activate from `D:\government_asset_platform\venv`.

Now start Django:

```
python manage.py runserver 0.0.0.0:8000
```

Why `0.0.0.0`: this tells Django to listen to ALL devices in the room (your phone, your laptop), not just itself. Without this, your phone cannot reach Django at all, even on the same hotspot.

Why `8000`: this is your port number — the specific door Django listens on.

Keep this terminal open. Do not close it.

Step 4 — Connect Your Phone (Do This Every Time the IP Changes)

Every time you switch WiFi networks or hotspots, your IP address changes. You must update these **four things** every single time:

4A — Update the Flutter App's IP

Open this file:

```
C:\Users\Hemed\govasset_mobile\lib\config\api_config.dart
```

Find this line and change the IP:

```
static const String serverIp = 'PUT_YOUR_NEW_IP_HERE';
```

For example, if your IP is `192.168.43.105`:

```
static const String serverIp = '192.168.43.105';
```

Save the file.

4B — Update Django's Allowed Hosts

Open this file:

```
D:\government_asset_platform\config\settings.py
```

Find `ALLOWED_HOSTS` and add your new IP:

```
ALLOWED_HOSTS = [
    'localhost',
    '127.0.0.1',
    '.localhost',
    '192.168.43.105',   # ← your new IP here
]
```

Save the file.

4C — Register the New IP as a Domain in Django (Most Forgotten Step)

This is the step that causes the **404 error** and **"no internet connection"** on the phone if skipped. Django-tenants checks every incoming request against a registered domain list. If your new IP is not in that list, it turns the phone away before your code even runs.

Run these commands:

```
cd D:\government_asset_platform
venv\Scripts\activate
python manage.py shell
```

Once inside the shell, paste these lines exactly, replacing the IP with yours:

```
from tenants.models import Domain, Ministry
public = Ministry.objects.get(schema_name='public')
Domain.objects.get_or_create(domain='192.168.43.105', tenant=public,
is_primary=False)
print("Done")
exit()
```

You should see `Done` printed. Now exit the shell with `exit()`.

4D — Reinstall the App on Your Phone

Connect your phone to your laptop with a USB cable. Make sure **USB debugging** is ON in your phone's developer settings. Then run:

```
cd C:\Users\Hemed\govasset_mobile
flutter run
```

This rebuilds the app with the new IP address baked in and installs it fresh on your phone. Wait until the app opens on your phone screen before unplugging the USB cable.

Why you must rebuild: the IP address is baked into the app at build time. Changing the file alone does nothing — the app needs to be rebuilt and reinstalled so the new IP is inside the installed version on your phone.

After `flutter run` finishes and the app opens on your phone, you can unplug the USB cable. The app will keep working over WiFi/hotspot.

Step 5 — Load Clean Demo Data (Run Before Every Presentation)

```
cd D:\government_asset_platform
venv\Scripts\activate
python manage.py setup_demo_data
```

This command is a **complete environment reset** — run it before every presentation to guarantee a known, predictable state.

What it does:

1. **Cleans** all assets, audit logs, login attempts, and ALL users (orphan + old test accounts)
2. **Creates** 5 asset categories (ICT, VEH, FURN, MEDICAL, LAND) in every schema
3. **Creates** org units (MOH → MNH → RAD for moh_schema, MOF for mof_schema)
4. **Creates** 6 demo users in Django — and optionally in Keycloak
5. **Creates** 13 assets + audit logs across 2 ministries

What it creates:

What	Details
5 categories	ICT, VEH, FURN, MEDICAL, LAND — auto-created in every schema
Org units	MOH → Muhimbili National Hospital → Radiology (moh_schema); Ministry of Finance (mof_schema)
6 users	superadmin, moh_admin, mnh_manager, rad_clerk, moh_auditor, mof_admin (all password: `Admin@123`)
13 assets	9 for Ministry of Health + 4 for Ministry of Finance
Audit logs	Sample login, create, and update records per ministry

Keycloak sync:

- By default, users are ALSO created in Keycloak so SSO web login works immediately
- If Keycloak is not running, use `--no-keycloak`:

  ```
  python manage.py setup_demo_data --no-keycloak
  ```

  Users will exist in Django only (API login works, SSO login will not).

Flags:

```
python manage.py setup_demo_data              # Full reset + Keycloak sync
(default)
python manage.py setup_demo_data --no-keycloak # Full reset, skip Keycloak
python manage.py setup_demo_data --clean       # Clean only (no seed)
python manage.py setup_demo_data --seed        # Seed only (no clean)
```

Run with no flags for a complete reset. One command. Every time. Predictable.

State after running this command:

```
=== USERS ===
  ID 28: superadmin  | SUPER_ADMIN    | schema=None
  ID 29: moh_admin   | MINISTRY_ADMIN | schema=moh_schema
  ID 30: mnh_manager | AGENCY_MANAGER | schema=moh_schema
  ID 31: rad_clerk   | FACILITY_CLERK | schema=moh_schema
  ID 32: moh_auditor | AUDITOR        | schema=moh_schema
  ID 33: mof_admin   | MINISTRY_ADMIN | schema=mof_schema
  Total: 6 users (all password: Admin@123)

=== MINISTRIES (with data) ===
  moh_schema — 9 assets, 3 audit logs, 5 categories, 3 org units
  mof_schema — 4 assets, 3 audit logs, 5 categories, 1 org unit

=== ASSETS ===
  moh_schema (9): Dell Laptop, HP Printer, Cisco Switch, Samsung TV,
                  Land Cruiser Prado, Hiace Ambulance, Haematology Analyzer,
                  Patient Monitor, Executive Desk Set
  mof_schema (4): HP EliteBook Laptop, Lenovo Desktop,
                  Toyota Fortuner, Conference Furniture Set

=== AUDIT LOGS ===
  Each ministry with assets: 1 LOGIN + 1 CREATE + 1 UPDATE

Orphan users (test_user, mos_admin, moa_admin) deleted.
Stale schemas (moe_schema, mos_schema, moa_schema) remain with no data.
```

Run `--seed` to recreate without cleaning. Run `--clean` for a blank slate.

Step 6 — Open Everything in the Browser

Once Django and Keycloak are both running:

What	Address
Web platform	`http://localhost:8000`
API documentation (Swagger)	`http://localhost:8000/api/docs/`
Keycloak admin panel	`http://localhost:8180`
Django admin panel	`http://localhost:8000/admin/`

To test from your phone's browser on the same hotspot, replace `localhost` with your IP:

```
http://192.168.43.105:8000
```

Quick Troubleshooting Reference

Problem you see	Most likely cause	Fix
Phone shows "No internet connection"	IP in `api_config.dart` is old	Update `serverIp` and run `flutter run` again
Phone gets a 404 error	New IP not registered as Domain	Run Step 4C again with new IP
Web login does nothing or errors	Keycloak not started	Run Step 2, wait for it to fully load
`ModuleNotFoundError` when starting Django	Wrong virtual environment	Run `venv\Scripts\activate` from `D:\government_asset_platform`
Django says `ALLOWED_HOSTS` error	New IP not in settings.py	Run Step 4B
Phone app shows old data after IP change	App not rebuilt	Reconnect USB and run `flutter run` again
Keycloak page not loading	Still starting up	Wait 60 seconds, try again
Web page has no styling / CSS broken	Browser cached old CSS	Press **Ctrl+Shift+R** (hard refresh) — or clear browser cache

---

Test Accounts (All Use Password: `Admin@123`)

Username	Role	Ministry
`superadmin`	Super Admin	Platform-wide
`moh_admin`	Ministry Admin	Ministry of Health
`mnh_manager`	Agency Manager	Ministry of Health
`rad_clerk`	Facility Clerk	Ministry of Health
`moh_auditor`	Auditor	Ministry of Health
`mof_admin`	Ministry Admin	Ministry of Finance

The Correct Startup Order (Every Time)

```
1. Check IP (ipconfig)
2. Start Keycloak (wait for it to fully load)
3. Start Django (python manage.py runserver 0.0.0.0:8000)
4. If IP changed → update api_config.dart + settings.py + register domain +
flutter run
5. Test web login in browser
6. Test mobile login on phone
7. Run setup_demo_data if doing a presentation
```

Print or screenshot this page and keep it next to your laptop.

PART 26: THE FULL 10-GROUP PROJECT — Where Group 1 Fits

> **In this part:**
>
> - The complete project structure
> - Group 1 is the FOUNDATION
> - Dependency matrix — every group depends on us
> - What each group delivers
> - How our API serves all groups
> - Panel talking points

The Complete Project Structure

The whole project is called **"Government Asset Management System"** and is split into **10 groups**. Each group builds one module. Together they cover the complete lifecycle of government assets — from registration to disposal, including maintenance, finance, GIS, compliance, and analytics.

Group	Module Name	Focus Area	Dissertation Title
1	**Administration, Security & Multitenancy**	RBAC, SSO, audit trails, master data, multi-tenant isolation	*Design and Implementation of a Secure Multi-Tenant Platform for*

Group	Module Name	Focus Area	Dissertation Title
			Government Asset Management Systems
2	Asset & Property Register Module	Centralized asset registry, classification, hierarchy, document management	*Centralized Digital Asset Registry with Lifecycle Tracking for Public Sector Properties*
3	Lifecycle Planning & Capital Projects	Asset lifecycle planning, condition assessment, CAPEX analysis	*Lifecycle Cost Analysis and Capital Project Portfolio Management System*
4	Maintenance & Operations (CMMS)	Work orders, preventive maintenance, service contracts, mobile technician app	*Computerized Maintenance Management System with Mobile Field Service Capabilities*
5	Financial Management & Valuation	Depreciation, lease management, budgeting, financial reporting	*Integrated Financial Management System for Asset Valuation, Depreciation and Lease Accounting*
6	Land Management & GIS Integration	Cadastral data, GIS visualization, space management, room bookings	*GIS-Enabled Spatial Management System with BIM Integration for Smart Facilities*
7	Energy, Sustainability & IoT	Utility monitoring, smart meters, carbon footprint, ESG dashboards	*IoT-Enabled Energy Monitoring and Sustainability Management System for Smart Buildings*
8	Compliance, Risk & Safety	Statutory inspections, risk registers, incident reporting, CAPA tracking	*Integrated Compliance Tracking and Risk Management System for Facility Safety*
9	Inventory, Stores & Procurement	Spare parts, barcode/RFID, purchase orders, supplier management	*RFID-Based Inventory Management System with Automated Procurement Workflows*
10	Analytics, Dashboards & Reporting	KPIs, standard reports, drill-down, predictive analytics, BI	*Business Intelligence and Decision Support System for Property Management Analytics*

Group 1 Is the FOUNDATION — Every Other Group Depends on Us

Group 1 is not just one module among ten. **Group 1 is the platform that every other module runs on top of.** Here is why:

What Group 1 provides to everyone else:

1. **Authentication** — Every user in every group is authenticated by our system. Groups without their own login use our Keycloak SSO page directly. Groups with their own login call our API endpoint — but the

user is still verified against our database and role system. Either way, we are the sole authentication provider.
2. **Authorization (RBAC)** — Our 5-role system (Super Admin → Facility Clerk) controls what each user can access. Groups 2-10 simply check "what role does this user have?"
3. **Multi-Tenant Isolation** — Each ministry's data is in a separate PostgreSQL schema. Groups 2-10 never see data from other ministries. Our django-tenants setup makes this automatic.
4. **API Security (JWT)** — Our SimpleJWT configuration issues tokens that all other groups use to authenticate API calls. The `/api/auth/verify-token/` endpoint is how Groups 2-10 validate users.
5. **Audit Trail** — Every action across ALL modules is logged in our AuditLog table with IP, user, timestamp, old/new values.
6. **Master Data** — Asset categories, locations, cost centres — these are configured in our master data tools and used by all other groups.
7. **User Management** — Only Group 1 creates and manages users. Groups 2-10 never need to create user accounts.

The dependency matrix from the project document:

```
Group 1 → depends on: NOBODY
         → everyone depends on: Groups 2, 3, 4, 5, 6, 7, 8, 9, 10

Group 2 → depends on: Group 1
Group 3 → depends on: Groups 1, 2
Group 4 → depends on: Groups 1, 2, 9
Group 5 → depends on: Groups 1, 2, 3
Group 6 → depends on: Groups 1, 2
Group 7 → depends on: Groups 1, 2, 6
Group 8 → depends on: Groups 1, 2, 4
Group 9 → depends on: Groups 1, 2, 4
Group 10 → depends on: ALL groups for data
```

Read this carefully: Group 1 is the **only group that depends on nobody**. Every single other group needs Group 1 to function. If our login system is down, the entire system stops — no one can access anything.

Dependency Matrix Explained Simply

Think of it like building a house:

- **Group 1** = The foundation and electricity. Without it, nothing works.
- **Group 2** = The rooms and walls (asset register). Built on the foundation.
- **Groups 3-9** = The plumbing, wiring, painting (modules). Each needs the rooms and the foundation.
- **Group 10** = The security cameras (analytics). Monitors everything.

If you try to build the walls (Group 2) without the foundation (Group 1), the walls collapse. If you try to paint (Group 7) without rooms (Group 2), where do you paint? This is exactly how the dependency matrix works.

What Group 1 Specifically Delivers (From the Project Document)

Scope (what we cover):

- Role-Based Access Control with hierarchical organization (ministry → agency → facility)
- Single Sign-On integration with Keycloak
- Full audit trails for changes to master data and transactions
- Configurable master data: asset categories, locations, cost centres, user roles, workflows
- Multi-tenancy: separate data, users, and configurations per ministry
- Centralized management dashboard

Key Deliverables (what we built):

1. RBAC system with hierarchical organization and multi-tenancy support
2. SSO integration (Keycloak) setup and configuration
3. Audit trail logs and reporting interface
4. Master data configuration tools (categories, locations, cost centres, roles)
5. User management dashboard and admin controls
6. Tenant management interface (ministry onboarding with schema isolation)
7. Security and access management documentation
8. REST API with JWT authentication for mobile and other groups

How Our API Serves All Groups 2-10

The API endpoints we built are not just for our own mobile app. They are the **official integration point** for all 9 other groups. Here is what each group uses from us:

Group	What they need from our API
2 (Asset Register)	User authentication + role check before CRUD operations
3 (Lifecycle)	Verify who can create/edit capital projects
4 (Maintenance)	Authenticate technicians on mobile app + assign work orders by role
5 (Finance)	Check if user has MINISTRY_ADMIN role to approve budgets
6 (GIS)	Authenticate users viewing spatial data
7 (Energy/IoT)	Authenticate system-to-system API calls from smart meters
8 (Compliance)	Check AUDITOR role for inspection reports
9 (Inventory)	Authenticate store clerks scanning barcodes
10 (Analytics)	Role-based dashboard access (executive vs technician views)

The key endpoint:

```
GET /api/auth/verify-token/
Authorization: Bearer <token>
→ Returns: { "role": "MINISTRY_ADMIN", "ministry_schema": "moh_schema", ... }
```

Any group can call this endpoint with a user's JWT token and instantly know their role, ministry, and permissions. This is how the entire system shares authentication through us.

Panel Talking Points — What to Say About the 10-Group Structure

If they ask: "How does your system connect to other groups?"

> *"Our platform is the security and administration foundation. We provide authentication, role-based access control, and multi-tenant data isolation that all 9 other groups depend on. Every user in the entire system logs in through our Keycloak SSO. Other groups call our API to verify user identities and check permissions. This is documented in the project's dependency matrix — Group 1 depends on nobody, but every other group depends on us."*

If they ask: "What if another group wants to add their own login?"

> *"They have two options. If their users don't mind leaving their site to authenticate, they use our Keycloak SSO — the user sees one unified government login across all 10 groups. If they want users to stay on their own site, their backend calls our API login endpoint directly — the user never leaves their site. In both cases, authentication still goes through us. We are the sole authentication provider — users are always verified against our database and our role system."*

If they ask: "How do you handle data isolation between ministries?"

> *"Each ministry has its own PostgreSQL schema, managed by django-tenants. When a user logs in, our system knows which schema to route them to based on their ministry_schema field. Groups 2-10 never see cross-ministry data because the database itself enforces the isolation at the schema level."*

If they ask: "What is your dissertation title?"

> *"Design and Implementation of a Secure Multi-Tenant Platform for Government Asset Management Systems."*

If they ask: "What specific security features did you implement?" (refer to Part 8 for the full 15-feature list, but here are the top ones relevant to Group 1's role):

> *"We implemented 15 security features including: Keycloak SSO authentication, role-based access control with 5 role levels, multi-tenant data isolation via PostgreSQL schemas, JWT token authentication with 30-minute access tokens, tamper-proof audit trail with IP tracking, account lockout after 5 failed attempts, and session management with configurable timeouts."*

If they ask: "How is your project different from the other groups?"

> *"We are the platform team. Other groups focus on specific business modules — asset register, maintenance, finance, GIS, compliance. We focus on security, identity, and data isolation. Without our platform, the other groups would have no authentication system, no user management, no audit trail, and no data isolation between ministries. Every single group depends on us."*

Why Group 1 Has Asset Registration (When Group 2 Is the Asset Register)

The question you will get:

> *"Group 2 is supposed to build the asset register. Why does your system also have asset registration? Isn't that overlapping?"*

The honest answer:

The asset registration code in Group 1's platform is a **proof-of-concept reference implementation**, not a competing module. It exists for two specific reasons:

1. **To prove multi-tenancy works.** Without assets stored in separate schemas, Group 1 cannot demonstrate that Ministry of Health data is isolated from Ministry of Finance data. The panel needs to see two different sets of assets under two different logins, confirmed by two different database schemas. Login pages and user lists alone don't prove data isolation.

2. **To provide demo seed data.** When showing the system to evaluators, Group 1 needs 13 realistic assets across two ministries so panelists can click around, see real data, and understand the user interface. Empty tables do not make a convincing demo.

How it coexists with Group 2's module:

There are three scenarios, and you are prepared for all of them:

Scenario	What happens	What you say
Group 2 built a Django app that extends our Asset model	Group 2's app adds richer fields (classifications, documents) alongside our simple asset records. Both live in the same database, same schemas, protected by our multi-tenancy.	*"Our asset registration is a minimal reference implementation to demonstrate the platform. Group 2 builds the full-featured asset register on top of our foundation."*
Group 2 built a standalone system that calls our API	Their system has its own asset database. Our internal assets are just demo data for our own UI. No data conflict.	*"Our internal assets are demo seed data for the platform UI. Group 2's system authenticates through our API and manages their own asset records independently."*
Group 2 did not build a working asset register	Our asset registration becomes the fallback that makes the overall project demo possible.	*"We built asset registration as a proof of concept to demonstrate multi-tenancy. It serves as a reference for how any group can build on our platform."*

The key point for the panel:

> *"Asset registration was never in Group 1's scope. Our deliverables are authentication, RBAC, multi-tenancy, audit trails, master data, and the REST API. The asset CRUD screens you see are a reference implementation — they prove that our multi-tenant platform actually works end-to-end. Group 2's module is the production-grade asset register. Our simple version either becomes demo seed data or gets replaced by Group 2's implementation. Either way, they depend on our authentication, our schemas, and our API to function."*

What Integration Means for Group 2's System — Audit Trail, Security, and Why We Matter

The question you asked:

> *"Will the integration include the user's identity, role, and ministry?" "Will audit trail and other security features also work for their system?" "Is our work truly significant? They said all groups depend on us."*

Let me answer each one clearly.

---

1. Yes — Group 2 gets the user's identity, role, AND ministry

Every API response from our system includes:

- `user.id` — Who the user is (database record)
- `user.username` — Their login name
- `user.full_name` — Their real name (Amina Hassan)
- `user.role` — Their permission level (MINISTRY_ADMIN, etc.)
- `user.ministry_schema` — Their ministry's database schema (moh_schema)
- `user.ministry` — Their ministry's display name (Ministry of Health)

This means Group 2 can:

- Show a user's name in their UI header
- Hide buttons the user doesn't have permission for (e.g., only MINISTRY_ADMIN can create assets)
- Scope ALL data queries to the user's ministry (never show Ministry of Finance data to a Ministry of Health user)
- Log the user's identity in their own audit trail

Without our API, Group 2 would have no way to know:

> *Who is using their system? What are they allowed to do? Which ministry's data should they see?*

---

2. Audit Trail — YES, but only for actions that go through our API

There are two levels:

What our system logs automatically:

Action	Logged in our AuditLog?	Details
User logs in via our API	✅ Yes	IP address, timestamp, username
User logs out	✅ Yes	Refresh token blacklisted + logout record
Token refresh	❌ No	Not logged (no security value)

What depends on how Group 2 integrates:

Integration method	Audit trail for asset operations	Security features active
Group 2 calls our API for every CRUD operation on assets	✅ Full audit — every create, update, delete logged with old/new values, IP, user	✅ All 15 security features (lockout, JWT, RBAC, multi-tenancy, etc.)
Group 2 has their own database and only calls our API for login/verify	❌ Only login/logout are logged in our system. Group 2 must build their own audit trail	✅ Authentication + role info is secure. Data isolation is Group 2's responsibility

For the panel:

> *"Our platform guarantees that every authentication event is logged. For asset operations, if Group 2 uses our API endpoints, the audit trail is automatic. If they store data in their own system, they need their own audit trail — but our API still tells them who performed each action."*

---

3. Is Group 1's work truly significant? The honest answer.

Short answer: Yes. Without Group 1, Groups 2-10 cannot function as designed.

Here is the exact dependency:

What Group 1 provides	What happens if Group 1's system is down
User login	No user in any group's system can log in. Every system is locked.
Role info	No group knows what a user is allowed to do. All permissions are blind.
Ministry identity	No group knows which ministry's data to show. Data leaks between ministries.
Master data	Categories, cost centers, location types — none available for dropdowns.
Token verification	Every API call from any group fails with 401. The entire ecosystem stops.

The dependency matrix says it clearly:

```
Group 1 depends on: NOBODY
Group 2 depends on: Group 1
Group 3 depends on: Group 1, Group 2
Group 4 depends on: Group 1, Group 2, Group 9
...
Group 10 depends on: ALL groups
```

Group 1 is the ONLY group that depends on nobody. Every other group needs Group 1 for their system to work at all. This is not exaggeration — without authentication, there is no session, no user identity, no role, no ministry context, no data isolation. Each group's system would be a standalone app with no connection to the rest of the project.

What you should say to the panel:

> *"Our platform is the authentication and security foundation for the entire 10-group project. When a user logs into Group 2's system, Group 2 calls our API to verify the user's identity and get their role and ministry. Without this call, Group 2 cannot know who the user is or what they should see. Every single group depends on this same mechanism. We are the only group that depends on nobody — because we are the foundation everything else is built on."*

PART 27: TONIGHT & TOMORROW CHECKLIST — What to do, when to do it

Tonight (Sunday July 5)

- [ ] **Verify everything starts** — Run startup sequence once more
- [ ] **Check your laptop** — Fully charged, charger packed, no pending Windows updates
- [ ] **Test the demo** — Click through login → assets → audit log → API docs
- [ ] **Open every page** — Dashboard, Assets, Audit Log, Users, Ministries, Master Data, Org Units, API Swagger
- [ ] **Read Part 19 aloud** — The one-paragraph answer. Say it until it flows naturally
- [ ] **Pack for tomorrow:** Laptop + charger, mouse, notebook, pen, water bottle, your phone (for mobile demo)
- [ ] **Set alarm** — Give yourself 1.5 hours before panel time
- [ ] **Sleep** — A tired brain forgets even simple things

Tomorrow Morning (Monday July 6)

- [ ] **Start PostgreSQL** — `net start postgresql-x64-17` (or whatever version)
- [ ] **Open pgAdmin** — Have `\dn` or the schema list ready
- [ ] **Start Keycloak** — `cd C:\keycloak-25.0.1\bin && .\kc.bat start-dev --http-port=8180`
- [ ] **Wait 15+ seconds**, then open `http://localhost:8180` to confirm it's up
- [ ] **Start Django** — `python manage.py runserver 0.0.0.0:8000`
- [ ] **Open `http://localhost:8000`** — Confirm the login page loads
- [ ] **Log in as Super Admin** — Verify test data exists
- [ ] **Open Keycloak admin** — `http://localhost:8180` → admin/admin123 → Users tab
- [ ] **Swagger docs** — Open `http://localhost:8000/api/docs/` — confirm it renders
- [ ] **Leave everything running** — Don't close any terminal until the panel starts

30 Minutes Before Panel

- [ ] **Hard refresh browser** (Ctrl+Shift+R) — Clear any cached old CSS
- [ ] **Close all background apps** — Zoom/Teams/notifications off
- [ ] **Maximize the browser** — So the panel can see the full UI
- [ ] **Open 3 tabs** — Tab 1: Your app (logged in as Super Admin). Tab 2: Keycloak admin. Tab 3: Swagger docs
- [ ] **Put your phone on silent** — No buzzes during demo
- [ ] **Deep breath** — You know this system. You built it.

What to Bring to the Panel Table

Item	Why
Laptop (charged)	The main demo machine
Charger	Just in case
Mouse	More precise than trackpad for clicking through UI
Notebook + pen	Write down panel questions you want to address
Water bottle	Dry mouth is real
Your phone	For the mobile app demo (if needed)

What NOT to Do

- ❌ **Don't read from the screen** — Look at the panel, not your laptop
- ❌ **Don't say "I don't know" and stop** — Say "I haven't tested that, but based on the design..." (Part 15 style)
- ❌ **Don't apologise** — Never say "Sorry, this is not perfect." Just say what it does
- ❌ **Don't rush** — Speak at normal pace. Pauses are fine. Silence is better than "ummm"
- ❌ **Don't minimise a broken tab** — Just close it. Don't show them the error

---

PART 28: DEFENSE STRATEGIES — What If Things Go Wrong

What If: The demo freezes or crashes

Stay calm. Say: "*Let me restart that process.*" Close the terminal, restart Django. Keep talking while it loads:

> "*While that restarts — Django's development server reloads in about 5 seconds. This is a development environment, so there is no production-grade load balancer or process manager. In production, systems like Gunicorn and NGINX would handle this automatically with zero downtime.*"

This turns a crash into a demonstration that you understand production deployment.

What If: Keycloak won't start (port already in use)

Say: "*Let me check what is using port 8180.*" Run:

```
netstat -ano | findstr :8180
```

Then `taskkill /PID [number] /F`. If that doesn't work, restart your PC (5 minutes). If asked why:

> "*Keycloak needs port 8180 for its authentication service. If another program grabbed that port, Keycloak cannot start. In production, we run Keycloak on a separate server so this never happens.*"

What If: The panel asks about a feature you don't have

Never say "We didn't have time." Instead:

> "*That is on our roadmap. The current system focuses on the core asset register and security foundation. Features like [their feature] would be the next logical step after deployment. Our architecture supports it because we built with extensibility in mind.*"

What If: The panel asks "So what did YOU actually code?"

This is a trap question — they want to know if you copied code or understood it. Answer with specific code references:

> "*I wrote the Keycloak OIDC backend in authentication/oidc_backend.py — about 90 lines that bridge Keycloak login to Django users. I designed the multi-tenancy setup with django-tenants in tenants/models.py. I built the 5-role access control system with the decorators and permission classes. I*

> *created the audit log system with tamper-proof records. I wrote all the API endpoints for mobile authentication and the Swagger documentation.*"

What If: The panel asks "Why did you choose this technology?"

Use the "Three Reasons" structure — gives a confident, structured answer:

PostgreSQL: "*Three reasons: (1) Schema support — it is the only major database that lets us give each ministry its own private set of tables. (2) Mature and well-documented — the most trusted open-source database for government systems. (3) Django has first-class PostgreSQL support including JSON fields and array fields.*"

Django: "*Three reasons: (1) The ORM makes database queries simple and secure — no raw SQL. (2) django-tenants integrates natively with Django's ORM for multi-tenancy. (3) Django has built-in admin, authentication, and middleware systems that saved us months of development time.*"

Keycloak: "*Three reasons: (1) It is the industry standard for SSO — used by governments and enterprises worldwide. (2) It handles password security, brute-force protection, and session management so we don't have to build those from scratch. (3) It supports OpenID Connect, which lets us authenticate both web browser users and API users with the same identity source.*"

django-tenants: "*Three reasons: (1) It automatically switches PostgreSQL schemas on every request — we do not have to write WHERE ministry_id = X on every query. (2) It provides true data isolation — the database itself prevents cross-ministry access. (3) It integrates with Django's ORM seamlessly — all our existing models work without changes.*"

What If: The panel says "Your system has no tests"

Be honest. Do not make excuses.

> "*You are correct — we have zero automated tests. This is a prototype, not a production system. The focus was on building the complete feature set first: multi-tenancy, SSO, API, audit trail, and all the CRUD operations. In production, we would add test coverage before deployment, starting with the authentication API since that is the most critical path.*"

If they push harder:

> "*For a production government system, I would write tests for: (1) Login and token validation — the most security-critical path. (2) Multi-tenancy isolation — proving a user from MOH cannot see MOF data. (3) Audit log integrity — proving records cannot be modified. In a real deployment, these would be mandatory.*"

What If: The panel asks about performance

> "*The current system ran 0 queries against live data — we have no real data. Based on the database design, each ministry's schema is independent, so query performance only depends on that ministry's data size. Indexes on asset_number, status, and category already exist. For large-scale deployment, I would add database connection pooling with PgBouncer and cache frequently accessed data with Redis.*"

What If: You forget something or freeze

Pause. Take a breath. Say:

> *"Let me think about that for a moment."*

A 5-second pause feels long to you but normal to the audience. Do not fill silence with "ummm" or "ahhh." If you still cannot remember, say:

> *"I know we handle that, but I want to give you an accurate answer. Let me check the code quickly."*

Then open the relevant file and read from it. This shows you know WHERE the answer is, even if you can't recite it.

PART 29: CONFIDENCE TIPS — How to Sound Like You Own This Project

Posture

- Sit up straight, both feet on the floor
- Put your hands on the table — not in your lap, not crossed
- When explaining, use hand gestures (open palms) — it makes you look confident

Eye Contact

- Look at the person who asked the question
- When answering, rotate eye contact across all panel members
- Do NOT stare at your screen while explaining

Voice

- Speak slower than you think you need to — nerves speed you up
- Pause between sentences. A 1-second pause sounds natural
- Drop your pitch at the end of sentences (not up — up sounds like a question)
- The last word of each sentence should be LOUDER, not quieter

The "Confident Stumble" Recovery

If you trip over a word or lose your train of thought:

1. **Stop.**
2. **Smile.**
3. **Say:** *"Let me rephrase that."*
4. **Start the sentence again from the beginning.**

This makes you look thoughtful, not confused. The panel will respect the recovery more than the stumble.

The 3-Bullet Rule

Never give a long rambling answer. Structure everything into 3 points:

> *"There are three reasons for that. First... Second... Third..."*

This works for: technology choices, security features, design decisions, comparison questions. Practise it.

Own Your Weaknesses

The panel WILL find something missing. If you get defensive, you lose. If you own it, you win.

Bad: "*We didn't have time for tests.*"

Good: "*Tests are the most important thing we would add before production. The authentication API needs tests. The multi-tenancy isolation needs tests. The audit log integrity needs tests. I know exactly what to test and how to test it — we just focused on features first.*"

The Panel's Real Questions

The panel is not trying to fail you. They are asking:

They ask	They really mean
"Why this database?"	"Do you understand trade-offs?"
"What about security?"	"Did you think about attacks?"
"How would you scale this?"	"Do you know what production looks like?"
"What did you learn?"	"Can you reflect on your work?"
"What would you change?"	"Are you honest about weaknesses?"

Answer the real question, not the literal question.

Final Pep Talk

You have read this entire guide. You know:

- How every request flows from URL to response
- Why you chose every technology
- How multi-tenancy works at the database level
- How Keycloak SSO works end to end
- What every file in your project does
- How your system connects to all 9 other groups
- What to say when things go wrong

You are the most prepared person in that room.

The panel has not read a 4,700-line guide. They have a rubric. You have deep knowledge. When they ask a question, you will either know the answer or know exactly where to find it.

Now close this file. Stand up. Walk to the mirror. Say the one-paragraph answer from Part 19 out loud. Do it three times. Then sleep.

You have got this.